PROJECT GUIDE

Nexusphere

Verifiable evidence of what AI agents do, for whom and with whose permission, across organizations.

Version 1.0.0 · October 2026

Contents

Part 1 Introduction 4
- Nexusphere at a Glance 5
- Project Description 9

Part 2 Getting Started 13
- Ledger Guide 14
- Local Environment Setup 19

Part 3 Nexusphere Ledger 21
- Nexusphere Ledger 22
- Nexusphere Ledger Chain 65
- Nexusphere Ledger Mandate 66
- Nexusphere Ledger Server 67
- Nexusphere Ledger Verifier 69
- Nexusphere Ledger Demo MCP 71
- Nexusphere Ledger Benchmarks 72
- Nexusphere Ledger Conformance Vectors 75
- Nexusphere Ledger Changelog 77

Part 4 SDKs and Integrations 81
- Nexusphere Ledger Python SDK 82
- Nexusphere Ledger TypeScript SDK 91
- Nexusphere Ledger OpenTelemetry Exporter 101
- Nexusphere Ledger Adapters 105
- Nexusphere Ledger for LangGraph 107
- Nexusphere Ledger for the OpenAI Agents SDK 110
- Nexusphere Ledger for Amazon Bedrock AgentCore 113
- Nexusphere Ledger for agentgateway 116
- Nexusphere Package Verifier for the Browser 118

Part 5 Web UI 122
- Nexusphere Frontend 123

Part 6 Operations 125
- Operations 126
- DevOps Step by Step 133

Part 7 Testing 136
- Nexusphere Ledger End-to-End Tests 137

Part 8 Build and Delivery Toolkit 142

mvn-devops 143
Installation 147
Installing the prerequisites 149
What your Maven project needs 151
GitHub tokens and SSH keys 153
Getting started, step by step 155
Ready-made pipelines 160
Modules and stages 162
Orchestrators 164
Where the tools run 166
Container images and deployment 168
Project site on GitHub Pages 177
Releasing your project 178
Reaching tools on your machine from the internet (ngrok) 179
IDE settings (IntelliJ IDEA) 180
Troubleshooting 181
Design 183
Writing a module 186
Development 189
Changelog 190

PART 1

Introduction

What Nexusphere is, the problem it solves and how it is organized.

1.1 · README.md

Nexusphere at a Glance

Verifiable evidence of what AI agents do, for whom and with whose permission, across organizations.

Documentation

Read the documents in this order. The first three take a newcomer from nothing to a verified evidence package; the rest are references to open when they are needed. All of them are also in one book, Nexusphere Guide (PDF).

Step	Document	What it gives
1	This page	What the ledger is and the quick start
2	Project Description	The problem, the concepts, the architecture and the standards
3	Ledger Guide	Install, register agents, grant, call tools and verify step by step
4	Nexusphere Ledger	Every setting, format and endpoint, and one README per module
5	Nexusphere Frontend	The web UI for operators and principals
6	Operations	Secrets, signing keys, backup, monitoring, alerts and rate limits
7	Local Environment Setup	Developer machine settings for Windows, shells and the IDE
8	DevOps Step by Step	The build pipelines with mvn-devops
9	Ledger End-to-End Tests	What the end-to-end tests cover
10	Conformance Vectors	Test data for another implementation of the formats
11	Python SDK	Record evidence from Python and verify packages and mandates
12	TypeScript SDK	The same for Node and browsers
13	OpenTelemetry Exporter	Evidence from the agent spans an OpenTelemetry Collector receives
14	Adapters	LangGraph, OpenAI Agents SDK, AgentCore and agentgateway
15	Browser Verifier	Verify a package in a browser, from one file, with no network
16	Changelog	What each release contains

Nexusphere Ledger

The Nexusphere Ledger is a self-hosted evidence layer for AI agents: every action an agent takes for a person or an organization becomes an entry that a third party can verify offline, without trusting the operator. It does not replace the agent platforms and gateways that already decide what an agent may do; it sits beside them and keeps the proof. A gateway or agent platform sends its decisions and outcomes to `POST /api/v1/evidence`, and where no gateway exists the ledger's own MCP and A2A gateways decide and record.

Need	Already provided by	What the ledger adds
Agent identity and tool-call decisions	Amazon Bedrock AgentCore, Google Agent Gateway, Microsoft Entra Agent ID, agentgateway	Records their decisions as evidence, or decides itself through its MCP and A2A gateways
Evidence a third party can verify	Logs and audit trails the operator controls	Hash chain, Merkle transparency log with witness cosignatures, SCITT statements and receipts, offline verifier
Proof of authority across organizations	AP2 and Verifiable Intent, for payments only	Grants with principal consent, SD-JWT VC mandates and signed A2A receipts on both sides
Disclosure to an auditor or a counterparty	Exports of raw logs	Evidence packages that disclose only the selected entries and prove the rest of the log

The formats follow open standards so other tools can check them: RFC 9162 Merkle trees, C2SP signed notes and the `tlog-witness` protocol, IETF SCITT signed statements with RFC 9942 receipts, SD-JWT VCs and the IETF Token Status List. Conformance Vectors let another implementation test itself against the ledger.

Quick Start

These steps start the ledger with its database, Keycloak, the web UI and a demo MCP server, run the demo and stop everything again. Every command runs in the root of the repository.

Step 1. Check the tools; each command must print a version:

Tool	Version	Check
Java	21	`java -version`
Maven	3.9	`mvn -version`
Docker	any	`docker compose version`
jq	any	`jq --version`

Windows, shell and IDE settings are in Local Environment Setup.

Step 2. Build the server, the verifier and the demo MCP server:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
```

Step 3. Start the services:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d --build
```

Step 4. Check that the ledger is up; the answer must be `{"status":"UP"}`:

```
curl -X GET http://localhost:8090/actuator/health
```

Step 5. Run the demo; it must end with `Result: VALID` for the package and `Result: INVALID` for the changed copy:

```
ledger/demo-mcp/demo.sh
```

Step 6. Open the web UI on http://localhost:5173, sign in with the operator key `nexusphere-ledger-development-key-change-me`, and look at the evidence the demo recorded. Sign out and sign in again as `alice` with the password `alice` to see her grants.

Step 7. Stop everything and remove the data:

```
docker compose --file ledger/compose.yaml --project-name ledger down
docker volume prune -f
```

Next, the Ledger Guide registers your own agents, grants them tools and connects real agents such as Claude Code.

Build and Test

Step 1. Compile every module:

```
mvn validate clean compile
```

Step 2. Run the unit tests:

```
mvn test
```

Step 3. Run every test, including the integration and end-to-end tests; Docker must be running for Testcontainers:

```
mvn verify
docker volume prune -f
```

Step 4. Build the jars without tests:

```
mvn package -DskipTests=true
```

The web UI is built and tested with npm, as described in Nexusphere Frontend. The SDKs, the exporter, the adapters and the browser verifier use their own tools, as described in Python SDK, TypeScript SDK, OpenTelemetry Exporter, Adapters and Browser Verifier.

Kubernetes

`ledger/kube-dev.yaml` runs the ledger, PostgreSQL, Adminer, Keycloak, the web UI and the demo MCP server in the `dev` namespace. The steps are in Nexusphere Ledger.

DevOps

The project ships mvn-devops 1.0.0 in the `mvn-devops` folder. It starts the chosen tools in Docker, configures them and runs the pipeline stages against this Maven project, either from the menu or from one of its 16 ready-made pipelines. Step-by-step use for this project, from `maven-sonarqube-`

`nexus` up to `jenkins-complete`, is in DevOps Step by Step; its own guide is mvn-devops/README.md. Values and tokens are kept in `.devops/`, which is never committed.

```
mvn-devops/devops.sh doctor
mvn-devops/devops.sh pipelines
mvn-devops/devops.sh setup --pipeline maven-sonarqube-nexus
mvn-devops/devops.sh stages
mvn-devops/devops.sh run
mvn-devops/devops.sh urls
```

Windows:

```
mvn-devops\devops.bat setup
```

Upgrade with `mvn-devops/devops.sh upgrade`, which replaces the folder with the latest release after checking its `SHA256SUMS`, or by deleting the `mvn-devops` folder and extracting the new release zip in its place under the same name.

Service URLs

Service	URL	Sign in
Web UI	http://localhost:5173	Operator key, or `alice` / `alice` and `bob` / `bob`
Ledger API	http://localhost:8090	`Authorization: Bearer {key}`
Health	http://localhost:8090/actuator/health	None
Demo MCP	http://localhost:8091/mcp	None, reached through the ledger at `/mcp/demo`
Keycloak	http://localhost:8180	`admin` / `admin`
Adminer	http://localhost:8092	Server `ledger-postgresql`, user and password `ledger`
Prometheus	http://localhost:9090	None
Alertmanager	http://localhost:9093	None
Vault	http://localhost:8200	Token `nexusphere-vault-development-token`
OTLP	localhost:4317 (gRPC), http://localhost:4318 (HTTP)	None, the Collector sends with the operator key

Repository Layout

Folder	Content
`ledger`	The ledger: chain, mandate, server, verifier, demo MCP, tests, conformance, SDKs, adapters
`frontend`	The web UI in React and TypeScript
`docs`	The documents above and the PDF guide
`mvn-devops`	The build and delivery toolkit

1.2 · docs/README.md

Project Description

The problem the Nexusphere Ledger solves, its concepts, guarantees, architecture and standards.

Problem

AI agents now read files, send email, place orders and talk to agents of other companies on behalf of people. When something goes wrong, three questions come up: what did the agent do, for whom, and who allowed it. Platform logs answer them only for whoever runs the platform, and that party can change its own logs. An auditor, a regulator, the person the agent acted for or a company on the other side of an exchange has to take the operator's word.

The Nexusphere Ledger turns every agent action into evidence that anyone can check without trusting the operator. It is self-hosted and sits beside the agent platforms and gateways an organization already uses, such as Amazon Bedrock AgentCore, Google Agent Gateway, Microsoft Entra Agent ID or agentgateway; it keeps the proof of what they decided. Where nothing decides yet, its own MCP and A2A gateways decide and record.

Concepts

Concept	Meaning
Agent	An identity with its own API key; the program that uses the key is the agent
Principal	The person or organization the agent acts for, such as `alice`
Operator	Whoever runs the ledger and holds its operator key
Grant	What a principal lets an agent do: actions, targets, validity and a use limit
Consent	The principal's approval of a grant with their own sign-in at an OpenID Connect provider
Decision	ALLOW or DENY for one action, with a reason code, followed by the reported outcome
Evidence entry	One record in the hash chain: who, what, for whom, the decision and the outcome
Checkpoint	The chain head signed with the ledger key, so later changes are detectable
Transparency log	A Merkle tree over all entries whose signed checkpoints prove that an entry is in the log
Witness	An independent party that cosigns log checkpoints and refuses a log that rewrites its history
Signed statement	One entry as an IETF SCITT COSE statement, with a receipt that proves it is in the log
Package	Selected entries with the proof that they belong to the signed chain and log, for an auditor or counterparty
Mandate	A signed SD-JWT VC an agent carries to another organization to prove what it may do
Receipt	The signed answer of a receiving ledger in an A2A exchange, so both sides hold matching evidence

Guarantees

ID	Guarantee
G-1	Evidence is append-only: the database refuses any change or deletion, and the hash chain shows any change
G-2	Every checkpoint is signed; a retired key stays published, so everything it signed still verifies
G-3	A package verifies offline with the ledger's public key, without the server
G-4	Witnesses refuse a smaller or forked log, so a ledger cannot hide or rewrite entries they have seen
G-5	A decision is allowed only by an active grant that covers the action and target; the default is DENY
G-6	A grant created by the operator or an agent allows nothing until the principal approves it, when OIDC is on
G-7	Grant, consent, mandate and key changes are evidence too
G-8	A receiving ledger accepts an A2A request only with a valid mandate, and both sides record the exchange
G-9	The ledger refuses to start with the published development secrets outside the `dev` profile

Actors

Actor	Does
Operator	Runs the ledger, registers agents, creates grants, exports packages
Agent	Asks for decisions, reports outcomes, or calls tools and agents through the gateways
Principal	Signs in, approves, denies, creates and revokes grants, and reads the evidence about them
Gateway or platform	Sends the decisions it made and their outcomes to the ledger as evidence
Auditor	Verifies packages, statements and receipts offline
Counterparty ledger	Checks mandates of incoming A2A requests and signs receipts
Witness	Cosigns log checkpoints of the logs it watches

How It Works

1. The operator registers an agent; the agent gets its own API key.
2. A grant names the principal, the agent, the actions, the targets, the validity and the use limit.
3. With principal sign-in on, the principal approves the grant in the web UI.
4. The agent calls a tool through the MCP gateway, or an external gateway decides and sends the decision as evidence.
5. The ledger decides against the grant, forwards only allowed calls, and records the decision and the outcome.
6. Each entry is linked to the previous one by its hash and added to the Merkle log.
7. At a fixed interval the ledger signs the chain head and the log checkpoint, and witnesses cosign the checkpoint.
8. An auditor receives a package of selected entries and verifies it offline with the ledger's public key.
9. For another organization, the agent carries a mandate; the receiving ledger checks it and answers with a receipt.

Architecture

The ledger is a Maven multi-module project in `ledger/`. The formats live in plain Java libraries without framework dependencies, so the verifier and other organizations use exactly the code the server uses. The server is one Spring Boot application on PostgreSQL. The web UI in `frontend/` is a separate single-page application that talks to the server's API.

Module	Responsibility
`ledger/chain`	Evidence format, hash chain, Merkle log, signed notes, COSE statements and receipts
`ledger/mandate`	SD-JWT VC mandates, status lists, JWK keys and the mandate verifier SDK
`ledger/server`	Evidence API, gateways, grants, consent, checkpoints, witnesses and verification
`ledger/verifier`	Offline verifier for packages, statements and mandates, as a library and a jar
`ledger/demo-mcp`	Demo MCP server and the demo script
`ledger/ledger-e2e-tests`	End-to-end tests against the server and PostgreSQL
`ledger/conformance`	Conformance vectors for every format
`ledger/sdk/python`	Python client and offline verifier, on the same conformance vectors
`ledger/sdk/typescript`	TypeScript client and offline verifier for Node and browsers
`ledger/otel-exporter`	OpenTelemetry Collector exporter from agent spans to evidence
`ledger/adapters`	Adapters for LangGraph, OpenAI Agents SDK, AgentCore and agentgateway
`ledger/verifier-web`	Package verifier in the browser, as one HTML file with no network access
`frontend`	Web UI for operators and principals

Stack

- Java 21, Spring Boot 4.1, Maven
- PostgreSQL 18 with the `ledger` schema and Flyway migrations
- Spring JDBC, Spring Security OAuth2 JOSE for principal tokens, Micrometer with Prometheus
- HashiCorp Vault transit as an optional home for the signing key
- React and TypeScript for the web UI and the browser verifier
- Python and TypeScript SDKs that implement every format independently
- Go for the OpenTelemetry Collector exporter
- JUnit 5, AssertJ and Testcontainers for tests; Vitest for the web UI

Server Packages

```
com.nexusphere.ledger
├── evidence      evidence entries, the hash chain, checkpoints and packages
├── transparency  Merkle log, log checkpoints, proofs, witnesses, SCITT statements and receipts
├── agent         agents and their API keys
├── authorization grants, consent, decisions and mandates
├── mcp           MCP gateway
├── a2a           A2A gateway, inbound verification and receipts
└── server        configuration, security, signing keys, rate limits and errors
```

Standards

Standard	Used for
RFC 9162	Merkle tree, inclusion and consistency proofs
C2SP signed-note, tlog-checkpoint	Log checkpoints
C2SP tlog-witness, tlog-cosignature	Witness cosigning
IETF SCITT, RFC 9052 COSE	Signed statements
RFC 9942	Receipts for signed statements
IETF SD-JWT VC	Mandates with selective disclosure
IETF Token Status List	Mandate revocation
RFC 8037, RFC 8032	Ed25519 keys and signatures
OpenID Connect, OAuth 2.0 PKCE	Principal sign-in
MCP, A2A	Gateways for tool calls and agent-to-agent calls
OpenTelemetry GenAI conventions	Evidence from agent spans
AP2, Verifiable Intent	Mapping of mandates to payment intents

Out of Scope

- Running agents or hosting models; the ledger only records and decides their actions
- Replacing agent platforms and gateways; the ledger sits beside them
- A policy language; grants are explicit lists of actions and targets
- Payments; AP2 and other payment protocols carry the money, the ledger keeps the proof
- Certification for a regulation; the ledger provides evidence that audits can use

PART 2

Getting Started

Trying the ledger with agents, and setting up a machine for development.

2.1 · docs/ledger-guide.md

Ledger Guide

Install the Nexusphere Ledger, run agents through it, and present it.

What It Does

An agent is software that acts for a person. The ledger stands between the agent and the tools it uses. Every tool call is checked against a grant the person gave, allowed or denied, and written to a hash chain that nobody can change unnoticed, not even the operator. Anyone can later check what an agent did, for whom and with whose permission, with a package of evidence and a verifier that runs offline.

Word	Meaning
Agent	An identity with its own API key; the program that uses the key is the agent
Principal	The person or organization the agent acts for, such as `alice`
Grant	What a principal lets an agent do: actions, targets, expiry and a use limit
Decision	ALLOW or DENY for one call, with a reason code
Evidence	One entry in the hash chain: who, what, for whom, the decision and the outcome
Checkpoint	The chain head signed with the ledger key, so later changes are detectable
Package	The evidence of one agent or principal with the proof that it belongs to the signed chain
Mandate	A signed token an agent carries to another organization to prove what it may do

Install

Prerequisites

Tool	Version	Check
Java	21	`java -version`
Maven	3.9	`mvn -version`
Docker	any	`docker compose version`
jq	any	`jq --version`

Start

Every command runs in the root of the repository.

Step 1. Build the server, the verifier and the demo MCP server:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
```

Step 2. Start the services:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d --build
```

Step 3. Check the ledger; the answer must be `{"status":"UP"}`:

```
curl http://localhost:8090/actuator/health
```

Service	URL	Sign in
Web UI	http://localhost:5173	Operator key, or `alice` / `alice` and `bob` / `bob`
Ledger API	http://localhost:8090	`Authorization: Bearer {key}`
Demo MCP	http://localhost:8091/mcp	None, reached through the ledger at `/mcp/demo`
Keycloak	http://localhost:8180	`admin` / `admin`
Adminer	http://localhost:8092	Server `ledger-postgresql`, user and password `ledger`
Prometheus	http://localhost:9090	None
Alertmanager	http://localhost:9093	None

The development operator key is `nexusphere-ledger-development-key-change-me`. It only works with the `dev` profile; production keys and secrets are described in Operations.

Run the Demo

Step 4. Run the demo; it must end with `Result: VALID` for the package and `Result: INVALID` for the changed copy:

```
ledger/demo-mcp/demo.sh
```

The script registers an agent, asks `alice` for a grant and approves it with her Keycloak sign-in, calls two allowed tools and one denied tool through the gateway, lists the evidence, exports a package, verifies it offline and shows that a changed entry fails verification.

Agents

These steps use the stack started above. Keep one terminal open for them, since step 1 sets a variable the later steps use.

Register Agents

Step 1. Keep the operator key in a variable and register two agents:

```
OP="Authorization: Bearer nexusphere-ledger-development-key-change-me"
curl -s -X POST http://localhost:8090/api/v1/agents -H "$OP" -H "Content-Type: application/json" -
d '{"agentId":"invoice-agent","name":"Invoice agent","ownerId":"acme"}'
curl -s -X POST http://localhost:8090/api/v1/agents -H "$OP" -H "Content-Type: application/json" -
d '{"agentId":"support-agent","name":"Support agent","ownerId":"acme"}'
```

Each answer carries the agent's `apiKey` once. Keep it; a lost key is replaced with `POST /api/v1/agents/{agentId}/key`. The Agents page of the web UI does the same. An agent that talks to other organizations should also register an Ed25519 key of its own, so a copied API key is not enough to act for it; see Agent Keys and Key Binding.

Grant Permissions

Step 2. Create a grant for each agent:

```
curl -s -X POST http://localhost:8090/api/v1/grants -H "$OP" -H "Content-Type: application/json" -
d '{"principalId":"alice","agentId":"invoice-agent","actions":["tools/call"],"targets":
["demo/read_file","demo/send_email"],"expiresAt":"2026-12-31T00:00:00Z","maxUses":20}'
curl -s -X POST http://localhost:8090/api/v1/grants -H "$OP" -H "Content-Type: application/json" -
d '{"principalId":"alice","agentId":"support-agent","actions":["tools/call"],"targets":
["demo/read_file"],"expiresAt":"2026-12-31T00:00:00Z"}'
```

The compose ledger asks principals for consent, so both grants start `PENDING`. Sign in to the web UI as `alice` and approve them; both must then show `ACTIVE`. Targets are `{server}/{tool}`; a trailing `*` matches a prefix, such as `demo/*`.

Call Tools

Step 3. Call one granted and one not granted tool with the `apiKey` of `invoice-agent`:

```
curl -s http://localhost:8090/mcp/demo -H "Authorization: Bearer {apiKey}" -H "X-Ledger-Principal:
alice" -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"read_file","arguments":
{"path":"/invoices/2026-10.txt"}}}'
curl -s http://localhost:8090/mcp/demo -H "Authorization: Bearer {apiKey}" -H "X-Ledger-Principal:
alice" -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":2,"method":"tools/call","params":{"name":"delete_file","arguments":
{"path":"/invoices/2026-10.txt"}}}'
```

The first call reaches the demo server and returns its answer. The second is denied with JSON-RPC error `-32003` and never reaches the server. `X-Ledger-Principal` names the principal the agent acts for; the call is allowed only when that principal gave the agent a matching grant.

See the Evidence

Step 4. List the evidence of `invoice-agent`; the denied call is there too:

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=invoice-agent" -H "$OP" | jq '.items[] |
{sequence, action, target, decision, outcome}'
```

Step 5. Export a package and verify it offline; the verifier must print `Result: VALID`:

```
curl -s -X POST http://localhost:8090/api/v1/packages -H "$OP" -H "Content-Type: application/json"
-d '{"agentId":"invoice-agent"}' -o package.json
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar --public-key "$(curl -s
http://localhost:8090/api/v1/keys -H "$OP" | jq -r '.[] | select(.status == "ACTIVE") |
.publicKey')" package.json
```

Real Agents and MCP Servers

Claude Code

```
claude mcp add --transport http ledger-demo http://localhost:8090/mcp/demo --header
"Authorization: Bearer {apiKey}" --header "X-Ledger-Principal: alice"
```

Claude Code then lists the demo tools and every call it makes goes through the ledger: granted tools run, others are denied, and both are evidence. Any MCP client that speaks Streamable HTTP and can send headers works the same way, such as Claude Desktop or an agent framework.

Your Own MCP Server

Each upstream MCP server is configured on the ledger by name and served at `/mcp/{name}`. The server must speak Streamable HTTP; a stdio server needs an HTTP bridge in front of it. A key the upstream needs is sent by the ledger, so the agent never sees it. Every request also carries the ledger's RFC 9421 HTTP message signature, so the server can refuse anything that did not come through the ledger; see HTTP Message Signatures.

```
LEDGER_MCP_SERVERS_FILES_URL: http://files-mcp:3000/mcp
LEDGER_MCP_SERVERS_FILES_AUTHORIZATION: Bearer upstream-secret
```

Another Organization

When an agent works with an agent of another organization, both sides run a ledger. The sending ledger decides the call and attaches a mandate, or the agent's own key-bound presentation when it has a signing key, the receiving ledger checks it and answers with a signed receipt, and both keep matching evidence. The A2A gateway and its settings are in Nexusphere Ledger.

Beside an Existing Gateway

When agents already run behind a platform or gateway that decides their tool calls, such as Amazon Bedrock AgentCore, Google Agent Gateway or agentgateway, keep it and let the ledger hold the proof. Register one ledger agent per real agent, then have the gateway, a hook of the platform or the agent itself send every decision and its outcome as evidence with that agent's key. The entries get the same chain, log, witnesses, statements and packages as calls that pass through the ledger's own gateway.

```
curl -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer {agentApiKey}" -H
"Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","target":"send_email","decision":"ALLOW","reaso
n":"agentcore-policy","outcome":"SUCCEEDED","correlationId":"session-42","attributes":
{"gateway":"agentcore"}}'
```

When the platform already sends OpenTelemetry traces, nothing has to call the API: point its OTLP exporter at the Collector of the OpenTelemetry Exporter, or add the `nexusphere` exporter to an existing Collector. Every `execute_tool`, `invoke_agent` and MCP `tools/call` span becomes an evidence entry, with the trace id as its correlation id, and other spans are ignored. Add the model, its version and the hash of the system prompt to each entry with `model` in the evidence, the `X-Ledger-Model` headers of the gateways, or the span attributes `gen_ai.response.model`, `nexusphere.model.version` and `nexusphere.prompt.hash`, so a change of model or prompt shows up in the evidence.

For LangGraph, the OpenAI Agents SDK, Amazon Bedrock AgentCore Gateway and agentgateway, the Adapters connect the ledger in a few lines and can also let it decide each call.

Present It

Step	Show	Point
1	The problem in one sentence	Agents act, but nobody can prove what they did
2	`demo.sh`: two tools allowed, `delete_file` denied	The decision happens before the tool runs
3	The web UI as `alice`: approve, then revoke a grant	The person, not the vendor, decides
4	A package verified in the browser verifier, then one changed entry fails	Nobody can change the record, not even the operator
5	Claude Code calling tools through the ledger	Works with real agents without changing them
6	Two ledgers and a mandate, if the audience works across organizations	Trust between companies, not only inside one
7	The Reports page: a dispute package for one incident	Ready for an insurer, arbitrator or court

Start the stack before the talk and run `demo.sh` once, so images and Keycloak are warm. Keep the web UI open in a second window signed in as `alice`.

Stop

Step 1. Stop the services and remove the data:

```
docker compose --file ledger/compose.yaml --project-name ledger down
docker volume prune -f
```

Next Steps

- Production secrets, backups and monitoring: Operations
- Every setting and endpoint: Nexusphere Ledger
- The rules of your jurisdiction: Compliance Profiles
- The web UI: Nexusphere Frontend

2.2 · docs/local-setup.md

Local Environment Setup

One-time settings on a developer machine for building Nexusphere and running mvn-devops.

Tools

Tool	Version	Check
Java (JDK)	21	`java -version`
Maven	3.9	`mvn -version`
Docker	any	`docker compose version`
Git	any	`git --version`
curl	any	`curl --version`
jq	any	`jq --version`

Install steps per operating system are in mvn-devops/docs/prerequisites.md.

Git Line Endings

The Bash scripts must keep LF line endings. The repository enforces this with `.gitattributes`, but a working tree cloned with `core.autocrlf=true` before that may still hold CRLF files. On Windows, prefer:

```
git config --global core.autocrlf input
```

If `devops.sh doctor` reports files with CRLF line endings, write them again from Git at the project root:

```
rm -rf mvn-devops
git checkout HEAD -- mvn-devops
```

Windows Shell

mvn-devops runs in Bash. On Windows use Git Bash or Cygwin, not WSL and not PowerShell.

Git Bash

Works without extra settings. `mvn-devops\devops.bat` also starts the scripts with Git Bash.

Cygwin

Cygwin must use its own `bash`. When the Windows folders come first in `PATH`, `bash` resolves to `C:\Windows\System32\bash.exe`, which is WSL, and the scripts run inside WSL: `java.exe` and `jq.exe` are not found there, and CRLF files fail with `set: pipefail: invalid option name`.

Check it:

```
command -v bash
```

The answer must be `/usr/bin/bash`. If it is `/cygdrive/c/Windows/system32/bash`, put the Cygwin folders first in `~/.bashrc`. `~` is the Cygwin `$HOME`, which can be the Windows home such as `C:\Users\<user>`; `echo "$HOME"` shows it.

```
echo 'export PATH="/usr/local/bin:/usr/bin:$PATH"' >> ~/.bashrc
```

Open a new terminal and run `command -v bash` again.

The first build of mvn-devops 1.0.0 passed Cygwin paths such as `/cygdrive/c/...` to `docker.exe`, which reads them as `C:\cygdrive\c\...`, so `setup` failed with `couldn't find env file`. The release rebuilt on 2026-10-08 converts these paths with `cygpath`. If `setup` still fails with that message, run the Docker commands from Git Bash.

IntelliJ IDEA Terminal

Settings > Tools > Terminal > Shell path:

Shell	Shell path	Environment variables
Git Bash	`"<git>\bin\bash.exe" --login -i`	
Cygwin	`<cygwin>\bin\bash.exe`	
Cygwin	`<cygwin>\bin\bash.exe --login -i`	`CHERE_INVOKING=1`

`<git>` is the Git for Windows folder, for example `C:\Program Files\Git`, and `<cygwin>` is the Cygwin folder, for example `C:\cygwin64`. A Cygwin login shell moves to the home folder unless `CHERE_INVOKING=1` is set, so the terminal would not start in the project folder. Without `--login`, Cygwin needs the `PATH` line from Cygwin.

Pipeline variables for IntelliJ IDEA run configurations, Checkstyle and coverage are described in mvn-devops/docs/ide.md.

GitHub Access

Tokens and the SSH key used by the pipeline are described in mvn-devops/docs/github-setup.md. With the `maven` orchestrator a classic token with `repo`, `write:packages` and `read:packages` is enough; the SSH key is needed only to publish the site.

Check

```
mvn-devops/devops.sh doctor
```

Every line must be ✔. The SSH key line may stay a warning when the site is not published from this machine.

PART 3

Nexusphere Ledger

The product: the evidence layer for AI agents, its formats, modules and conformance vectors.

3.1 · ledger/README.md

Nexusphere Ledger

A self-hosted, tamper-evident evidence ledger for the actions of AI agents.

Purpose

Every action an agent takes for a person or an organization is recorded as an evidence entry: which agent acted, for which principal, what it did, on what target, whether it was allowed or denied and why, and how it ended. Entries form a SHA-256 hash chain, the database rejects any change or deletion, and the ledger signs its head with Ed25519 at a fixed interval. Anyone holding the published public key can check the chain and the checkpoints without trusting the server.

The ledger is an evidence layer, not another agent platform. Agent identity and per-call decisions are already offered by Amazon Bedrock AgentCore, Google Agent Gateway, Microsoft Entra Agent ID and open gateways such as agentgateway. The ledger keeps them and records what they decide: a gateway, a platform hook or the agent itself sends each decision and outcome to `POST /api/v1/evidence` (or `POST /api/v1/evidence/batch`) with its own agent key. Where nothing decides yet, the ledger's MCP and A2A gateways do it. What the ledger adds is the proof: a transparency log that witnesses cosign, SCITT statements and receipts, evidence packages with selective disclosure, mandates other organizations can check, and receipts both sides of an exchange hold.

Modules

Module	Responsibility
`ledger/chain`	Evidence format, hash chain, Merkle log, signed notes, COSE statements, receipts. No framework dependencies
`ledger/mandate`	SD-JWT VC mandates, status lists, JWK keys and the mandate verifier SDK for other organizations
`ledger/server`	Spring Boot service: evidence API, PostgreSQL storage, checkpoint scheduler, verification
`ledger/verifier`	Command-line verifier for evidence packages and mandates, with no server dependency
`ledger/demo-mcp`	Demo MCP server with file and mail tools, and the demo script
`ledger/benchmarks`	Load scenarios for recording, decisions, the MCP gateway and packages, with the published results
`ledger/ledger-e2e-tests`	End-to-end tests against the server and PostgreSQL
`ledger/sdk/python`	Python client and offline verifier for packages, statements, receipts and mandates
`ledger/sdk/typescript`	TypeScript client and offline verifier for Node and browsers, with no runtime dependencies
`ledger/otel-exporter`	OpenTelemetry Collector exporter that turns agent tool call spans into evidence, and a ready Collector
`ledger/adapters`	Adapters for LangGraph, the OpenAI Agents SDK, Amazon Bedrock AgentCore and agentgateway
`ledger/verifier-web`	Package verifier as one HTML file that runs in the browser with no network

Getting Started

Every command runs in the root of the repository. Each path below is a list of steps; do them in order and check the result of each step before the next one.

Prerequisites

Tool	Version	Check
Java	21	`java -version`
Maven	3.9	`mvn -version`
Docker	any	`docker compose version`
jq	any	`jq --version`

Dockerized

Step 1. Build the server, the verifier and the demo MCP server:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
```

Step 2. Start every service:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d --build
```

Step 3. Check that the ledger is up; the answer must be `{"status":"UP"}`:

```
curl -X GET http://localhost:8090/actuator/health
```

Service	URL	Description
`ledger`	http://localhost:8090	The ledger with the `dev` profile
`demo-mcp`	http://localhost:8091/mcp	Demo MCP server, reached through `/mcp/demo`
`ledger-postgresql`	localhost:5433	PostgreSQL, user, password and database `ledger`
`ledger-adminer`	http://localhost:8092	Adminer for `ledger-postgresql`
`keycloak`	http://localhost:8180	Keycloak, admin `admin` / `admin`
`ledger-ui`	http://localhost:5173	The web UI, built from `frontend`
`vault`	http://localhost:8200	Vault in development mode with a transit key
`otel-collector`	localhost:4317, 4318	OpenTelemetry Collector, OTLP gRPC and HTTP
`prometheus`	http://localhost:9090	Prometheus with the ledger alerts
`alertmanager`	http://localhost:9093	Alertmanager, no channel until one is set

Keycloak imports the `nexusphere` realm from `ledger/keycloak/nexusphere-realm.json` with the public client `nexusphere-ledger` and the principals `alice` and `bob`, whose passwords are their names. The ledger in the compose file trusts this realm, so grants created by the operator wait for the principal's approval.

The web UI on http://localhost:5173 opens with the operator key or a Keycloak sign-in; it is described in Nexusphere Frontend. The MCP addresses have no web page. `/mcp/demo` on the ledger and `/mcp` on `demo-mcp` accept only MCP JSON-RPC `POST` requests, and the gateway also needs an agent API key, so opening them in a browser shows nothing. Run the demo below to see calls go through the gateway.

Step 4. Run the demo; it must end with `Result: VALID` for the package and `Result: INVALID` for the changed copy:

```
ledger/demo-mcp/demo.sh
```

The script registers an agent, asks `alice` to grant it `read_file` and `send_email` on the demo server, signs `alice` in with Keycloak to approve the grant when the ledger asks for consent, calls both through the gateway, has `delete_file` denied, lists the recorded evidence, exports an evidence package, verifies it offline with the ledger's published key and shows that a changed entry fails verification. `LEDGER_URL`, `LEDGER_API_KEY`, `KEYCLOAK_URL`, `VERIFIER_JAR` and `WORK_DIR` override its defaults.

Step 5. Stop everything and remove the data:

```
docker compose --file ledger/compose.yaml --project-name ledger down
docker volume prune -f
```

Kubernetes

`ledger/kube-dev.yaml` runs the same services as the compose file in the `dev` namespace. The Keycloak realm comes from the `keycloak-realm` ConfigMap. Keycloak and the UI are reached on the same local ports as with compose, so the sign-in redirect and the token issuer stay `http://localhost:8180` and `http://localhost:5173`.

Step 1. Build the jars and the images:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
docker compose --file ledger/compose.yaml --project-name ledger build
```

Step 2. Apply the manifest:

```
kubectl apply -f ledger/kube-dev.yaml
```

Step 3. Wait until every pod is `Running`:

```
kubectl get all -n dev
```

Step 4. Forward the ports, each in its own terminal:

```
kubectl port-forward service/ledger 8090:8090 -n dev
kubectl port-forward service/keycloak 8180:8080 -n dev
kubectl port-forward service/ledger-ui 5173:80 -n dev
kubectl port-forward service/ledger-adminer 8092:8080 -n dev
kubectl port-forward service/otel-collector 4318:4318 -n dev
```

Step 5. Check the ledger and run the demo as in Dockerized:

```
curl -X GET http://localhost:8090/actuator/health
ledger/demo-mcp/demo.sh
```

Step 6. Remove everything:

```
kubectl delete -f ledger/kube-dev.yaml
```

Run

This path runs the server from Maven against the PostgreSQL of the compose file, for development.

Step 1. Start PostgreSQL only:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d ledger-postgresql
```

Step 2. Build and start the server:

```
mvn install -DskipTests=true
LEDGER_PROFILES=postgresql,dev LEDGER_DATABASE_PORT=5433 mvn -pl ledger/server spring-boot:start
```

Step 3. Record an entry, sign a checkpoint and verify the chain; the last answer must hold `"valid": true`:

```
curl -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer nexusphere-ledger-
development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","target":"send_email","decision":"ALLOW","outco
me":"SUCCEEDED","attributes":{"server":"mail"}}'
curl -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer nexusphere-ledger-
development-key-change-me"
curl -X GET http://localhost:8090/api/v1/verification -H "Authorization: Bearer nexusphere-ledger-
development-key-change-me"
```

Step 4. Stop the server and PostgreSQL:

```
mvn -pl ledger/server spring-boot:stop
docker compose --file ledger/compose.yaml --project-name ledger down
```

Verify

Step 1. Run the unit, integration and end-to-end tests; Docker must be running for Testcontainers:

```
mvn -pl ledger/ledger-e2e-tests -am verify
```

Release

Ledger releases are tagged `ledger-v{version}` and described in the Changelog. The release artifacts are the server jar (`ledger/server/target/server-*-exec.jar`), the verifier jar (`ledger/verifier/target/verifier-*-exec.jar`) and the two Docker images built by the compose file.

Environment Variables

```
LEDGER_HOST: 0.0.0.0
LEDGER_PORT: 8090
LEDGER_PROFILES: postgresql
LEDGER_DATABASE_HOST: localhost
LEDGER_DATABASE_PORT: 5432
LEDGER_DATABASE_DB: ledger
LEDGER_DATABASE_USERNAME: ledger
LEDGER_DATABASE_PASSWORD: ledger
LEDGER_API_KEY:
LEDGER_SIGNING_PROVIDER: local
LEDGER_SIGNING_PRIVATE_KEY:
LEDGER_SIGNING_PUBLIC_KEY:
LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY:
LEDGER_SIGNING_UNENDORSED_ROTATION: false
LEDGER_SIGNING_VAULT_ADDRESS:
LEDGER_SIGNING_VAULT_TOKEN:
LEDGER_SIGNING_VAULT_TOKEN_FILE:
LEDGER_SIGNING_VAULT_NAMESPACE:
LEDGER_SIGNING_VAULT_MOUNT: transit
LEDGER_SIGNING_VAULT_KEY: nexusphere-ledger
LEDGER_SIGNING_VAULT_TIMEOUT: 10s
LEDGER_CHECKPOINT_INTERVAL: 1m
LEDGER_COMPLIANCE_PROFILES: baseline
LEDGER_COMPLIANCE_PROFILE_DIRECTORY:
LEDGER_COMPLIANCE_REGION:
LEDGER_COMPLIANCE_RETENTION_INTERVAL: 1h
LEDGER_TIMESTAMP_URL:
LEDGER_TIMESTAMP_CERTIFICATES:
LEDGER_TIMESTAMP_TIMEOUT: 10s
LEDGER_REKOR_URL:
LEDGER_REKOR_KEY:
LEDGER_REKOR_TIMEOUT: 10s
LEDGER_LOG_ORIGIN:
LEDGER_LOG_WITNESS_TIMEOUT: 10s
LEDGER_LOG_WITNESSES_{NAME}_URL:
LEDGER_LOG_WITNESSES_{NAME}_KEY:
LEDGER_LOG_WATCHED_{NAME}_KEY:
LEDGER_MCP_TIMEOUT: 60s
LEDGER_MCP_SERVERS_{NAME}_URL:
LEDGER_MCP_SERVERS_{NAME}_AUTHORIZATION:
LEDGER_MANDATE_ISSUER: http://localhost:8090
LEDGER_MANDATE_STATUS_LIST_TTL: 5m
LEDGER_A2A_TIMEOUT: 60s
LEDGER_A2A_REQUEST_MAX_AGE: 5m
LEDGER_A2A_STATUS_LIST_CACHE: 30s
LEDGER_A2A_TRUSTED_ISSUERS:
LEDGER_A2A_PEERS_{NAME}_URL:
LEDGER_A2A_PEERS_{NAME}_ISSUER:
LEDGER_A2A_AGENTS_{NAME}_URL:
LEDGER_A2A_AGENTS_{NAME}_AUTHORIZATION:
LEDGER_STREAM_TIMEOUT: 1h
LEDGER_OIDC_ISSUER:
LEDGER_OIDC_AUDIENCE:
LEDGER_OIDC_JWKS_URI:
LEDGER_OIDC_PRINCIPAL_CLAIM: sub
LEDGER_OIDC_CLIENT_ID:
LEDGER_SECRETS_DIR: /run/secrets/
LEDGER_MANAGEMENT_PORT:
LEDGER_RATE_LIMIT_PER_MINUTE: 1200
LEDGER_RATE_LIMIT_BURST: 200
LEDGER_POLICY_DEFAULT_EFFECT: permit
```

```
LEDGER_APPROVAL_TIMEOUT: 15m
LEDGER_APPROVAL_POLL_INTERVAL: 5s
LEDGER_APPROVAL_WEBHOOK_URL:
LEDGER_APPROVAL_CIBA_ENDPOINT:
LEDGER_APPROVAL_CIBA_TOKEN_ENDPOINT:
LEDGER_APPROVAL_CIBA_CLIENT_ID:
LEDGER_APPROVAL_CIBA_CLIENT_SECRET:
LEDGER_APPROVAL_CIBA_SCOPE: openid
LEDGER_INSTANCE_ID:
LEDGER_CLUSTER_HEARTBEAT: 5s
LEDGER_ANOMALY_ENABLED: true
LEDGER_ANOMALY_INTERVAL: 5m
LEDGER_ANOMALY_WINDOW: 1h
LEDGER_ANOMALY_BASELINE: 7d
LEDGER_ANOMALY_MIN_HISTORY: 1d
LEDGER_ANOMALY_THRESHOLD: 3
LEDGER_ANOMALY_MIN_EVENTS: 10
```

`LEDGER_API_KEY` and the signing keys have no default outside the `dev` profile, and the ledger refuses to start with the published development secrets unless `dev` is active. The private key is a base64 PKCS#8 Ed25519 key and the public key is its base64 X.509 encoding; the ledger refuses to start when they do not match. With `LEDGER_SIGNING_PROVIDER=vault` the private key stays in a HashiCorp Vault transit key and the `LEDGER_SIGNING_VAULT_*` settings replace both keys, as in Keys in Vault. The tables live in the `ledger` schema, so the ledger can share a database with other applications. Any setting can also come from a file of the same name in `LEDGER_SECRETS_DIR`, which is how a secret manager hands over keys; backups, keys and metrics are described in Operations. Each caller, by key or by address when it has none, gets `LEDGER_RATE_LIMIT_PER_MINUTE` requests a minute with bursts up to `LEDGER_RATE_LIMIT_BURST`; over it the answer is 429 `RATE_LIMIT_EXCEEDED` with `Retry-After`, and `0` turns the limit off.

```
openssl genpkey -algorithm ed25519 -outform DER -out ledger-signing.der
export LEDGER_SIGNING_PRIVATE_KEY=$(base64 -w0 ledger-signing.der)
export LEDGER_SIGNING_PUBLIC_KEY=$(openssl pkey -inform DER -in ledger-signing.der -pubout -
outform DER | base64 -w0)
```

High Availability

Several ledger instances can share one PostgreSQL. Every write takes its lock in the database, such as the chain head for evidence and checkpoints, so requests can go to any instance and the chain stays one. The scheduled work (checkpoints, timestamps, Rekor anchors, witness cosignatures, retention, approval notifications and anomaly detection) runs on one instance only, the leader: the instance that holds the PostgreSQL session advisory lock of the ledger on a connection of its own. When the leader stops or loses its connection, PostgreSQL releases the lock and another instance takes it at its next heartbeat, every `LEDGER_CLUSTER_HEARTBEAT`. Each instance is named by `LEDGER_INSTANCE_ID`, by default its host name with a random suffix, and reports `ledger_leader` as 1 or 0. `GET /api/v1/cluster` shows the instance that answers, whether it leads, and every instance seen in the last three heartbeats. Rate limits count per instance.

Step 1. Start PostgreSQL as in Run and build the server jar:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d ledger-postgresql
mvn install -DskipTests=true
```

Step 2. Start two instances on ports 8090 and 8094:

```
for port in 8090 8094; do
  LEDGER_PORT=$port LEDGER_INSTANCE_ID=ledger-$port LEDGER_CLUSTER_HEARTBEAT=1s
LEDGER_PROFILES=postgresql,dev LEDGER_DATABASE_PORT=5433 \
    java -jar ledger/server/target/server-1.0.0-SNAPSHOT-exec.jar > ledger-$port.log 2>&1 &
done
```

Step 3. Ask both; exactly one must answer `"leader":true`, and both must list `ledger-8090` and `ledger-8094`:

```
for port in 8090 8094; do curl -s http://localhost:$port/api/v1/cluster -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{instanceId, leader, members: [.members[].instanceId]}'; done
```

Step 4. Record one entry through each instance; the sequences must follow each other:

```
for port in 8090 8094; do curl -s -X POST http://localhost:$port/api/v1/evidence -H
"Authorization: Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d
'{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","target":"send_email","decision":"ALLOW","outco
me":"SUCCEEDED"}' | jq .sequence; done
```

Step 5. Stop the leader; within a few seconds the other instance must answer `"leader":true`:

```
LEADER=$(for port in 8090 8094; do curl -s http://localhost:$port/api/v1/cluster -H
"Authorization: Bearer $LEDGER_API_KEY" | jq -r "select(.leader) | \"$port\""; done)
kill $(lsof -t -i :$LEADER -sTCP:LISTEN)
sleep 5
OTHER=$([ "$LEADER" = 8090 ] && echo 8094 || echo 8090)
curl -s http://localhost:$OTHER/api/v1/cluster -H "Authorization: Bearer $LEDGER_API_KEY" | jq -c
'{instanceId, leader}'
```

Evidence Format

Field	Description
`format`	`nexusphere-ledger/evidence/v2`
`id`	UUID assigned by the ledger
`sequence`	Position in the chain, starting at 1 without gaps
`occurredAt`	When the action happened, at most five minutes in the future
`recordedAt`	When the ledger recorded it
`agentId`	The acting agent (required)
`principalId`	The person or organization the agent acted for (required, personal)
`action`	What was done, for example `tools/call` (required)
`target`	The tool, resource or counterparty (personal)
`decision`	`ALLOW` or `DENY`
`reason`	Why the decision was taken (personal)
`delegationId`	The grant or mandate the agent acted under
`inputHash`	Lowercase SHA-256 of the input

Field	Description
`outputHash`	Lowercase SHA-256 of the output
`outcome`	`SUCCEEDED`, `FAILED`, `DENIED` or `PENDING` (required); `DENY` requires `DENIED`, `PENDING` requires `ALLOW`
`model`	The model the agent ran: `name`, `version`, `promptHash` and `configHash`, each optional
`correlationId`	Caller correlation, for example a conversation (personal)
`attributes`	Up to 32 string attributes (personal)
`salts`	A random 16-byte base64url salt for each personal field; `null` once erased
`commitments`	The commitment of each personal field
`erased`	`true` when the personal fields were erased
`previousHash`	Hash of the previous entry; 64 zeros for the first entry
`hash`	SHA-256 of the canonical JSON of `format`, `sequence`, `previousHash` and `contentHash`

The personal fields are `principalId`, `target`, `reason`, `correlationId` and each attribute as `attributes.{name}`. The hash never covers their values, only their commitments: the SHA-256 of the canonical JSON of `field`, `salt` and `value`, where an absent value is `null`. The content hash is the SHA-256 of the canonical JSON of `format`, `id`, `occurredAt`, `recordedAt`, `agentId`, `action`, `decision`, `delegationId`, `inputHash`, `outputHash`, `outcome`, `model` when the entry names one, and `commitments`. Whoever holds an entry with its salts recomputes the commitments and the hash; once the salts and values are erased, the stored commitments still prove the entry's place in the chain. The link (`sequence`, `previousHash`, `contentHash`, `hash`) can be checked without the content, so a package can keep entries private and still prove the chain. Canonical JSON has sorted keys, no whitespace and `null` for absent fields. Timestamps are UTC with microseconds, for example `2026-10-01T08:00:00.123456Z`. A checkpoint signs the canonical JSON of `format` (`nexusphere-ledger/checkpoint/v2`), `sequence`, `headHash`, `createdAt`, `keyId` and `profiles`, the `id` and `digest` of each active compliance profile sorted by `id`. Checkpoints signed before the ledger had profiles keep `nexusphere-ledger/checkpoint/v1` without `profiles`, and still verify. The key ID is the first 16 hex characters of the SHA-256 of the X.509 public key.

The ledger keeps the personal values and salts of each entry encrypted with AES-256-GCM under a data key of its principal, in `evidence_personal` and `principal_key`; `evidence_record` holds only the commitments and a random reference to the principal. Format v2 replaced v1 before release 1.0.0, and a ledger database that still holds v1 evidence does not migrate: recreate it.

Verification

`GET /api/v1/verification` recomputes every hash and link, checks the signature of every checkpoint against the entry it names and checks the head. The same checks run outside the server with `ledger/chain`: page through `GET /api/v1/evidence`, feed the entries to `ChainVerifier`, and verify each checkpoint with `SignedCheckpoint.verify` and the key it names from `GET /api/v1/keys`.

Transparency Log and Witnesses

Besides the hash chain, every entry is a leaf of an RFC 9162 Merkle tree: the leaf is the 32-byte entry hash, so leaf `n - 1` is the entry with sequence `n`. Whenever the ledger signs a checkpoint it also signs a log checkpoint in the C2SP signed-note format (`origin`, tree size and base64 root hash, then an Ed25519 signature line). The origin is `LEDGER_LOG_ORIGIN`, or the mandate issuer without its scheme, for example `localhost:8090`. With the tree, anyone can prove that one entry is in the log with about `log2(n)` hashes, and that a later checkpoint extends an earlier one without rewriting it.

A ledger alone can still show different histories to different people. Witnesses close that gap: each witness keeps the last checkpoint it saw for a log and cosigns a new one only with a valid consistency proof from it. Witnesses follow the C2SP `tlog-witness` protocol and sign `tlog-cosignature/v1` lines, so any compatible witness works, and every ledger is one: set `LEDGER_LOG_WATCHED_{NAME}_KEY` to the verifier key of each log it should witness, with `{NAME}` equal to the log's origin. To ask witnesses for cosignatures, set `LEDGER_LOG_WITNESSES_{NAME}_URL` to the witness's base URL (the ledger appends `/add-checkpoint`) and `LEDGER_LOG_WITNESSES_{NAME}_KEY` to its verifier key. The ledger asks every witness after each checkpoint and keeps each cosignature it gets; a witness that is down is asked again next time.

```
curl http://localhost:8090/public/v1/log/key
curl http://localhost:8090/public/v1/witness/key
curl http://localhost:8090/public/v1/log/checkpoint
curl "http://localhost:8090/api/v1/log/proofs/inclusion?sequence=7" -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me"
curl "http://localhost:8090/api/v1/log/proofs/consistency?firstSize=7&secondSize=42" -H
"Authorization: Bearer nexusphere-ledger-development-key-change-me"
```

```
localhost:8090
42
0dpbGc4h7JcvSy6dXq0XpLM4pN0iA0p9cX0XQ4nHqEo=

— localhost:8090 b4Ex7xF0kZ0G1Xo9ZfPu0eQyD3l8Q3mJ2tV9...
— partner.example b8Gk2wAAAABnBl0w0Tq9m3Q1...
```

`GET /api/v1/verification` also rebuilds the tree from the chain and checks every log checkpoint against it, so a log checkpoint that does not match the evidence is reported like a broken link.

Log Archives

The log is also served as C2SP `tlog-tiles`, without an API key, so anyone can keep a full copy of it. The base URL is `/public/v1/log/`: `checkpoint` is the latest signed note, `tile/{L}/{N}` holds 256 hashes of tree level `8L` and `tile/entries/{N}` holds 256 entries, each a two-byte length and the 32-byte entry hash. `N` is written in groups of three digits, such as `x001/x234/067`, and the last tile of a level is partial, `.p/{W}` with its width `W`. A tile never changes once served, so every tile carries `Cache-Control: immutable` and can sit behind any CDN or static mirror. Since the leaves are entry hashes, the archive discloses no evidence content.

The verifier's `archive` command mirrors the log into a directory. It fetches the checkpoint, then only the tiles it does not hold yet, and removes partial tiles a later size replaced, so running it on a schedule keeps an independent copy current. The directory has the same layout as the endpoints, so copying it to any static host or object store, for example with `aws s3 sync ledger-log`

`s3://bucket/log`, publishes a mirror that `archive` and any C2SP client read like the ledger itself. The `tiles` command checks the copy offline: the checkpoint signature against the pinned key, every entry against its leaf, every upper tile against the leaves, and the root against the checkpoint. With `--package`, it also checks that every link of an evidence package is the entry the archive holds at that sequence, so a ledger that later rewrote its history is caught with no access to it. Both exit with `0`, `1` or `2` as the package verifier does.

Step 1. Start the ledger as in Run, then record an entry and sign a checkpoint:

```
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","target":"send_email","decision":"ALLOW","outco
me":"SUCCEEDED"}' | jq .sequence
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq .sequence
```

Step 2. Fetch the level 0 tile of the checkpoint's tree size without an API key; it must answer `200` with `Cache-Control: max-age=31536000, public, immutable`:

```
SIZE=$(curl -s http://localhost:8090/public/v1/log/checkpoint | sed -n 2p)
curl -s -D - -o /dev/null http://localhost:8090/public/v1/log/tile/0/000.p/$SIZE | grep -i
"HTTP/\|cache-control"
```

Step 3. Build the verifier and mirror the log:

```
mvn -pl ledger/verifier -am package -DskipTests=true
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar archive
http://localhost:8090/public/v1/log ledger-log
```

```
Nexusphere Ledger log archive
  Origin      : localhost:8090
  Tree size   : 3
  Fetched     : 2 tiles
  Kept        : 0 tiles already archived
  Removed     : 0 superseded partial tiles
```

Step 4. Export a package and check it against the copy; the result must be `VALID` and the exit code `0`:

```
KEY="$(curl -s http://localhost:8090/api/v1/keys -H "Authorization: Bearer $LEDGER_API_KEY" | jq -
r '.[] | select(.status == "ACTIVE") | .publicKey')"
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' -o package.json
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar tiles --public-key "$KEY" --
package package.json ledger-log; echo "exit $?"
```

```
Nexusphere Ledger log archive verification
  Result      : VALID
  Origin      : localhost:8090
  Tree size   : 3
  Root hash   : 7s86Ob1qgo0Yb7qHu+MaJ9g7aRbBSnbVlZqqkzKbclA=
  Tiles       : 2
  Entries     : 3
  Package     : 3 links found in the archive, 0 beyond it
exit 0
```

Step 5. Change one byte of the copy; the result must be `INVALID` and the exit code `1`:

```
printf '\x00' | dd of=ledger-log/tile/entries/000.p/$SIZE bs=1 seek=10 conv=notrunc
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar tiles --public-key "$KEY"
ledger-log; echo "exit $?"
```

```
Nexusphere Ledger log archive verification
  Result      : INVALID
  Origin      : localhost:8090
  Tree size   : 3
  Root hash   : 7s86Ob1qgo0Yb7qHu+MaJ9g7aRbBSnbVlZqqkzKbclA=
  Tiles       : 2
  Entries     : 3
  Problem     : Entry 1 does not hash to its leaf in tile/0
exit 1
```

SCITT Statements and Receipts

Every entry is also available as an IETF SCITT signed statement and a log receipt, both COSE_Sign1 (`application/cose`) signed with EdDSA by the ledger key. The statement is a COSE hash envelope: its payload is the 32-byte content hash (`payload_hash_alg` SHA-256, preimage content type `application/vnd.nexusphere.evidence+json`), its CWT claims name the mandate issuer as `iss` and `urn:uuid:{id}` as `sub` , and the protected headers `nexusphere-sequence` and `nexusphere-previous-hash` place it in the chain, so the Merkle leaf can be computed from the statement alone. The receipt follows RFC 9942 with the RFC 9162 tree algorithm: the inclusion proof is in the unprotected header (396) and the signed payload is the tree root, which is left out of the message. A receipt is issued for a log checkpoint, the latest by default or the one named by `treeSize` , and the ledger signs a new checkpoint first when the entry is not in the latest one yet.

Only the content hash leaves the ledger in a statement, so a statement can be shared without the entry. Whoever has the entry checks it against the statement's payload. The formats are standard COSE and any COSE library can read them, but the evidence fields are Nexusphere's own; they are not an IETF agent-action profile. A2A receipts stay JWS and name the evidence sequence and hash of the exchange, whose statement and receipt the receiving ledger serves like any other.

```
curl http://localhost:8090/api/v1/evidence/{id}/statement -H "Authorization: Bearer nexusphere-
ledger-development-key-change-me" -o statement.cose
curl http://localhost:8090/api/v1/evidence/{id}/receipt -H "Authorization: Bearer nexusphere-
ledger-development-key-change-me" -o receipt.cose
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar statement --public-key "$(curl -
s http://localhost:8090/api/v1/keys -H 'Authorization: Bearer nexusphere-ledger-development-key-
change-me' | jq -r '.[] | select(.status == "ACTIVE") | .publicKey')" --receipt receipt.cose
statement.cose
```

```
Nexusphere Ledger evidence statement
  Issuer      : http://localhost:8090
  Subject     : urn:uuid:7c0e5a52-8f0b-4d1e-9a51-3c1f0b8e2d44
  Sequence    : 7
  Content hash: 5e1c...
  Entry hash  : 9a40...
  Signature   : valid, key 3f2a9c0d41b7e865
  Receipt     : valid, log localhost:8090 at 42 entries, root
0dpbGc4h7JcvSy6dXq0XpLM4pN0iA0p9cX0XQ4nHqEo=
Result: VALID
```

Evidence Packages

`POST /api/v1/packages` exports an evidence package for an auditor, a court or a counterparty. The request selects evidence by `agentId`, `principalId`, `fromSequence` and `toSequence`. The package (`nexusphere-ledger/package/v1`) holds the public keys with their rotation records, an anchor checkpoint (the last checkpoint before the first selected entry, or genesis), an end checkpoint (the first checkpoint at or after the last selected entry, created at the head when none exists yet) and every link between them. Only selected links carry their full entry; all others carry only their link, so nothing outside the selection is disclosed. A package holds at most 200,000 links. Its `log` field carries the log checkpoint at the end checkpoint's sequence with every cosignature the ledger holds, and for each disclosed entry an inclusion proof, its SCITT statement and its receipt for that checkpoint.

The verifier needs no server and no database. It checks the signatures of both checkpoints, recomputes every link from the anchor to the end checkpoint, checks every disclosed entry against its content hash and the scope of the package, and exits with `0` for a valid package, `1` for an invalid one and `2` for a usage error. Pass the ledger's public key from `GET /api/v1/keys` (or a key published elsewhere) with `--public-key` so the verifier does not trust the key inside the package. When the package has a `log`, the verifier also checks that the log checkpoint is signed by the ledger, covers the end checkpoint and proves every disclosed entry, and that each statement and receipt is signed by a trusted key, describes its entry and proves it in the same log checkpoint. Pass each witness's verifier key with `--witness` and the number of cosignatures needed with `--witnesses-required` (all given witnesses by default) to require that independent witnesses saw the same log. Pass the ledger's current key list from `GET /api/v1/keys` with `--keys` so the verifier also applies key revocations that happened after the package was exported, as in Key Revocation.

```
curl -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer nexusphere-ledger-
development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' -o
package.json
mvn -pl ledger/verifier -am package -DskipTests=true
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar --public-key "$(curl -s
http://localhost:8090/api/v1/keys -H 'Authorization: Bearer nexusphere-ledger-development-key-
change-me' | jq -r '.[] | select(.status == "ACTIVE") | .publicKey')" package.json
```

```
Nexusphere Ledger evidence package
  Signing key : 3f2a9c0d41b7e865 (pinned)
  Checkpoint  : sequence 42, signed at 2026-10-07T12:00:00.000123Z
  Anchor      : checkpoint 40
  Chain       : 2 links from sequence 41
  Disclosed   : 1 entries, agent invoice-agent
  Log         : localhost:8090, 42 entries, 1 disclosed entries proven
  Receipts    : 1 SCITT statements with receipts
  Witnesses   : partner.example
Result: VALID
```

Index Proofs

A package shows what happened; an index proof shows that something did not. The ledger keeps an index of the evidence in a sparse Merkle tree keyed by four kinds of term: an `agent` by `agentId`, an `action` by `agentId` and `action`, an `input` by `inputHash`, and a `grant` by `delegationId`, counting only the actions the grant allowed. Each leaf holds how many entries

name the term and the first and last sequence among them. For a checkpoint the ledger signs the root of the index with its key in the format `nexusphere-ledger/index/v1`, over `sequence`, `headHash`, `root`, `createdAt` and `keyId`, once, the first time someone asks, and keeps that signed root. `POST /api/v1/index/proofs` takes a `term` and an optional checkpoint `sequence`, by default the latest, and answers a proof that the term occurs with its count, or that no entry up to that checkpoint names it, with the path to the signed root and the ledger's keys. `nexusphere-ledger-verify index` checks it offline, and with `--package` that it belongs to the checkpoint of a package. The index uses no personal field, so a proof never discloses a principal, a target or a reason.

The root is a statement signed by the ledger, not a proof computed by the verifier: a false absence would be a signed statement the ledger cannot take back, which an auditor with the full evidence can expose by recomputing the index.

Step 1. Start the ledger as in Run, then record an action and sign a checkpoint:

```
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"payments-
agent","principalId":"alice","action":"tools/call","target":"bank/pay_invoice","inputHash":"9f86d0
81884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08","decision":"ALLOW","outcome":"SUCCEEDE
D"}' | jq -c '{sequence}'
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{sequence}'
```

Step 2. Ask for a proof that the payment input was recorded; the answer must show a `count` of 1:

```
curl -s -X POST http://localhost:8090/api/v1/index/proofs -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"term":
{"type":"input","inputHash":"9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08"}}'
| jq -c .leaf
```

Step 3. Ask for a proof that the agent never deleted anything; the answer must show `null`:

```
curl -s -X POST http://localhost:8090/api/v1/index/proofs -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"term":
{"type":"action","agentId":"payments-agent","action":"files/delete"}}' > proof.json
jq -c .leaf proof.json
```

Step 4. Verify the proof with the ledger's key pinned; the output must show `Finding : no entry up to sequence` and `Result: VALID`:

```
LEDGER_PUBLIC_KEY=$(curl -s http://localhost:8090/api/v1/keys -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -r '.[] | select(.status == "ACTIVE") | .publicKey')
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar index --public-key "$LEDGER_PUBLIC_KEY"
proof.json
```

Key Rotation

The ledger keeps every signing key it has used. Checkpoints, mandates, status lists and A2A receipts are signed with the active key, and everything signed with a retired key still verifies, because each signature names its key and the retired keys stay published in `GET /api/v1/keys`, `GET /public/v1/keys` and every evidence package. A revoked key stays in `GET /api/v1/keys` and in packages, with its revocation, but leaves `GET /public/v1/keys`.

To rotate, start the ledger with a new key pair in `LEDGER_SIGNING_PRIVATE_KEY` and `LEDGER_SIGNING_PUBLIC_KEY` and the old private key in `LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY`. At startup the ledger retires the old key and stores a rotation record (`nexusphere-ledger/key-rotation/v1`) that both keys sign: the old key endorses the new one and the new key names the old one as its predecessor. A verifier that pinned either key reaches the other through this record. The rotation is recorded as a `key/rotate` evidence entry (the first key as `key/activate`), and the previous key can be removed from the settings after the restart. Every instance has to restart with the new key.

When the old private key is lost, set `LEDGER_SIGNING_UNENDORSED_ROTATION` to `true` instead. The rotation record then carries only the new key's signature, so a verifier that pinned the old key does not trust the new key, while one that pins the new key still trusts the old checkpoints. A retired key can never become active again, and the ledger refuses to start when the configured key changes without one of these two settings.

```
openssl genpkey -algorithm ed25519 -outform DER -out ledger-signing-2.der
export LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY=${LEDGER_SIGNING_PRIVATE_KEY}
export LEDGER_SIGNING_PRIVATE_KEY=$(base64 -w0 ledger-signing-2.der)
export LEDGER_SIGNING_PUBLIC_KEY=$(openssl pkey -inform DER -in ledger-signing-2.der -pubout -
outform DER | base64 -w0)
curl http://localhost:8090/api/v1/keys -H "Authorization: Bearer nexusphere-ledger-development-
key-change-me"
```

Keys in Vault

With `LEDGER_SIGNING_PROVIDER=vault` the ledger never holds its private key. It reads the public keys of every version of an Ed25519 key in the Vault transit engine, signs with the latest version through `POST /v1/{mount}/sign/{key}`, and checks at startup that Vault's signature matches the published key. The token comes from `LEDGER_SIGNING_VAULT_TOKEN`, or from `LEDGER_SIGNING_VAULT_TOKEN_FILE`, which is read again on every call so the Vault agent can renew it. A rotation in Vault (`vault write -f transit/keys/{key}/rotate`) becomes a ledger rotation at the next restart, endorsed by the previous version, which Vault still holds. Moving from a local key to Vault is endorsed when `LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY` holds the local key for that one restart.

Step 1. Start the stack with Vault signing; the compose file runs Vault in development mode and creates the key:

```
LEDGER_SIGNING_PROVIDER=vault docker compose --file ledger/compose.yaml --project-name ledger up -
d --build
```

Step 2. Check the keys; the last one must be `ACTIVE` with a rotation endorsed by the development key:

```
curl -s http://localhost:8090/api/v1/keys -H "Authorization: Bearer nexusphere-ledger-development-
key-change-me" | jq '.[] | {keyId, status, previous: .rotation.previousKeyId}'
```

Step 3. Rotate the key in Vault and restart the ledger; one more key must appear, endorsed by the one before:

```
docker exec -e VAULT_TOKEN=nexusphere-vault-development-token vault vault write -f
transit/keys/nexusphere-ledger/rotate
docker compose --file ledger/compose.yaml --project-name ledger restart ledger
```

A retired key never signs again, so going back to `local` after this needs a new local key and its own rotation. The Vault policy of the ledger's token needs only `read` on `{mount}/keys/{key}` and `update` on `{mount}/sign/{key}`.

Key Revocation

When a retired key leaks, the operator revokes it with the time from which it must be treated as compromised. The active key signs a revocation record (`nexusphere-ledger/key-revocation/v1`) with the key ID, `compromisedAt`, `revokedAt`, the reason and the revoker's key ID; the ledger records it as a `key/revoke` evidence entry and shows the key as `REVOKED`. The active key cannot be revoked: rotate first, so another key can sign the revocation. A revoked key leaves the public JWKS, so mandates it signed no longer verify.

A signature carries no trustworthy time of its own, since whoever holds the leaked key can write any time into it. A verifier therefore accepts a checkpoint, log checkpoint, statement or receipt of a revoked key only when witness cosignatures prove that the log checkpoint existed before `compromisedAt`: at least one cosignature, or `--witnesses-required` of them, with an earlier time. A rotation endorsed by a revoked key at or after `compromisedAt` gives no trust either, so pin the key that signed the revocation. A revocation counts only when a key the verifier trusts signed it.

Step 1. Rotate to a new key, as above, if the leaked key is still active.

Step 2. Revoke the leaked key; `compromisedAt` is the earliest time the key may have been copied:

```
curl -s -X POST http://localhost:8090/api/v1/keys/{keyId}/revocation -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d
'{"compromisedAt":"2026-10-08T00:00:00Z","reason":"backup copy leaked"}'
```

Step 3. Check the keys; the leaked key must show `REVOKED` with its revocation:

```
curl -s http://localhost:8090/api/v1/keys -H "Authorization: Bearer nexusphere-ledger-development-
key-change-me" | tee keys.json | jq '.[] | {keyId, status, compromisedAt:
.revocation.compromisedAt}'
```

Step 4. Verify an older package with the new key pinned, the key list and the witnesses; it must end with `Result: VALID` and name the revoked key:

```
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar --public-key "$(jq -r '.[] |
select(.status == "ACTIVE") | .publicKey' keys.json)" --keys keys.json --witness "{witness
verifier key}" package.json
```

Agents, Grants and Decisions

The operator holds `LEDGER_API_KEY`. Every agent is registered by the operator and gets its own API key, shown only once and stored as a SHA-256 hash. An agent can record and read only its own evidence, grants and decisions; registering agents, creating and revoking grants, creating checkpoints and running the full verification belong to the operator.

A grant lets one agent act for one principal: a list of actions and a list of targets, each exact or ending with `*`, valid until `expiresAt`, optionally from `notBefore` and for at most `maxUses` allowed decisions. Its terms are hashed as the canonical JSON of `nexusphere-ledger/grant/v1` (`GrantTerms` in `ledger/chain`), and the hash is the `inputHash` of the `grant/create` evidence, so anyone can check that a grant was not changed after it was recorded.

Before acting, an agent asks for a decision. The ledger picks the active grant that covers the action and target and expires first, counts one use, and records the decision as evidence with outcome `PENDING`, or `DENIED` with the reason. The agent then reports `SUCCEEDED` or `FAILED` once; this is a second evidence entry carrying the `decisionId` attribute, the grant as `delegationId` and the same `inputHash` and `correlationId`. Concurrent decisions never use a grant more than `maxUses` times. A reported `FAILED` keeps the use, because the ledger cannot know whether the action ran. The gateways return the use themselves when they see that the call never ran: the MCP or A2A server could not be reached, the MCP server rejected the request (HTTP 4xx, or JSON-RPC parse, invalid request, unknown method or invalid params), or the peer ledger refused the A2A request without a receipt. The outcome entry then carries `grant.useReturned`.

Reason code	Decision	Meaning
`ALLOWED_BY_GRANT`	ALLOW	An active grant covers the action and the target
`AGENT_NOT_ACTIVE`	DENY	The agent is unknown or disabled
`NO_ACTIVE_GRANT`	DENY	The principal has no active grant for this agent
`NOT_COVERED`	DENY	Active grants exist, but none covers the action and target
`NOT_YET_VALID`	DENY	The covering grant starts later
`USES_EXHAUSTED`	DENY	The covering grant has used all its uses
`POLICY_DENIED`	DENY	A grant covers the action, but the active policies deny it
`APPROVAL_PENDING`	DENY	Policies ask for an approval that has not been answered yet
`ALLOWED_BY_APPROVAL`	ALLOW	A grant covers the action and its approval was given
`APPROVAL_DENIED`	DENY	The approver denied the approval
`APPROVAL_EXPIRED`	DENY	Nobody answered the approval in time

Agent, grant and decision changes are evidence too: `agent/register`, `agent/disable`, `agent/rotate-key`, `agent/signing-key`, `grant/create`, `grant/approve`, `grant/deny` and `grant/revoke`, with the owner or the granting principal as `principalId`.

```
curl -X POST http://localhost:8090/api/v1/agents -H "Authorization: Bearer nexusphere-ledger-
development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","name":"Invoice agent","ownerId":"acme"}'
curl -X POST http://localhost:8090/api/v1/grants -H "Authorization: Bearer nexusphere-ledger-
development-key-change-me" -H "Content-Type: application/json" -d
'{"principalId":"alice","agentId":"invoice-agent","actions":["tools/call"],"targets":
["send_email","read_*"],"expiresAt":"2026-12-31T00:00:00Z","maxUses":100}'
curl -X POST http://localhost:8090/api/v1/decisions -H "Authorization: Bearer {agentApiKey}" -H
"Content-Type: application/json" -d
'{"principalId":"alice","action":"tools/call","target":"send_email"}'
curl -X POST http://localhost:8090/api/v1/decisions/{decisionId}/outcome -H "Authorization: Bearer
{agentApiKey}" -H "Content-Type: application/json" -d '{"outcome":"SUCCEEDED"}'
```

Policies

A grant says what a principal lets an agent do; policies say what the operator lets any agent do. The operator activates a Cedar policy set with `PUT /api/v1/policies`, and from then on every decision, including those of the MCP gateway and the A2A gateway, must be allowed by both a grant and the policies. When a grant allows an action the policies deny, the decision is `DENY` with `POLICY_DENIED`, the grant keeps its use, and the reason names the policies that forbid it. A policy that fails to evaluate denies too. Each policy is named by its `@id` annotation, and two policies with the same name are refused.

The principal is `Agent::"<agentId>"`, the action is `Action::"<action>"` and the resource is `Target::"<target>"`. The context carries `principalId`, `target`, `inputHash` when present, `attributes` as a record of strings, `model` with its `name` and `version` when present, and the UTC `hour` (0 to 23) and `weekday` (1 for Monday to 7). With `LEDGER_POLICY_DEFAULT_EFFECT=permit` the ledger adds the policy `ledger-default-permit`, so whatever no policy forbids is allowed; with `deny` only what a policy permits is allowed. Before a policy set is activated, grants alone decide.

A policy set is addressed by the SHA-256 of its UTF-8 text, at most 1 MiB, and is never changed or deleted; activating it records `policy/activate` evidence with that hash. Every decision records the attributes `policy.hash`, `policy.ids` (the policies that decided it) and `policy.defaultEffect`, so a client cannot send attributes starting with `policy.`. A package, report or dispute carries `policies`, the texts of the policy sets its disclosed entries name, and the verifier checks that each text matches its hash and is named by a disclosed entry.

Step 1. Start the ledger as in Run, then register an agent with a grant for every bank tool; the answers must show `201`:

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8090/api/v1/agents -H
"Authorization: Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d
'{"agentId":"payments-agent","name":"Payments agent","ownerId":"acme"}'
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8090/api/v1/grants -H
"Authorization: Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d "
{\"principalId\":\"alice\",\"agentId\":\"payments-agent\",\"actions\":
[\"tools/call\"],\"targets\":[\"bank/*\"],\"expiresAt\":\"$(date -u -d '+30 days' +%Y-%m-
%dT%H:%M:%SZ)\"}"
```

Step 2. Write a policy set that forbids wire transfers and invoices other than 100:

```
cat > policies.cedar <<'CEDAR'
@id("no-wire-transfers")
forbid(principal, action == Action::"tools/call", resource == Target::"bank/wire_transfer");

@id("small-payments")
forbid(principal, action, resource == Target::"bank/pay_invoice")
  unless { context.attributes has amount && context.attributes.amount == "100" };
CEDAR
```

Step 3. Activate it; the answer must show the same hash as `sha256sum` and both policy names:

```
jq -Rs '{text: .}' policies.cedar | curl -s -X PUT http://localhost:8090/api/v1/policies -H
"Authorization: Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d @- | jq -c '{hash,
policies, defaultEffect}'
sha256sum policies.cedar
```

Step 4. Ask for three decisions; the answers must show `POLICY_DENIED`, `POLICY_DENIED` and `ALLOWED_BY_GRANT`:

```
for request in '"target":"bank/wire_transfer"' '"target":"bank/pay_invoice","attributes":
{"amount":"900"}' '"target":"bank/pay_invoice","attributes":{"amount":"100"}'; do
  curl -s -X POST http://localhost:8090/api/v1/decisions -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d "{\"agentId\":\"payments-
agent\",\"principalId\":\"alice\",\"action\":\"tools/call\",$request}" | jq -r '.reasonCode'
done
```

Step 5. Export a package and verify it; the output must show `Policies : 1 policy set texts match the hashes their entries name` and `Result: VALID`:

```
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" > /dev/null
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"payments-agent"}' > package.json
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar package.json
```

Human Approval

A `forbid` policy annotated `@approval("principal")`, `@approval("operator")` or `@approval("any")` does not end the matter: when a grant covers the action and only such policies forbid it, the decision is `DENY` with `APPROVAL_PENDING` and an `approvalId`, and the ledger waits `LEDGER_APPROVAL_TIMEOUT` for the named approver. The operator answers with `POST /api/v1/approvals/{id}/approve` or `/deny`, the principal with the same routes under `/api/v1/principal/approvals` after signing in, or on their own device through OpenID Connect CIBA. When the agent asks again for the same action, target and input, an approved request is `ALLOW` with `ALLOWED_BY_APPROVAL` and uses the grant once; a denied one is `APPROVAL_DENIED`; and one nobody answered in time is `APPROVAL_EXPIRED`, so silence denies. The next request after that asks again. Agents read their own approvals with `GET /api/v1/approvals/{id}`.

Every answer is evidence: `approval/approve`, `approval/deny` or `approval/expire` with the attributes `approval.id`, `approval.action`, `approval.approver`, `approval.method` (`operator`, `principal`, `ciba` or `timeout`), the policy hash and, for a principal, the `consent.*` attributes of their token. The decision entries carry `approval.id` too, and a pending one `approval.expiresAt`. With `LEDGER_APPROVAL_WEBHOOK_URL` the ledger posts each new request as JSON of type `approval.requested`, signed as in HTTP Message Signatures, until an answer `2xx` arrives. With `LEDGER_APPROVAL_CIBA_ENDPOINT`, `LEDGER_APPROVAL_CIBA_TOKEN_ENDPOINT`, `LEDGER_APPROVAL_CIBA_CLIENT_ID` and `LEDGER_APPROVAL_CIBA_CLIENT_SECRET` the ledger starts a CIBA request in poll mode for every request a principal may answer, with the principal id as `login_hint`, and accepts the returned token only when it is valid for `LEDGER_OIDC_ISSUER` and names that principal. Pending requests are checked every `LEDGER_APPROVAL_POLL_INTERVAL`.

Step 1. Start a receiver that prints every webhook it gets:

```
python3 -c 'import http.server as h
class R(h.BaseHTTPRequestHandler):
    def do_POST(self):
        print(self.rfile.read(int(self.headers["Content-Length"])).decode(), flush=True);
self.send_response(204); self.end_headers()
h.HTTPServer(("localhost", 9100), R).serve_forever()' &
```

Step 2. Start the ledger as in Run with `LEDGER_APPROVAL_WEBHOOK_URL=http://localhost:9100/approvals`, then register the agent and its grant as in Step 1 of Policies.

Step 3. Activate a policy that lets the operator approve wire transfers; the answer must list `wire-needs-operator`:

```
cat > approvals.cedar <<'CEDAR'
@id("wire-needs-operator")
@approval("operator")
forbid(principal, action, resource == Target::"bank/wire_transfer");
CEDAR
jq -Rs '{text: .}' approvals.cedar | curl -s -X PUT http://localhost:8090/api/v1/policies -H
"Authorization: Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d @- | jq -c
'.policies'
```

Step 4. Ask for a wire transfer; the answer must show `APPROVAL_PENDING` with an `approvalId`, and the receiver must print an `approval.requested` with the same id:

```
WIRE='{"agentId":"payments-
agent","principalId":"alice","action":"tools/call","target":"bank/wire_transfer"}'
APPROVAL=$(curl -s -X POST http://localhost:8090/api/v1/decisions -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d "$WIRE" | tee /dev/stderr | jq -r
.approvalId)
```

Step 5. Approve it; the answer must show `APPROVED` by the `operator`:

```
curl -s -X POST http://localhost:8090/api/v1/approvals/$APPROVAL/approve -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"reason":"Invoice 42 checked"}' | jq -c
'{status, method}'
```

Step 6. Ask twice more; the answers must show `ALLOWED_BY_APPROVAL`, then `APPROVAL_PENDING` for a new request:

```
for i in 1 2; do curl -s -X POST http://localhost:8090/api/v1/decisions -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d "$WIRE" | jq -r .reasonCode; done
```

Anomaly Alerts

Every `LEDGER_ANOMALY_INTERVAL` the ledger compares each agent's last `LEDGER_ANOMALY_WINDOW` with its own `LEDGER_ANOMALY_BASELINE` before it, using simple statistics over the evidence it already holds. `VOLUME` fires when the number of entries is at least `LEDGER_ANOMALY_THRESHOLD` standard deviations above the mean of the baseline windows (with at least one as the deviation). `DENIALS` and `FAILURES` fire when the share of denied decisions or of failed outcomes is that many binomial standard errors above the agent's baseline share (at least 1%). Each check needs at least `LEDGER_ANOMALY_MIN_EVENTS` events in the window, and an agent is judged only once its first entry is `LEDGER_ANOMALY_MIN_HISTORY` old. No personal field is read.

A new anomaly is recorded as `anomaly/detect` evidence under the agent, with the kind, the observed and expected values, the score and the window as `anomaly.*` attributes, and `anomaly/resolve` follows once the agent is back within its baseline. While an anomaly is open, the metric `ledger_anomaly_score{agent, kind}` carries its score, and the rule `LedgerAgentAnomaly` in

`ledger/prometheus/alerts.yml` sends it through Alertmanager on the channels of Alert Channels. `GET /api/v1/anomalies` lists anomalies by `status` and `agentId`, and `POST /api/v1/anomalies/detect` runs the check at once.

Step 1. Start the ledger as in Run with short windows so that one minute of traffic counts:

```
LEDGER_ANOMALY_WINDOW=1m LEDGER_ANOMALY_BASELINE=1h LEDGER_ANOMALY_MIN_HISTORY=0s
LEDGER_ANOMALY_MIN_EVENTS=5 LEDGER_ANOMALY_INTERVAL=15s \
  LEDGER_PROFILES=postgresql,dev LEDGER_DATABASE_PORT=5433 mvn -pl ledger/server spring-boot:start
```

Step 2. Start Alertmanager and a Prometheus that scrapes this ledger with its alert rules; both must answer ready:

```
sed -e 's/alertmanager:9093/localhost:9093/' -e 's/ledger:8090/localhost:8090/'
ledger/prometheus/prometheus.yml > prometheus-local.yml
docker run -d --name alertmanager --network host -v
"$PWD/ledger/alertmanager:/etc/alertmanager:ro" --entrypoint /bin/sh prom/alertmanager:v0.28.1
/etc/alertmanager/entrypoint.sh
docker run -d --name prometheus --network host -v "$PWD/prometheus-
local.yml:/etc/prometheus/prometheus.yml:ro" -v
"$PWD/ledger/prometheus/alerts.yml:/etc/prometheus/alerts.yml:ro" prom/prometheus:v3.5.0
curl -s localhost:9090/-/ready; curl -s localhost:9093/-/ready
```

Step 3. Register an agent and let it ask six times for an action nobody granted; every answer must be `NO_ACTIVE_GRANT`:

```
curl -s -o /dev/null -X POST http://localhost:8090/api/v1/agents -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"agentId":"payments-
agent","name":"Payments agent","ownerId":"acme"}'
for i in 1 2 3 4 5 6; do curl -s -X POST http://localhost:8090/api/v1/decisions -H "Authorization:
Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"agentId":"payments-
agent","principalId":"alice","action":"tools/call","target":"bank/wire_transfer"}' | jq -r
.reasonCode; done
```

Step 4. After 20 seconds the ledger must list a `DENIALS` and a `VOLUME` anomaly for `payments-agent`:

```
sleep 20
curl -s "http://localhost:8090/api/v1/anomalies?status=OPEN" -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '.items[] | {agentId, kind, score}'
```

Step 5. After another 35 seconds Alertmanager must hold `LedgerAgentAnomaly` for both:

```
sleep 35
curl -s http://localhost:9093/api/v2/alerts | jq -r '.[].labels | "\(.alertname) \(.agent) \
(.kind)"'
```

Step 6. Stop Prometheus, Alertmanager and the ledger:

```
docker rm -f prometheus alertmanager
mvn -pl ledger/server spring-boot:stop
```

Principal Login and Consent

Without OIDC, the operator creates grants and the principal's consent is the operator system's job. When `LEDGER_OIDC_ISSUER` is set, principals sign in with that OpenID Connect provider (Keycloak, Entra ID, Auth0, Okta or any other) and decide on their own grants. The ledger accepts the provider's access or ID token as a bearer token on `/api/v1/principal/**` only. It checks the signature with the provider's keys (from the discovery document, or from `LEDGER_OIDC_JWKS_URI` when the ledger reaches the provider at another address than the issuer), the issuer, the expiry, and that `LEDGER_OIDC_AUDIENCE` is in `aud` or is the `azp` client. The principal ID is the claim named by `LEDGER_OIDC_PRINCIPAL_CLAIM`, for example `preferred_username` or `email`. `GET /public/v1/oidc` tells a sign-in page the issuer and the public client to use, `LEDGER_OIDC_CLIENT_ID` or else the audience.

With OIDC on, a grant created by the operator, or asked for by an agent for itself on `POST /api/v1/grants`, starts as `PENDING`. A pending grant allows nothing and gives no mandate. The principal lists it, then approves it, which makes it `ACTIVE` with `consent` `PRINCIPAL`, or denies it, which makes it `DENIED`. A principal can also create a grant on its own behalf, active at once, and revoke its own grants, which revokes their mandates. Each step is evidence (`grant/create`, `grant/approve`, `grant/deny`, `grant/revoke`) with `consent.issuer`, `consent.subject`, `consent.authTime` and `consent.token`, the SHA-256 of the token the principal used, so the consent can be traced to one sign-in at the provider without storing the token.

```
TOKEN=$(curl -s http://localhost:8180/realms/nexusphere/protocol/openid-connect/token -d
grant_type=password -d client_id=nexusphere-ledger -d username=alice -d password=alice | jq -r
.access_token)
curl -X GET "http://localhost:8090/api/v1/principal/grants?state=PENDING" -H "Authorization:
Bearer ${TOKEN}"
curl -X POST http://localhost:8090/api/v1/principal/grants/{id}/approve -H "Authorization: Bearer
${TOKEN}"
```

MCP Gateway

The ledger can stand in front of MCP servers that speak Streamable HTTP. Each server gets a name and a URL, for example `LEDGER_MCP_SERVERS_FILES_URL=http://files-mcp:3000/mcp`, and an optional `Authorization` header the ledger sends to it. The agent points its MCP client at `/mcp/{name}` with its own API key and names the principal it works for in the `X-Ledger-Principal` header.

Every `tools/call` is decided first, with action `tools/call` and target `{name}/{tool}`, so a grant on `files/*` or `files/read_file` covers it. A denied call never reaches the server and is answered with JSON-RPC error `-32003` and the decision ID and reason code in `error.data`. An allowed call is forwarded, and its outcome is reported from the answer: a JSON-RPC error, `isError: true`, a non-2xx status or an unreachable server (`-32002`, HTTP 502) count as `FAILED`. The input hash is the SHA-256 of the request body, the output hash is the SHA-256 of the answer, the MCP session ID is the correlation ID, and every answer to a decided call carries the `X-Ledger-Decision` header. Other messages, such as `initialize`, `tools/list` and notifications, pass through unchanged with the `Mcp-Session-Id` header. Every request to the server carries the ledger's HTTP message signature.

When the client accepts `text/event-stream`, a streamed answer is relayed event by event as it arrives, including the requests and notifications the server sends inside it (for example `elicitation/create` or progress), and the outcome is recorded when the stream ends from the

event that answers the call, with `mcp.events` (the number of events) and `mcp.stream` (`COMPLETE` or `BROKEN`) as attributes. A client that accepts only JSON gets the answer as one JSON response. `GET /mcp/{name}` relays the server-to-client stream with `Mcp-Session-Id` and `Last-Event-ID`, and the client's answers to server requests pass through on `POST`. A stream stays open for at most `LEDGER_STREAM_TIMEOUT`. JSON-RPC batches are not supported, because MCP removed them in protocol version 2025-06-18; a batch is answered with `-32600`.

```
curl -X POST http://localhost:8090/mcp/files -H "Authorization: Bearer {agentApiKey}" -H "X-Ledger-Principal: alice" -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"read_file","arguments":{"path":"/tmp/report.txt"}}}'
```

Mandates

A grant lives inside one ledger. When an agent acts for its principal at another organization, it carries a mandate: a signed, self-contained copy of the grant that the other side can check without calling back. A mandate is an IETF SD-JWT VC signed with the ledger's Ed25519 key (`alg` `EdDSA`, `typ` `dc+sd-jwt`, `kid` the ledger key ID, `vct` `urn:nexusphere:vct:agent-mandate:1`): the issuer-signed JWT followed by its disclosures, each ending in `~`. The operator, or the agent the grant was given to, asks for one with `POST /api/v1/mandates` and the grant ID, an optional `audience` (the organization that will receive it) and an optional `expiresAt` that may not be later than the grant's own expiry. Only an active or not-yet-valid grant can be turned into a mandate.

Claim	Meaning
`iss`	The ledger, `LEDGER_MANDATE_ISSUER`
`vct`	`urn:nexusphere:vct:agent-mandate:1`
`sub`	The agent ID
`aud`	The receiving organization, when given
`jti`	The mandate ID
`iat`, `nbf`, `exp`	Issued, valid from and expiry, in epoch seconds
`mandate.principal`	The person or organization the agent acts for (selectively disclosable)
`mandate.actions`	Allowed actions, exact or with a trailing `*`
`mandate.targets`	Allowed targets, exact or with a trailing `*`
`mandate.maxUses`	The grant's use limit, when it has one
`mandate.grant`	The grant ID (selectively disclosable)
`mandate.termsHash`	The grant terms' SHA-256, as in the grant's evidence (selectively disclosable)
`status.status_list`	`idx` and `uri` of the mandate's bit in the status list
`cnf.jwk`	The agent's Ed25519 public key, when the agent has registered one
`_sd_alg`	`sha-256`, the hash of the disclosures listed in `mandate._sd`

The principal, grant and terms hash are disclosures, so an agent can hand a mandate on without them: `SdJwt.parse(token).present(Set.of("grant"))` keeps only the grant, and the receiver still checks the signature, time, audience, coverage and revocation. Actions, targets, the use limit, time and status are always in the signed JWT and cannot be withheld. The A2A gateway always

sends the full mandate, because the receiving ledger records the principal. A mandate for an agent with a signing key names that key in `cnf` and is presented with a key binding, as described in Agent Keys and Key Binding.

Revocation follows the IETF Token Status List draft. `GET /public/v1/mandates/status` returns a signed `statuslist+jwt` with one bit per mandate (`1` is revoked), deflated with zlib and base64url encoded in `status_list.lst`, and an `exp` of `LEDGER_MANDATE_STATUS_LIST_TTL` after it was made. Revoking a mandate sets its bit, and revoking a grant revokes every mandate issued from it. The keys are published as a JWK set at `GET /public/v1/keys`. Both public endpoints need no API key. Issuing and revoking a mandate are recorded as evidence (`mandate/issue` with the SHA-256 of the token as the input hash, and `mandate/revoke`), with the grant as the delegation ID and the mandate ID as the target.

```
curl -X POST http://localhost:8090/api/v1/mandates -H "Authorization: Bearer {agentApiKey}" -H
"Content-Type: application/json" -d '{"grantId":"
{grantId}","audience":"https://supplier.example"}'
curl http://localhost:8090/public/v1/keys
curl http://localhost:8090/public/v1/mandates/status
```

Verifying a Mandate

An organization that receives a mandate checks it with `MandateVerifier` from `ledger/mandate`, a plain Java library with no Spring dependency. It trusts only the issuers it is told to trust, reads their keys from `{issuer}/public/v1/keys`, checks the signature, `nbf` and `exp` with a clock skew (60 seconds by default), the audience, and, when an action and target are given, that the mandate covers them. It then reads the status list named in the mandate, which must be served by the issuer, checks its signature, issuer, address and expiry, and looks up the mandate's bit. Keys and status lists are cached (5 minutes by default, and never past the status list's `exp`). When the keys or the status list cannot be read the mandate is not valid, so a verifier fails closed. Each problem has a code: `MALFORMED`, `WRONG_TYPE`, `UNTRUSTED_ISSUER`, `UNKNOWN_KEY`, `BAD_SIGNATURE`, `NOT_YET_VALID`, `EXPIRED`, `WRONG_AUDIENCE`, `NOT_COVERED`, `REVOKED`, `STATUS_UNAVAILABLE`, `KEY_BINDING_MISSING`, `KEY_BINDING_INVALID` and `KEY_NOT_BOUND`. `verifyBound(token, nonce)` also checks the key binding against the nonce the verifier expects, and `requireKeyBinding()` refuses mandates without `cnf`.

```
MandateVerifier verifier = MandateVerifier.builder()
        .trustIssuer("https://ledger.acme.example")
        .audience("https://supplier.example")
        .build();
MandateCheck check = verifier.verify(token, "a2a/send", "supplier/orders");
if (!check.valid()) {
    throw new AccessDeniedException(check.problems().toString());
}
String principal = check.claims().principalId();
```

The verifier jar checks a mandate from the command line. `--public-key` pins the issuer's key instead of reading it from the issuer, and `--skip-status` leaves out the revocation check for offline use. `--nonce` checks the key binding against an expected nonce and `--require-key-binding` refuses mandates without `cnf`. The exit codes are the same as for packages.

```
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar mandate --issuer
http://localhost:8090 --audience https://supplier.example --action a2a/send --target
supplier/orders mandate.jwt
```

AP2 and Verifiable Intent

AP2 and Mastercard's Verifiable Intent, now developed at the FIDO Alliance, carry payment mandates as SD-JWT credentials too. A Nexusphere mandate is not an AP2 mandate, but its parts line up, so a bridge can translate one into the other. AP2 signs with ES256 and Nexusphere with EdDSA; both bind mandates to the holder's key with `cnf` and a KB-JWT.

AP2 / Verifiable Intent	Nexusphere
Intent mandate signed for the user	Mandate issued from a grant the principal approved with their own sign-in
User authorization	The `grant/approve` evidence of the principal's consent, named by `termsHash`
Merchants or payees the intent allows	`aud` and `mandate.targets`
What the agent may do	`mandate.actions`
Intent expiry	`exp`, never later than the grant
Spending or use limit	`mandate.maxUses`, counted by both ledgers
Credential status	`status.status_list`, the IETF Token Status List
Cart and payment mandates	No equivalent: Nexusphere does not handle payments
`merchant_authorization` from the merchant	The receipt the receiving ledger signs for an A2A exchange
Audit trail of the transaction	Evidence on both ledgers, with SCITT statements and receipts

Agent Keys and Key Binding

An API key proves who calls the ledger, but whoever copies it can act as the agent. An agent can therefore register an Ed25519 public key of its own with `PUT /api/v1/agents/{agentId}/signing-key`; the private key never leaves the agent. The agent may set its key once, and only the operator can replace it afterwards, so a stolen API key cannot swap in another key. The ledger records `agent/signing-key` with the key ID and the previous key ID.

Every mandate issued for such an agent names its key in `cnf.jwk` (RFC 7800). The agent presents the mandate with a key binding JWT as in the SD-JWT specification: `typ` `kb+jwt`, signed with its own key, with `iat`, `aud` (the receiving organization), `nonce` and `sd_hash`, the base64url SHA-256 of the presentation up to its last `~`. A verifier checks the signature with the key in `cnf`, the `sd_hash`, the audience, the nonce and that `iat` is at most five minutes old. A mandate with `cnf` but no key binding is `KEY_BINDING_MISSING`; a wrong key, nonce, audience, `sd_hash` or age, or a key binding on a mandate without `cnf`, is `KEY_BINDING_INVALID`.

Once an agent has a signing key, the A2A gateway no longer makes mandates for it. The agent sends its own presentation in `X-Nexusphere-Mandate` with the peer's issuer as `aud` and the hex SHA-256 of the request body as `nonce`. The ledger answers HTTP 400 without the header and HTTP 403 with the problems when the presentation does not hold: a key binding problem, `WRONG_AGENT`, `WRONG_AUDIENCE`, `NOT_COVERED`, `WRONG_PRINCIPAL`, `UNDISCLOSED` when the principal or grant is withheld, `STALE_KEY` when the mandate names a replaced key, and `GRANT_MISMATCH` when the decision picked another grant. The receiving ledger checks the key binding with the same body hash, so a presentation cannot be used for another request, and records the sender's key ID as `a2a.keyBinding`.

Step 1. Make a key pair for the agent; only the public key goes to the ledger:

```
openssl genpkey -algorithm ed25519 -out agent.pem && openssl pkey -in agent.pem -pubout -outform
DER | base64 -w0 > agent.pub
```

Step 2. Register the public key with the agent's own API key; the answer shows its `signingKeyId`:

```
curl -s -X PUT http://localhost:8090/api/v1/agents/invoice-agent/signing-key -H "Authorization:
Bearer {agentApiKey}" -H "Content-Type: application/json" -d "{\"publicKey\":\"$(cat
agent.pub)\"}" | jq -r .signingKeyId
```

Step 3. Get a mandate for the grant; it now carries the agent's key in `cnf`:

```
curl -s -X POST http://localhost:8090/api/v1/mandates -H "Authorization: Bearer {agentApiKey}" -H
"Content-Type: application/json" -d '{"grantId":"
{grantId}","audience":"https://supplier.example"}' | jq -r .token > mandate.jwt
```

Step 4. Present it with a key binding from the agent, here with the Python SDK, and check it; it must end with `Result: VALID` and show the `signingKeyId` from Step 2 as `Agent key`:

```
python3 -c "from nexusphere_ledger import PrivateKey, present_bound; from
cryptography.hazmat.primitives.serialization import load_pem_private_key as load;
print(present_bound(open('mandate.jwt').read().strip(), PrivateKey(load(open('agent.pem',
'rb').read(), None)), 'https://supplier.example', 'ab' * 32))" > presented.jwt
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar mandate --issuer
http://localhost:8090 --audience https://supplier.example --nonce "$(python3 -c "print('ab' *
32)")" presented.jwt
```

Agent Track Records

Another organization that is about to trust an agent wants to know how it has behaved, without reading the evidence. `POST /api/v1/agents/{agentId}/track-record`, called by the operator or by the agent itself, answers a signed SD-JWT VC of type `urn:nexusphere:vct:agent-track-record:1` with `typ` `dc+sd-jwt`. It counts the evidence that names the agent up to the latest checkpoint: the `total`, the first and last sequence and when the ledger recorded them, and, as claims the agent may withhold, the `outcomes`, the number of distinct `principals` and the count of each action. It names the checkpoint `sequence`, its `headHash` and the signed index root of Index Proofs, so the `total` must equal the count an index proof gives for the agent at the same checkpoint. The body takes an optional `audience` and `validFor`, an ISO-8601 duration between `PT1M` and `P365D`, by default `P30D`. When the agent has a signing key, the credential names it in `cnf` and must be presented with a key binding as a mandate is. The ledger records `track-record/issue` with the SHA-256 of the credential.

`nexusphere-ledger-verify track-record` checks the credential against the issuer's published keys and, with `--index-proof`, that a proof for the `agent` term at the same checkpoint verifies and gives the same index root and counts. A track record is a snapshot: it does not say what happened after its checkpoint, and it is not revoked; it expires.

Step 1. Start the ledger as in Run, register an agent and record two actions under it:

```
AGENT_KEY=$(curl -s -X POST http://localhost:8090/api/v1/agents -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"agentId":"quote-agent","name":"Quote
agent","ownerId":"acme"}' | jq -r .apiKey)
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $AGENT_KEY" -H
"Content-Type: application/json" -d '{"agentId":"quote-
agent","principalId":"alice","action":"tools/call","outcome":"SUCCEEDED"}' | jq -c '{sequence}'
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $AGENT_KEY" -H
"Content-Type: application/json" -d '{"agentId":"quote-
agent","principalId":"bob","action":"tools/call","outcome":"FAILED"}' | jq -c '{sequence}'
```

Step 2. Sign a checkpoint and let the agent ask for its track record for a supplier:

```
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{sequence}'
curl -s -X POST http://localhost:8090/api/v1/agents/quote-agent/track-record -H "Authorization:
Bearer $AGENT_KEY" -H "Content-Type: application/json" -d
'{"audience":"https://supplier.example","validFor":"P7D"}' > track-record.json
jq -r .credential track-record.json > track-record.jwt
```

Step 3. Get an index proof for the agent at the same checkpoint:

```
curl -s -X POST http://localhost:8090/api/v1/index/proofs -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d "{\"term\":
{\"type\":\"agent\",\"agentId\":\"quote-agent\"},\"sequence\":$(jq .checkpoint track-
record.json)}" > agent-proof.json
```

Step 4. Verify both as the supplier; the output must show `Entries : 3`, `Outcomes : FAILED 1, SUCCEEDED 2`, `Principals : 3` and `Result: VALID`:

```
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar track-record --issuer
http://localhost:8090 --audience https://supplier.example --index-proof agent-proof.json track-
record.jwt
```

A2A Gateway

Two organizations that each run a ledger can let their agents talk over A2A (JSON-RPC over HTTP) with evidence on both sides that matches entry for entry. The sending ledger names each peer with the URL of the peer's inbound endpoint and the peer ledger's issuer, for example `LEDGER_A2A_PEERS_SUPPLIER_URL=https://ledger.supplier.example/a2a/in/sales` and `LEDGER_A2A_PEERS_SUPPLIER_ISSUER=https://ledger.supplier.example`. The receiving ledger lists the issuers it trusts in `LEDGER_A2A_TRUSTED_ISSUERS` (comma separated) and names its own agents with their A2A URL and an optional `Authorization` header, for example `LEDGER_A2A_AGENTS_SALES_URL=http://sales-agent:9000/a2a`.

The agent sends its JSON-RPC request to `/a2a/out/{peer}` with its own API key and the `X-Ledger-Principal` header. The ledger decides action `a2a/send` on target `{peer}/{method}`, so a grant on `supplier/*` covers it. A denied request never leaves and is answered with JSON-RPC error `-32003`. An allowed request carries two headers: `X-Nexusphere-Mandate`, a mandate for the grant with the peer's issuer as audience (reused while it is valid for at least another minute), or the agent's own key-bound presentation when the agent has a signing key, and `X-Nexusphere-Request`, a request proof signed by the sending ledger (`typ` `nexusphere-a2a-request+jwt`) that binds the mandate, the method, the SHA-256 of the body and the decision ID as the request ID.

The receiving ledger answers on `/a2a/in/{agent}` without an API key. It verifies the mandate with `MandateVerifier` (trusted issuer, key from the issuer's JWKS, signature, time, its own issuer as audience, the `a2a/send` action, the revocation status and, for a mandate with `cnf`, the key binding with the body hash as nonce), the sender's HTTP message signature and the request proof (same issuer, key, mandate and agent, its own issuer as audience, the method, the body hash and an `iat` within `LEDGER_A2A_REQUEST_MAX_AGE`). A request ID is accepted once. A mandate or proof that cannot be trusted is answered with HTTP 401 and is not recorded. A trusted mandate that is revoked, expired or for another audience is answered with HTTP 403 and recorded as an `a2a/receive` DENY entry. Otherwise the request goes to the local agent and one `a2a/receive` entry records it with the same input and output hashes as the sender, the mandate as delegation ID and the request ID as correlation ID. The answer carries `X-Nexusphere-Receipt`, a receipt signed by the receiving ledger (`typ nexusphere-a2a-receipt+jwt`) with the request and response hashes, the HTTP status, the outcome, and the sequence and hash of its evidence entry.

The sending ledger verifies the receipt with the peer's published key, records the outcome with `a2a.receipt` set to `VERIFIED`, `INVALID` or `MISSING` and the peer's evidence sequence and hash, and returns the answer with `X-Ledger-Decision`, `X-Ledger-Exchange`, `X-Ledger-Receipt` and the receipt. Both ledgers keep the mandate, the request proof and the receipt of every exchange at `GET /api/v1/a2a/exchanges/{id}`, so either side can later show what the other signed. The receiving ledger also counts the uses of every mandate on its own: once the requests it accepted under a mandate reach `mandate.maxUses`, it denies the next one with `USES_EXHAUSTED`, so a sender that ignores its own count gets nowhere. A request that could not reach the receiving agent does not count.

Streaming methods (`message/stream`, `tasks/resubscribe`) work the same way. When the agent answers with server-sent events, both ledgers relay every event as it arrives. The response hash of a stream is the SHA-256 of the `data` of every event, each followed by a newline, so both sides compute the same value. The exchange fails when an event is a JSON-RPC error, a task ends `failed` or `rejected`, or the stream breaks. When the stream ends, the receiving ledger records its entry and sends the receipt as a last event named `nexusphere-receipt`; the sending ledger takes that event out, verifies the receipt and records the outcome with `a2a.events` and `a2a.stream`, so the agent sees only the events of the task.

```
curl -X POST http://localhost:8090/a2a/out/supplier -H "Authorization: Bearer {agentApiKey}" -H
"X-Ledger-Principal: alice" -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":"1","method":"message/send","params":{"message":
{"role":"user","messageId":"1","parts":[{"kind":"text","text":"Order 40 pallets"}]}}}'
```

Reconciling Two Ledgers

Each ledger holds its own side of every exchange, so two organizations can check that their records agree without sharing their evidence. `POST /api/v1/a2a/statements` (operator) takes a `peer`, a configured peer name or a trusted issuer, and a window `from` and `to` of at most 31 days, and answers a statement signed by the ledger (`typ nexusphere-a2a-statement+jwt`) of every exchange with that peer whose request proof was issued in the window: its direction, request ID, request time, method, mandate ID, request and response hashes, HTTP status, outcome and the SHA-256 of its receipt. It names no principal, agent or content.

The operator sends the statement to the peer by any channel. `POST /api/v1/a2a/reconciliations` takes the peer's `statement`, checks its signature with the peer's published keys and that it names this ledger, builds this ledger's own statement for the same window and compares them request by request. An exchange is `NOT_RECEIVED` when the sender has it and the receiver does not, unless the receiver refused it before recording it (HTTP 401) or could not be reached; `NOT_SENT` when the receiver holds a request the sender never sent, which only someone with the sender's key acting around the sender's ledger can cause; and `MISMATCH` with the fields that differ. The answer carries this ledger's signed statement, and the reconciliation is recorded as `a2a/reconcile` with the hashes of both statements. `nexusphere-ledger-verify reconcile` compares two statements offline.

Step 1. Start a stand-in sales agent on port 9000 that accepts every order:

```
cat > sales-agent.py <<'EOF'
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

class Agent(BaseHTTPRequestHandler):
    def do_POST(self):
        request = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        answer = {"jsonrpc": "2.0", "id": request["id"], "result": {"kind": "message", "role":
"agent",
                  "messageId": "a-" + str(request["id"]), "parts": [{"kind": "text", "text":
"Order accepted"}]}}
        body = json.dumps(answer).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

HTTPServer(("localhost", 9000), Agent).serve_forever()
EOF
python3 sales-agent.py &
```

Step 2. Start the supplier's ledger on port 8092 with its own database and key, trusting the ledger on port 8090:

```
docker run -d --name supplier-ledger-db -e POSTGRES_DB=ledger -e POSTGRES_USER=ledger -e
POSTGRES_PASSWORD=ledger -p 5435:5432 postgres:18-alpine && sleep 5
openssl genpkey -algorithm ed25519 -out supplier.pem
LEDGER_PORT=8092 LEDGER_DATABASE_PORT=5435 LEDGER_SIGNING_PRIVATE_KEY=$(openssl pkey -in
supplier.pem -outform DER | base64 -w0) LEDGER_SIGNING_PUBLIC_KEY=$(openssl pkey -in supplier.pem
-pubout -outform DER | base64 -w0) LEDGER_A2A_TRUSTED_ISSUERS=http://localhost:8090
LEDGER_A2A_AGENTS_SALES_URL=http://localhost:9000 java -jar ledger/server/target/server-1.0.0-
SNAPSHOT-exec.jar --spring.profiles.active=postgresql,dev > supplier-ledger.log 2>&1 &
until curl -sf http://localhost:8092/actuator/health > /dev/null; do sleep 1; done
```

Step 3. Start the buyer's ledger of Run with the supplier as its peer:

```
LEDGER_PROFILES=postgresql,dev LEDGER_DATABASE_PORT=5433
LEDGER_A2A_PEERS_SUPPLIER_URL=http://localhost:8092/a2a/in/sales
LEDGER_A2A_PEERS_SUPPLIER_ISSUER=http://localhost:8092 mvn -pl ledger/server spring-boot:start
```

Step 4. Register a buying agent, grant it `supplier/*` for alice and send two orders; the grant must be `ACTIVE` and both orders must answer `Order accepted`:

```
AGENT_KEY=$(curl -s -X POST http://localhost:8090/api/v1/agents -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d
'{"agentId":"buyer-agent","name":"Buyer agent","ownerId":"acme"}' | jq -r .apiKey)
curl -s -X POST http://localhost:8090/api/v1/grants -H "Authorization: Bearer nexusphere-ledger-
development-key-change-me" -H "Content-Type: application/json" -d "
{\"principalId\":\"alice\",\"agentId\":\"buyer-agent\",\"actions\":[\"a2a/send\"],\"targets\":
[\"supplier/*\"],\"expiresAt\":\"$(date -u -d '+30 days' +%Y-%m-%dT00:00:00Z)\"}" | jq -r .status
for id in 1 2; do curl -s -X POST http://localhost:8090/a2a/out/supplier -H "Authorization: Bearer
$AGENT_KEY" -H "X-Ledger-Principal: alice" -H "Content-Type: application/json" -d "
{\"jsonrpc\":\"2.0\",\"id\":\"$id\",\"method\":\"message/send\",\"params\":{\"message\":
{\"role\":\"user\",\"messageId\":\"$id\",\"parts\":[{\"kind\":\"text\",\"text\":\"Order 40
pallets\"}]}}}" | jq -r '.result.parts[0].text'; done
```

Step 5. Let the buyer's ledger sign its statement of the exchanges with the supplier from an hour ago to an hour from now; it must count `2`:

```
FROM=$(date -u -d '-1 hour' +%Y-%m-%dT%H:%M:%SZ) TO=$(date -u -d '+1 hour' +%Y-%m-%dT%H:%M:%SZ)
curl -s -X POST http://localhost:8090/api/v1/a2a/statements -H "Authorization: Bearer nexusphere-
ledger-development-key-change-me" -H "Content-Type: application/json" -d "
{\"peer\":\"supplier\",\"from\":\"$FROM\",\"to\":\"$TO\"}" > buyer-statement.json
jq -r .statement buyer-statement.json > buyer.jwt && jq .exchanges buyer-statement.json
```

Step 6. Let the supplier's ledger reconcile it with its own; it must show `"reconciled":true`, `"matched":2` and no discrepancies:

```
curl -s -X POST http://localhost:8092/api/v1/a2a/reconciliations -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d "
{\"statement\":\"$(cat buyer.jwt)\"}" > reconciliation.json
jq -c '{reconciled, matched, discrepancies}' reconciliation.json && jq -r .statement
reconciliation.json > supplier.jwt
```

Step 7. Compare both statements offline, as an auditor holding them; it must end with `Result: RECONCILED`:

```
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar reconcile buyer.jwt supplier.jwt
```

Step 8. Stop the supplier's ledger, its database and the sales agent:

```
kill %1 %2
docker rm -f supplier-ledger-db
```

HTTP Message Signatures

Every request the ledger sends on, to an MCP server, to a peer ledger or to a local A2A agent, carries an RFC 9421 HTTP message signature made with the ledger's signing key, so the receiver can check that the ledger sent it and that nothing changed on the way. The signature covers `@method`, `@authority` and `@path`, and, when there is a body, the RFC 9530 `Content-Digest` (`sha-256`). Its parameters are `created`, `expires` (five minutes later), a random `nonce`, `keyid` (the ledger key ID, as in `/public/v1/keys`), `alg` `ed25519` and `tag` `nexusphere-ledger`, under the label `nexusphere`:

```
Content-Digest: sha-256=:J5ADMGzC1HOdzr0zMO+g9eAsXJv5yEPKc6lRWCUkdvg=:
Signature-Input: nexusphere=("@method" "@authority" "@path" "content-
digest");created=1790841600;expires=1790841900;nonce="c2lnbmF0dXJlLW5vbmNl";keyid="06e3fd8fda29bb6
0";alg="ed25519";tag="nexusphere-ledger"
Signature:
nexusphere=:buTaWUXE/3h2wCIFRT7OWe2BLf1qwyqqz+5uHPOwjK/X4BsVeQ2CoHxhnCAQzwfEFquoEZdqxEaNFsoI98xlAQ
==:
```

A receiving ledger requires the signature on `/a2a/in/{agent}`: it reads the key from the issuer named in the mandate and answers HTTP 401 with problem `HTTP_SIGNATURE` when the signature is missing, stale, made with an unknown key, or does not match the method, address or body. Behind a proxy, the address the receiver sees must be the one the sender called, so the proxy has to pass the original `Host` or `X-Forwarded-Host` and port. An MCP server or agent checks the signature with `HttpSignatures.verify` from `ledger/mandate`, `verify_request` from the Python SDK or `verifyRequest` from the TypeScript SDK, with the keys from `/public/v1/keys`, and can keep the nonces it has seen to refuse replays.

Step 1. Save a stand-in MCP server that checks each request with the Python SDK as `check_signature.py`:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
from nexusphere_ledger import JwksKeys, verify_request
from nexusphere_ledger.mandate import http_fetcher

keys = JwksKeys(http_fetcher())

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        body = self.rfile.read(int(self.headers.get("Content-Length", 0)))
        url = "http://%s%s" % (self.headers["Host"], self.path)
        verified = verify_request("POST", url, dict(self.headers), body, lambda kid:
keys.resolve("http://localhost:8090", kid))
        print("verified", verified.key_id, flush=True)
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(b'{"jsonrpc":"2.0","id":1,"result":{"content":[]}}')

HTTPServer(("localhost", 3001), Handler).serve_forever()
```

Step 2. Start it, then start the ledger with `LEDGER_MCP_SERVERS_CHECK_URL=http://localhost:3001/mcp` set:

```
python3 check_signature.py
```

Step 3. Call a tool through the ledger with an agent that has a grant on `check/*`; the stand-in server must print `verified` and the ledger's active key ID from `/api/v1/keys`:

```
curl -s -X POST http://localhost:8090/mcp/check -H "Authorization: Bearer {agentApiKey}" -H "X-
Ledger-Principal: alice" -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"echo","arguments":{}}}'
```

Model and Prompt

Evidence names the model the agent ran in `model`: its `name`, its `version`, the `promptHash` of its system prompt and the `configHash` of its configuration, such as temperature and tools, each a lowercase SHA-256. The content hash covers `model`, so the model of an action cannot change afterwards, and a change of model, version or prompt shows up between two entries. Entries without a model hash as they did before. A compliance profile can require `model`, `modelVersion`, `promptHash` or `configHash`. Through the MCP gateway and the A2A gateway the agent sends the headers `X-Ledger-Model`, `X-Ledger-Model-Version`, `X-Ledger-Prompt-Hash` and `X-Ledger-Config-Hash`; `POST /api/v1/decisions` takes the same `model` object as evidence, and the ledger records it with the decision and its outcome.

The hash alone proves which prompt ran only to someone who has the text. `PUT /api/v1/prompts` stores a prompt or configuration text, addressed by the SHA-256 of its UTF-8 bytes, and answers its `hash`; storing the same text again changes nothing. An agent or the operator may store texts, only the operator reads them with `GET /api/v1/prompts/{hash}`, and the ledger never changes or deletes them. With `"includePrompts": true` a package, report or dispute carries `prompts`, the stored texts named by its disclosed entries, and the verifier checks that each text matches its hash and is named by a disclosed entry.

Step 1. Start the ledger as in Run.

Step 2. Store the system prompt; the answer must show the same hash as `sha256sum`:

```
PROMPT='You pay approved invoices of Acme up to 500 EUR.'
curl -s -X PUT http://localhost:8090/api/v1/prompts -H "Authorization: Bearer $LEDGER_API_KEY" -H
"Content-Type: application/json" -d "{\"text\":\"$PROMPT\"}" | jq -r .hash
printf '%s' "$PROMPT" | sha256sum
```

Step 3. Record an action with its model and sign a checkpoint; the answer must show the model:

```
PROMPT_HASH=$(printf '%s' "$PROMPT" | sha256sum | cut -d' ' -f1)
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d "{\"agentId\":\"payments-
agent\",\"principalId\":\"alice\",\"action\":\"tools/call\",\"target\":\"bank/pay_invoice\",\"deci
sion\":\"ALLOW\",\"outcome\":\"SUCCEEDED\",\"model\":{\"name\":\"claude-sonnet-5-
5\",\"version\":\"2026-09-01\",\"promptHash\":\"$PROMPT_HASH\"}}" | jq -c '{sequence, model}'
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{sequence}'
```

Step 4. Export a package with the prompt texts; the answer must show the prompt:

```
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"payments-agent","includePrompts":true}' >
package.json
jq -c '.prompts' package.json
```

Step 5. Verify the package; the output must show `Prompts : 1 prompt texts match the hashes their entries name` and `Result: VALID`:

```
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar package.json
```

Compliance Profiles

The ledger knows no law itself. The rules of a country or region live in a compliance profile, a JSON file that says how long each kind of evidence is kept at least and at most, whether personal data can be erased on request and how fast, where the ledger data may be kept, which evidence fields are required, which RFC 3161 timestamp authorities are accepted and which report templates apply. `LEDGER_COMPLIANCE_PROFILES` lists the active profiles, `baseline` by default. When several are active the strictest rule wins: the longest minimum and the shortest maximum retention, the shortest erasure deadline, the regions and timestamp authorities all of them allow, and every field any of them requires. A profile is a starting point for your own legal review, not legal advice. The ledger ships three:

Profile	Retention	Erasure on request	Region	Required fields	Reports
`baseline`	none	no	any	none	`dispute`
`eu`	6 months	within 1 month	`eea`	`target`, `inputHash`	`dispute`, `dora-ict-records`, `eu-ai-act-logging`
`us`	6 years	within 45 days	any	`target`	`dispute`, `finra-supervision`, `sec-17a-4-records`

Your own profile is a file `{id}.json` in `LEDGER_COMPLIANCE_PROFILE_DIRECTORY`; adding a jurisdiction needs no change to the ledger. A required field is one of `target`, `reason`, `delegationId`, `inputHash`, `outputHash`, `correlationId`, `occurredAt`, `model`, `modelVersion`, `promptHash` and `configHash`, or an attribute as `attributes.{name}`; evidence an agent records without it is refused with HTTP 400. When a profile limits the region, `LEDGER_COMPLIANCE_REGION` must name where the ledger and its database run, and the ledger does not start when it is missing or not allowed; the ledger trusts what it is told and cannot see where its servers really are. Every checkpoint signs the id and SHA-256 digest of the canonical JSON of each active profile, so a package shows which rules were in force, and each change of the active profiles or region is recorded as `compliance/activate` evidence by the agent `ledger`. `GET /public/v1/compliance` returns the active profiles with their digests and the combined rules.

Step 1. Save your own profile as `profiles/uk.json`:

```
{
  "id": "uk",
  "version": 1,
  "name": "United Kingdom",
  "description": "Our own reading of UK GDPR and FCA record keeping.",
  "retention": [{"actions": "*", "minimum": "P5Y"}],
  "erasure": {"onRequest": true, "deadline": "P1M"},
  "residency": ["uk", "eea"],
  "requiredFields": ["target", "attributes.model"],
  "timestampAuthorities": [],
  "reports": ["dispute"]
}
```

Step 2. Start the ledger with the EU profile and your own:

```
export LEDGER_COMPLIANCE_PROFILES=eu,uk
export LEDGER_COMPLIANCE_PROFILE_DIRECTORY=$PWD/profiles
export LEDGER_COMPLIANCE_REGION=eea
```

Step 3. Read the combined rules; the result must show `"residency":["eea"]`, retention `P5Y` and the required fields of both profiles:

```
curl -s http://localhost:8090/public/v1/compliance | jq -c .effective
```

Step 4. Record evidence without the required fields; the answer must be HTTP 400 naming `target`, `inputHash` and `attributes.model` with the profiles that require them:

```
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","outcome":"SUCCEEDED"}'
```

Step 5. Sign a checkpoint; the result must have `format` `nexusphere-ledger/checkpoint/v2` and the `eu` and `uk` profiles with the digests from Step 3:

```
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{format, profiles}'
```

Erasure

`POST /api/v1/principals/{principalId}/erasure` erases what the ledger holds about a principal, by crypto-shredding. For each entry whose retention under the active compliance profiles has passed, the encrypted values and salts are deleted; an entry still inside its retention, or of a principal under a legal hold, keeps them and the answer says how many and until when. The ledger continues such a pending erasure on its own once the retention ends and the hold is released. When nothing is retained the principal's data key is destroyed and the principal's name in grants, decisions, mandates and A2A exchanges becomes `erased:{principalRef}`; a later request for the same principal answers HTTP 404. The chain, the transparency log, every checkpoint and every package stay valid, because the hash of each entry covers only the commitments of its personal fields. A principal with an `ACTIVE` or `PENDING` grant is refused with HTTP 409 `GRANTS_OPEN`, so revoke or deny the grants first. Each erasure is recorded as `principal/erase` evidence by the agent `ledger`, naming only the principal reference. Backups hold the data until they expire, so keep them no longer than the erasure deadline of your profiles; see Operations.

Step 1. Record evidence for the principal `carol`:

```
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"carol","action":"tools/call","target":"read_invoice","outcome":"SUCCEEDED","
attributes":{"email":"carol@example.com"}}' | jq -r .id > entry.txt
```

Step 2. Erase `carol`; with the `baseline` profile the answer must show `"erasedEntries":1` and `"completed":true`:

```
curl -s -X POST http://localhost:8090/api/v1/principals/carol/erasure -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"reason":"request of the principal"}' |
jq -c .
```

Step 3. Read the entry; it must show `"erased":true` with `principalId`, `target` and `attributes` gone and its commitments kept:

```
curl -s http://localhost:8090/api/v1/evidence/$(cat entry.txt) -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{erased, principalId, target, attributes, commitments}'
```

Step 4. Verify the chain; the answer must show `"valid":true`:

```
curl -s http://localhost:8090/api/v1/verification -H "Authorization: Bearer $LEDGER_API_KEY" | jq
-c '{valid, checkedEntries}'
```

Retention and Legal Holds

A retention rule of a compliance profile can set a `maximum` next to its `minimum`, an ISO-8601 period no shorter than the minimum. Every `LEDGER_COMPLIANCE_RETENTION_INTERVAL`, one hour by default, the ledger erases the personal fields of each entry older than the shortest maximum of the active profiles for its action, by the same crypto-shredding as an erasure, records how many as `retention/expire` evidence and continues the pending erasures whose retention has ended. `POST /api/v1/retention/sweep` runs the same pass at once. Profiles whose maximum for an action is shorter than another profile's minimum cannot be combined, and the ledger does not start.

A legal hold keeps everything about a principal for a dispute, an investigation or a court order, whatever the profiles say: while a hold is active, neither an erasure nor the retention pass deletes anything about that principal. `POST /api/v1/legal-holds` places one with a required `reason`, `POST /api/v1/legal-holds/{id}/release` releases it, and both are recorded as `legal-hold/place` and `legal-hold/release` evidence by the agent `ledger`. The Privacy page of the web UI does the same.

Step 1. Save a profile that keeps evidence at most a year as `profiles/short.json`:

```
{
  "id": "short",
  "version": 1,
  "name": "Short",
  "retention": [{"actions": "*", "minimum": "P1D", "maximum": "P1Y"}]
}
```

Step 2. Start the ledger with it:

```
export LEDGER_COMPLIANCE_PROFILES=short
export LEDGER_COMPLIANCE_PROFILE_DIRECTORY=$PWD/profiles
```

Step 3. Record evidence from 2019 for `dave` and `erin`, and place a legal hold on `erin`:

```
for p in dave erin; do curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization:
Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"'$p'","action":"tools/call","target":"read_invoice","outcome":"SUCCEEDED","o
ccurredAt":"2019-01-01T00:00:00Z"}' | jq -r .id > $p.txt; done
curl -s -X POST http://localhost:8090/api/v1/legal-holds -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"principalId":"erin","reason":"dispute
42"}' | jq -r .id > hold.txt
```

Step 4. Run the retention pass; the answer must show `"expiredEntries":1`:

```
curl -s -X POST http://localhost:8090/api/v1/retention/sweep -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c .
```

Step 5. Read both entries; `dave` must show `"erased":true` and `erin` `"erased":false`:

```
for p in dave erin; do curl -s http://localhost:8090/api/v1/evidence/$(cat $p.txt) -H
"Authorization: Bearer $LEDGER_API_KEY" | jq -c '{erased, principalId}'; done
```

Step 6. Release the hold and run the pass again; the answer must show `"expiredEntries":1` for `erin`:

```
curl -s -X POST http://localhost:8090/api/v1/legal-holds/$(cat hold.txt)/release -H
"Authorization: Bearer $LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"reason":"dispute
settled"}' | jq -c '{releasedAt, releaseReason}'
curl -s -X POST http://localhost:8090/api/v1/retention/sweep -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c .
```

Checkpoint Timestamps

A signed checkpoint says when it was made, but only by the ledger's own clock. With `LEDGER_TIMESTAMP_URL` set to an RFC 3161 timestamp authority, the ledger asks that authority to timestamp the SHA-256 of every checkpoint's signed bytes, and the token proves to anyone that the checkpoint, and so every entry under it, existed at that time. `LEDGER_TIMESTAMP_CERTIFICATES` is a PEM file with the authority's certificate chain: the ledger keeps only tokens that chain to it and publishes it at `GET /public/v1/timestamp-authority`. When the authority cannot be reached, the checkpoint is signed anyway, and the scheduler timestamps it on a later round.

The token travels as `timestamp` in the checkpoint of an evidence package, next to `timestampedAt` in the checkpoint API. Every verifier checks it offline: the token must be for this checkpoint, its signature and the signer's certificate must hold, and a token more than five minutes older than the checkpoint is rejected. `--timestamp-certificates` pins the authority's chain and `--require-timestamp` rejects a checkpoint without a token. When the compliance profiles list `timestampAuthorities`, the ledger does not start without `LEDGER_TIMESTAMP_URL` or with an authority that is not listed.

Step 1. Install the Sigstore timestamp authority and start it; its signing chain is made in memory at every start:

```
go install github.com/sigstore/timestamp-authority/v2/cmd/timestamp-server@v2.1.4
timestamp-server serve --host 127.0.0.1 --port 3004 &
```

Step 2. Save its certificate chain; the answer must show `3`:

```
curl -s http://127.0.0.1:3004/api/v1/timestamp/certchain > tsa-chain.pem
grep -c BEGIN tsa-chain.pem
```

Step 3. Start the ledger with the authority:

```
export LEDGER_TIMESTAMP_URL=http://127.0.0.1:3004/api/v1/timestamp
export LEDGER_TIMESTAMP_CERTIFICATES=$PWD/tsa-chain.pem
```

Step 4. Record evidence and sign a checkpoint; the answer must show a `timestampedAt`:

```
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","target":"read_invoice","outcome":"SUCCEEDED"}'
| jq -c '{sequence}'
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{sequence, timestampedAt}'
```

Step 5. Export a package and verify it with the authority pinned; the output must show `Timestamp : ... by CN=Test TSA Timestamping,O=local (pinned)` and `Result: VALID`:

```
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' > package.json
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar --timestamp-certificates tsa-chain.pem
--require-timestamp package.json
```

Rekor Anchoring

A timestamp proves when a checkpoint existed, but only the ledger and its readers hold the checkpoint. With `LEDGER_REKOR_URL` set to a Rekor v2 log, such as the public Sigstore log or a private one, the ledger also writes every checkpoint to that log as a `hashedrekord` entry: the SHA-512 of the checkpoint's signed bytes, signed with the ledger key. Anyone can then find the checkpoint in a log the operator does not run, and a ledger that rewrote its history would have to explain two different checkpoints for the same sequence. `LEDGER_REKOR_KEY` is the verifier key of the log: the ledger keeps only entries whose log checkpoint that key signs, and publishes both at `GET /public/v1/rekor`. When the log cannot be reached, the checkpoint is signed anyway, and the scheduler anchors it on a later round.

The entry travels as `rekor` in the checkpoint of an evidence package, next to `rekorLogIndex` in the checkpoint API. Every verifier checks it offline: the entry must be for this checkpoint's bytes and this checkpoint's key, the inclusion proof must lead to the root of the log checkpoint, and `--rekor-key` pins the log that must have signed it. `--require-rekor` rejects a checkpoint without an entry. Rekor v2 records no time of its own, so anchoring goes with checkpoint timestamps rather than replacing them. Rekor needs Ed25519ph signatures, which Vault cannot make, so the ledger does not start with `LEDGER_SIGNING_PROVIDER=vault` and `LEDGER_REKOR_URL` together.

Step 1. Install a Rekor v2 log that keeps its tiles on disk, and make its signing key:

```
go install github.com/sigstore/rekor-tiles/v2/cmd/rekor-server/posix@v2.3.0
cp "$(go env GOPATH)/bin/posix" rekor-server
openssl genpkey -algorithm ed25519 -out rekor-key.pem
```

Step 2. Start the log as `rekor.local`:

```
./rekor-server serve --http-port 3005 --grpc-port 3006 --http-metrics-port 2115 --hostname
rekor.local --storage-dir rekor-log --signer-filepath rekor-key.pem --checkpoint-interval 1s &
```

Step 3. Build the verifier key of the log from its public key; the answer must start with `rekor.local+`:

```
REKOR_PUBLIC=$(openssl pkey -in rekor-key.pem -pubout -outform DER | tail -c 32 | base64 -w0)
REKOR_ID=$( (printf 'rekor.local\n\001'; echo "$REKOR_PUBLIC" | base64 -d) | openssl dgst -sha256
-binary | head -c 4 | od -An -tx1 | tr -d ' \n')
REKOR_KEY="rekor.local+$REKOR_ID+$( (printf '\001'; echo "$REKOR_PUBLIC" | base64 -d) | base64 -
w0)"
echo "$REKOR_KEY"
```

Step 4. Start the ledger with the log, then ask which log it anchors in; the answer must show the same key:

```
export LEDGER_REKOR_URL=http://127.0.0.1:3005
export LEDGER_REKOR_KEY=$REKOR_KEY
curl -s http://localhost:8090/public/v1/rekor | jq -c
```

Step 5. Record evidence and sign a checkpoint; the answer must show a `rekorLogIndex`:

```
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","target":"read_invoice","outcome":"SUCCEEDED"}'
| jq -c '{sequence}'
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{sequence, rekorLogIndex}'
```

Step 6. Export a package and verify it with the log pinned; the output must show `Rekor : entry 0 in rekor.local (pinned)` and `Result: VALID`:

```
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' > package.json
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar --rekor-key "$REKOR_KEY" --require-
rekor package.json
```

Reports and Dispute Packages

Each compliance profile names the report templates that apply to it, and `GET /api/v1/reports` lists those of the active profiles. `POST /api/v1/reports/{id}` exports an evidence package for the scope in the body, verifies it with the timestamp authority and Rekor log the ledger uses, and maps what the evidence shows to the requirements of the template. Every section names a requirement and its source, says `SHOWN`, `PARTLY_SHOWN` or `NOT_SHOWN`, and lists the findings and entry sequences behind it. A report maps evidence to requirements; it is not legal advice and not a certification of compliance.

Template	Profiles	Sections
`dispute`	all	Incident timeline, authority, outcomes, model, integrity, what the package shows
`eu-ai-act-logging`	`eu`	EU AI Act Articles 12(1), 12(2), 12(2)(a), 12(3), 19(1) with 26(6), and integrity
`dora-ict-records`	`eu`	DORA Articles 9(3), 9(4)(c), 17 and 28(3)
`finra-supervision`	`us`	FINRA Rule 3110(a), 3110(b) and the exceptions for review
`sec-17a-4-records`	`us`	SEC Rule 17a-4(b), 17a-4(f)(2) for storage and time, and 17a-4(f)(3) for the copy

The report is canonical JSON in the format `nexusphere-ledger/report/v1`, signed with the ledger key over everything but `signature`. It names the checkpoint, head hash, scope and a digest of the hashes of the entries it covers, so a verifier checks it against the package it came from: `nexusphere-ledger-verify report --package package.json report.json`. With `?format=html` the same report comes as one page for reading and printing. A profile that names a template the ledger does not have stops the ledger from starting.

`POST /api/v1/disputes` takes the scope with a `title` and an optional `description` of one incident and answers a ZIP archive for an insurer, arbitrator or court: the evidence package, the signed dispute report in JSON and HTML, the signing keys with their rotations and revocations, the compliance profiles, and, when the ledger uses them, the timestamp authority's chain, the Rekor log and the witness keys. `nexusphere-ledger-verify dispute dispute.zip` checks all of it at once. The keys and logs in the archive come from the ledger itself, so pin the ledger's key with `--public-key` from a copy obtained on your own.

Step 1. Start the ledger with the `eu` and `us` profiles:

```
export LEDGER_COMPLIANCE_PROFILES=eu,us
export LEDGER_COMPLIANCE_REGION=eea
```

Step 2. List the reports the profiles offer; the answer must show the five templates:

```
curl -s http://localhost:8090/api/v1/reports -H "Authorization: Bearer $LEDGER_API_KEY" | jq -r '.
[].id'
```

Step 3. Record an action and sign a checkpoint:

```
curl -s -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"payments-
agent","principalId":"alice","action":"tools/call","target":"bank/pay_invoice","inputHash":"9f86d0
81884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08","decision":"ALLOW","outcome":"SUCCEEDE
D"}' | jq -c '{sequence}'
curl -s -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -c '{sequence}'
```

Step 4. Create the EU AI Act report and its package; the answer must list the sections with their status:

```
curl -s -X POST http://localhost:8090/api/v1/reports/eu-ai-act-logging -H "Authorization: Bearer
$LEDGER_API_KEY" -H "Content-Type: application/json" -d '{"agentId":"payments-agent"}' >
report.json
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"payments-agent"}' > package.json
jq -r '.sections[] | "\(.reference): \(.status)"' report.json
```

Step 5. Verify the report against its package; the output must end with `Result: VALID` twice:

```
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar report --package package.json
report.json
```

Step 6. Create a dispute package and verify it with the ledger's key pinned; the output must show `Report : dispute (Dispute package)` and `Result: VALID`:

```
curl -s -X POST http://localhost:8090/api/v1/disputes -H "Authorization: Bearer $LEDGER_API_KEY" -
H "Content-Type: application/json" -d '{"agentId":"payments-agent","title":"Invoice 4711 paid
twice"}' -o dispute.zip
LEDGER_PUBLIC_KEY=$(curl -s http://localhost:8090/api/v1/keys -H "Authorization: Bearer
$LEDGER_API_KEY" | jq -r '.[] | select(.status == "ACTIVE") | .publicKey')
java -jar verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar dispute --public-key
"$LEDGER_PUBLIC_KEY" dispute.zip
```

Conformance Vectors

Conformance Vectors hold fixed inputs and outputs for every format the ledger produces, made with the published RFC 8032 test keys: canonical JSON, the evidence chain and checkpoints, Merkle roots and proofs, signed notes and cosignatures, SCITT statements and receipts, an SD-JWT VC mandate, a key-bound mandate with its KB-JWT, an HTTP message signature, a checkpoint that signs compliance profiles, an RFC 3161 token from a real timestamp authority, a Rekor entry from a real Rekor v2 log and a signed dispute report, an entry that names its model and prompt, and index proofs of presence and absence. Another implementation checks itself against them, and `ConformanceVectorsTest` in `ledger/verifier` fails when the ledger's own output changes.

API

All `/api/**`, `/mcp/**` and `/a2a/out/**` endpoints require `Authorization: Bearer {LEDGER_API_KEY}` or an agent API key, except `/api/v1/principal/**`, which requires a principal token from the OIDC provider and answers `403 FORBIDDEN` to everyone else. Operator-only endpoints answer `403 FORBIDDEN` to agents.

Method	Path	Description
POST	`/api/v1/evidence`	Record an entry with the fields the compliance profiles require; `201` with `Location`
POST	`/api/v1/evidence/batch`	Record up to 500 entries in order as one run, all or none; an item error names its `index`
GET	`/api/v1/evidence/{id}`	Get an evidence entry
GET	`/api/v1/evidence/{id}/statement`	The entry as a SCITT signed statement (`application/cose`)
GET	`/api/v1/evidence/{id}/receipt`	RFC 9942 receipt for the entry at `treeSize` , by default the latest log checkpoint
GET	`/api/v1/evidence`	List entries by `agentId` , `principalId` , `after` and `limit` (max 500)
GET	`/api/v1/ledger/head`	Current sequence and hash
POST	`/api/v1/checkpoints`	Sign the current head now; `409 LEDGER_EMPTY` when nothing is recorded (operator)
GET	`/api/v1/checkpoints/latest`	Latest checkpoint, with its RFC 3161 `timestamp` , `timestampedAt` and Rekor `rekorLogIndex` when set
GET	`/api/v1/checkpoints`	List checkpoints by `after` and `limit`
GET	`/api/v1/log/proofs/inclusion`	Inclusion proof for a `sequence` in the log at `treeSize` , by default the latest log checkpoint

Method	Path	Description
GET	`/api/v1/log/proofs/consistency`	Consistency proof from `firstSize` to `secondSize`, by default the latest log checkpoint
GET	`/api/v1/keys`	Signing keys with their status, rotation and revocation records
POST	`/api/v1/keys/{keyId}/revocation`	Revoke a retired key as compromised from `compromisedAt`, with a `reason` (operator)
POST	`/api/v1/packages`	Export a package by `agentId`, `principalId`, `fromSequence`, `toSequence`; `includePrompts` (operator)
POST	`/api/v1/packages/verify`	Verify a package with the verifier library, pinned to `publicKey` when given (operator)
GET	`/api/v1/reports`	The report templates of the active compliance profiles (operator)
POST	`/api/v1/reports/{id}`	A signed report over the package for the scope in the body; HTML with `format=html` (operator)
POST	`/api/v1/disputes`	A ZIP dispute package for the scope, `title` and `description` in the body (operator)
PUT	`/api/v1/policies`	Activate a Cedar policy set `text` and answer its `hash`, policy names and default effect (operator)
GET	`/api/v1/policies`	The active policy set (operator)
GET	`/api/v1/policies/{hash}`	A stored policy set by its hash (operator)
GET	`/api/v1/approvals`	Approvals by `status`, `agentId`, `principalId` and `limit` (operator, or the agent for its own)
GET	`/api/v1/approvals/{id}`	An approval with its status (operator or its agent)
POST	`/api/v1/approvals/{id}/approve`	Approve with an optional `reason` (operator, when the policy allows)
POST	`/api/v1/approvals/{id}/deny`	Deny with an optional `reason` (operator, when the policy allows)
GET	`/api/v1/principal/approvals`	The signed-in principal's approvals by `status` and `limit`
POST	`/api/v1/principal/approvals/{id}/approve`	Approve as the signed-in principal, with consent evidence
POST	`/api/v1/principal/approvals/{id}/deny`	Deny as the signed-in principal, with consent evidence
GET	`/api/v1/anomalies`	Anomalies by `status` (`OPEN`, `RESOLVED`), `agentId` and `limit` (operator)
GET	`/api/v1/anomalies/{id}`	An anomaly with its score and evidence (operator)
POST	`/api/v1/anomalies/detect`	Run the anomaly check now and answer the new anomalies (operator)
GET	`/api/v1/cluster`	This instance, whether it leads, and the instances seen recently (operator)
PUT	`/api/v1/prompts`	Store a prompt or configuration `text` and answer its `hash`; `201` when new (agent or operator)
GET	`/api/v1/prompts/{hash}`	A stored prompt text with when and by whom it was stored (operator)

Method	Path	Description
POST	`/api/v1/index/proofs`	A signed proof that a `term` occurs, with its count, or never does, up to a checkpoint (operator)
GET	`/api/v1/verification`	Full verification report (operator)
POST	`/api/v1/agents`	Register an agent and return its API key once (operator)
GET	`/api/v1/agents`	List agents by `after` and `limit` (operator)
GET	`/api/v1/agents/{agentId}`	Get an agent
POST	`/api/v1/agents/{agentId}/disable`	Disable an agent and its key (operator)
POST	`/api/v1/agents/{agentId}/key`	Issue a new API key; the old one stops working (operator)
PUT	`/api/v1/agents/{agentId}/signing-key`	Register the agent's Ed25519 `publicKey`; the agent sets it once, the operator can replace it
POST	`/api/v1/agents/{agentId}/track-record`	A signed SD-JWT VC of the agent's evidence counts up to the latest checkpoint (operator or the agent)
POST	`/api/v1/grants`	Create a grant (operator), or ask for one for itself (agent, only with OIDC)
GET	`/api/v1/grants`	List grants by `agentId`, `principalId`, `after` and `limit`
GET	`/api/v1/grants/{id}`	Get a grant with its status and uses
POST	`/api/v1/grants/{id}/revoke`	Revoke a grant with an optional `reason` (operator)
POST	`/api/v1/principals/{principalId}/erasure`	Erase the principal's personal data past retention, with an optional `reason` (operator)
POST	`/api/v1/legal-holds`	Place a legal hold on a `principalId` with a `reason` (operator)
GET	`/api/v1/legal-holds`	List legal holds, only the active ones with `active=true` (operator)
GET	`/api/v1/legal-holds/{id}`	Get a legal hold (operator)
POST	`/api/v1/legal-holds/{id}/release`	Release a legal hold with a `reason` (operator)
POST	`/api/v1/retention/sweep`	Erase what the profiles no longer allow and continue pending erasures now (operator)
GET	`/api/v1/principal/evidence`	The evidence about the principal by `after` and `limit`
GET	`/api/v1/principal`	The signed-in principal
GET	`/api/v1/principal/grants`	List the principal's grants by `state`, `after` and `limit`
GET	`/api/v1/principal/grants/{id}`	Get one of the principal's grants
POST	`/api/v1/principal/grants`	Create a grant on the principal's own behalf; returns `201`
POST	`/api/v1/principal/grants/{id}/approve`	Approve a pending grant
POST	`/api/v1/principal/grants/{id}/deny`	Deny a pending grant with an optional `reason`
POST	`/api/v1/principal/grants/{id}/revoke`	Revoke one of the principal's grants with an optional `reason`
POST	`/api/v1/decisions`	Decide an action for an agent and record it; returns `201`
GET	`/api/v1/decisions/{id}`	Get a decision and its outcome
POST	`/api/v1/decisions/{id}/outcome`	Report `SUCCEEDED` or `FAILED` once for an allowed decision

Method	Path	Description
POST	`/api/v1/mandates`	Issue a signed mandate from a grant; returns `201`
GET	`/api/v1/mandates`	List the mandates of a `grantId`
GET	`/api/v1/mandates/{id}`	Get a mandate, its token and status
POST	`/api/v1/mandates/{id}/revoke`	Revoke a mandate with an optional `reason` (operator)
GET	`/public/v1/keys`	Active and retired signing keys as a JWK set, no API key
GET	`/public/v1/mandates/status`	Signed mandate status list, no API key
GET	`/public/v1/log/checkpoint`	Latest log checkpoint as a signed note with its cosignatures, no API key
GET	`/public/v1/log/tile/{path}`	C2SP `tlog-tiles` hash tiles and entry bundles of the log, cached as immutable, no API key
GET	`/public/v1/log/key`	Verifier key of the log, no API key
GET	`/public/v1/witness/key`	Verifier key this ledger cosigns with as a witness, no API key
POST	`/public/v1/witness/add-checkpoint`	C2SP witness endpoint: cosign a watched log's checkpoint, `404` when it watches none, no API key
GET	`/public/v1/compliance`	Active compliance profiles with their digests and the combined rules, no API key
GET	`/public/v1/timestamp-authority`	The RFC 3161 timestamp authority and its certificate chain in PEM, no API key
GET	`/public/v1/rekor`	The Rekor log the ledger anchors checkpoints in and its verifier key, no API key
GET	`/public/v1/oidc`	The OIDC issuer and client for principal sign-in, `404` without OIDC
POST	`/mcp/{server}`	MCP gateway: decide, forward and record a `tools/call`; forward other messages
GET	`/mcp/{server}`	Relay the MCP server-to-client stream
DELETE	`/mcp/{server}`	Close an MCP session on the server
POST	`/a2a/out/{peer}`	A2A gateway: decide, attach a mandate and request proof, forward and record with the peer's receipt
POST	`/a2a/in/{agent}`	A2A inbound: verify the mandate and proof, forward to the agent, record and sign a receipt; no API key
GET	`/api/v1/a2a/exchanges/{id}`	Get an A2A exchange with its mandate, request proof and receipt
POST	`/api/v1/a2a/statements`	A signed statement of the exchanges with a `peer` in a window `from` to `to` (operator)
POST	`/api/v1/a2a/reconciliations`	Reconcile a peer's signed `statement` with this ledger's own and record the result (operator)
GET	`/actuator/health`	Health, open for probes

```
{
  "code": "INVALID_REQUEST",
  "message": "The evidence has invalid fields.",
  "details": {
    "fields": {
      "principalId": "must not be blank"
    }
  }
}
```

3.2 · ledger/chain/README.md

Nexusphere Ledger Chain

The evidence format and its cryptography, with no framework.

Purpose

Anyone who checks the ledger, including an auditor or a counterparty without the server, needs the exact rules for hashing and signing. This library is those rules in plain Java, shared by the server, the verifier and anyone who wants to verify evidence on their own.

Responsibilities

- Canonical JSON, so the same content always hashes to the same bytes
- `EvidenceEntry` with its content hash and link hash, and `EvidenceLink` for entries disclosed without their content
- `ChainVerifier`, which checks the hash chain from genesis or from an anchor
- Ed25519 keys and signatures (`SigningKeys`) and signed checkpoints (`Checkpoint` , `SignedCheckpoint`)
- Signed key rotations (`KeyRotation`) and the keys a pinned key reaches through them (`TrustedKeys`)
- `GrantTerms`, the hashed terms of a grant
- `MerkleTree`, the RFC 9162 tree hash with inclusion and consistency proofs and their verification
- `LogTiles`, C2SP `tlog-tiles` paths, hash tiles and entry bundles
- `LogCheckpoint` and `NoteKey`, C2SP signed-note checkpoints, `tlog-cosignature/v1` cosignatures and verifier keys
- `Cbor` and `CoseSign1`, deterministic CBOR and COSE_Sign1 with EdDSA, and `EvidenceStatement` and `LogReceipt`, the SCITT signed statement of an entry and its RFC 9942 inclusion receipt

Dependencies

Depends on	Used by
none	`mandate` , `server` , `verifier`

3.3 · ledger/mandate/README.md

Nexusphere Ledger Mandate

Mandates and the formats two organizations exchange, with a verifier SDK.

Purpose

A grant lives inside one ledger. When an agent acts at another organization, that organization needs a signed, self-contained proof of what the agent may do, and a way to check it without trusting the sender. This library is that proof, its revocation list and the verifier the receiving side uses.

Responsibilities

- Compact JWS signing and parsing with EdDSA (`Jws`) and Ed25519 keys as JWK (`Jwk`)
- `MandateClaims` and `Mandates` : the mandate issued from a grant as an SD-JWT VC, and `SdJwt` to parse it, check its disclosures and present it with fewer of them
- `StatusList` : the signed revocation list in the IETF Token Status List format
- `MandateVerifier` : trusted issuers, keys from the issuer's JWKS, signature, time, audience, coverage and revocation, failing closed
- `TrackRecordClaims` , `TrackRecords` and `TrackRecordVerifier` : the agent track record as an SD-JWT VC with the agent's evidence counts up to a checkpoint, its selective claims and key binding, and its verifier
- `ExchangeStatement` and `Reconciliation` : a ledger's signed statement of its A2A exchanges with a peer, and the comparison of two such statements request by request
- `ExchangeRequest` and `ExchangeReceipt` : the signed request proof and receipt of an A2A exchange between two ledgers

Dependencies

Depends on	Used by
`chain`	`server` , `verifier`

3.4 · ledger/server/README.md

Nexusphere Ledger Server

The ledger service: evidence, authorization and gateways for agents.

Purpose

This is the product that runs: a self-hosted service that records every agent action as tamper-evident evidence, decides whether an agent may act, and sits in front of MCP servers and A2A peers so nothing passes without a decision and a record.

Responsibilities

- Evidence API on an append-only SHA-256 hash chain in PostgreSQL, single or in batches, with signed checkpoints and full verification
- Agents with their own API keys, grants from principals to agents, and ALLOW or DENY decisions with outcomes
- Cedar policy sets that the operator activates and every decision must also pass, with the policy hash in evidence
- Human approval of decisions that policies hold back, by the operator, the principal or CIBA, with a webhook
- Anomaly detection per agent against its own baseline, recorded as evidence and alerted through Alertmanager
- Several instances on one database, with leader election by advisory lock for the scheduled work
- MCP gateway that decides and records every tool call
- Evidence packages with selective disclosure for auditors
- Transparency log over the evidence with inclusion and consistency proofs, cosigned by witnesses, served as C2SP `tlog-tiles` for archives, and a C2SP witness endpoint so the ledger can witness other logs
- SCITT signed statements and RFC 9942 receipts for every entry
- Signed mandates, the revocation status list and the public keys as JWKS
- Signing key history: rotation at startup with a cross-signed rotation record, retired keys kept for verification
- A2A gateway that sends and receives requests between two ledgers with evidence and signed receipts on both sides
- Principal sign-in with an OpenID Connect provider and principal consent on grants
- Health probes and Prometheus metrics for the chain head, checkpoints and recorded evidence

Dependencies

Depends on	Used by
`chain`, `mandate`, `verifier`	none

API

Path	Purpose
`/api/v1/**`	Evidence, checkpoints, log proofs, packages, agents, grants, decisions, mandates
`/mcp/{server}`	MCP gateway
`/a2a/out/{peer}`, `/a2a/in/{agent}`	A2A gateway
`/public/v1/**`	Public keys, status list, OIDC settings, log checkpoint and witness endpoint

Storage

Tables live in the `ledger` schema of PostgreSQL, created by the Flyway migrations in `src/main/resources/db/migration/ledger`.

Configuration, every endpoint and the formats are described in Nexusphere Ledger.

3.5 · ledger/verifier/README.md

Nexusphere Ledger Verifier

A command-line tool that checks ledger output without the ledger.

Purpose

Evidence is only worth something if someone else can check it. The verifier runs anywhere with Java and no server or database, so an auditor, a court or a counterparty can verify what a ledger handed them.

Responsibilities

- Verify an evidence package: checkpoint signatures, every hash link, and that each disclosed entry belongs to the chain
- Check the package's log checkpoint and the inclusion proof of every disclosed entry, and require witness cosignatures with `--witness` and `--witnesses-required`
- Check the SCITT statement and receipt of every disclosed entry, and a single statement and receipt with the `statement` command
- Verify a mandate with the `mandate` command: issuer, key, signature, time, audience, coverage and revocation, and the agent's key binding against `--nonce`, refusing unbound mandates with `--require-key-binding`
- Pin the ledger's public key with `--public-key`, print JSON with `--json`, and exit with 0 (valid), 1 (invalid) or 2 (usage error)
- Accept checkpoints signed by other keys of the ledger only when signed key rotations lead to them from the pinned key
- Apply key revocations from the package or from the key list given with `--keys`: a revoked key counts only where witness cosignatures prove that the log checkpoint predates its compromise
- Report the compliance profiles each checkpoint signs, by id and digest
- Check the RFC 3161 timestamp of the checkpoint, pin the authority's chain with `--timestamp-certificates` and refuse a checkpoint without one with `--require-timestamp`
- Check the Rekor entry of the checkpoint and its inclusion proof, pin the log with `--rekor-key` and refuse a checkpoint without an entry with `--require-rekor`
- Check a signed report against its package with `report --package`, and a whole dispute package with `dispute`
- Check an index proof with `index`: the term, the path to the signed index root and, with `--package`, that the index belongs to the package's checkpoint
- Check an agent track record with `track-record`: the issuer's signature, validity, audience, key binding and, with `--index-proof`, that a proof for the agent at the same checkpoint gives the same index root and counts
- Reconcile two ledgers' signed A2A statements with `reconcile`: each statement's signature, that they name each other over the same window, and every exchange one side holds that the other lacks or records differently

- Check entries in evidence format v2 from their personal values and salts, or from their commitments once erased
- Check that the prompt texts of a package match their hashes and are named by the model of a disclosed entry
- Check that the policy set texts of a package match their hashes and are named by a disclosed entry
- Mirror a ledger's C2SP `tlog-tiles` log with `archive`, and check the copy offline with `tiles`: the checkpoint signature, every entry, every tile and the root, and with `--package` every link of a package
- Run the Conformance Vectors in `ConformanceVectorsTest`

Dependencies

Depends on	Used by
`chain`, `mandate`	`server`

Usage is in Evidence Packages, SCITT Statements and Receipts, Verifying a Mandate, Reports and Dispute Packages and Log Archives.

3.6 · ledger/demo-mcp/README.md

Nexusphere Ledger Demo MCP

A small MCP server and a script that show the ledger end to end.

Purpose

The ledger is easiest to understand by watching it stop and record a real tool call. This module is a toy MCP server with harmless tools and the script that walks through the whole flow against the compose stack.

Responsibilities

- Serve the MCP tools `read_file`, `send_email` and `delete_file` over Streamable HTTP on port 8091
- `demo.sh`: register an agent, ask `alice` for a grant and approve it with a Keycloak sign-in, call tools through the gateway, see an allowed and a denied call, export a package and verify it, then show that a tampered package fails

Dependencies

Depends on	Used by
none	none

How to run it is in Demo.

3.7 · ledger/benchmarks/README.md

Nexusphere Ledger Benchmarks

Repeatable load scenarios against a running ledger, and the published results.

Purpose

Before an operator puts a ledger in front of agents, they need to know what it costs per call and where its limit is. The benchmark is a small Java program that drives a running ledger over its public API, so the same numbers can be measured on any machine and compared with the ones below.

Scenarios

Scenario	Request	What it measures
`record`	`POST /api/v1/evidence` with the agent's key	Appending one entry to the chain and the log
`decide`	`POST /api/v1/decisions` covered by a grant	A decision, which is also recorded as evidence
`gateway`	`tools/call` through `/mcp/{name}` to the demo MCP server	Decision, forwarding and the recorded outcome
`package`	`POST /api/v1/packages` over the last 100, 1,000 and 10,000	Exporting a package with proofs, statements and
	entries, after a checkpoint	receipts for every disclosed entry

Each load scenario registers its own agent and grant, warms up for `--warmup` seconds, then runs `--concurrency` clients in a closed loop for `--duration` seconds and reports the requests, the errors, the throughput and the latency percentiles. The package scenario signs a checkpoint, records exactly the number of entries to export and reports the median of three exports. `--json` also writes the report to a file. The exit code is `1` when any request failed.

Run

Step 1. Start PostgreSQL and build everything:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d ledger-postgresql
mvn install -DskipTests=true
```

Step 2. Start the demo MCP server and a ledger with the rate limits raised and the demo server named `demo`:

```
java -jar ledger/demo-mcp/target/demo-mcp-1.0.0-SNAPSHOT-exec.jar > demo-mcp.log 2>&1 &
LEDGER_PROFILES=postgresql,dev LEDGER_DATABASE_PORT=5433 LEDGER_RATE_LIMIT_PER_MINUTE=10000000
LEDGER_RATE_LIMIT_BURST=100000 \
  LEDGER_MCP_SERVERS_DEMO_URL=http://localhost:8091/mcp java -jar ledger/server/target/server-
1.0.0-SNAPSHOT-exec.jar > ledger.log 2>&1 &
until curl -s http://localhost:8090/actuator/health | grep -q UP; do sleep 1; done
```

Step 3. Run the load scenarios with 16 clients; the Errors column must be `0` and the exit code `0`:

```
java -jar ledger/benchmarks/target/benchmarks-1.0.0-SNAPSHOT-exec.jar --url http://localhost:8090
--api-key nexusphere-ledger-development-key-change-me --concurrency 16 --scenarios
record,decide,gateway --json bench-16.json; echo "exit $?"
```

```
Nexusphere Ledger benchmark against http://localhost:8090, 16 clients, 30 s per scenario
Scenario     Requests   Errors Per second    p50 ms    p95 ms    p99 ms    Max ms
record           9959        0      331.5     47.10     73.95     94.35    197.55
decide           7375        0      245.3     46.73    174.01    299.44    818.65
gateway          3698        0      122.8    106.40    271.50    387.00    844.54
exit 0
```

Step 4. Run the package scenario:

```
java -jar ledger/benchmarks/target/benchmarks-1.0.0-SNAPSHOT-exec.jar --url http://localhost:8090
--api-key nexusphere-ledger-development-key-change-me --scenarios package --package-sizes
100,1000,10000
```

Results

Measured on 2026-10-11 on one machine with 4 vCPU (Intel Xeon at 2.10 GHz) and 15 GB of memory, running OpenJDK 21.0.11, one ledger instance with a local Ed25519 key, PostgreSQL 18 in Docker and the demo MCP server, all on the same host. Each run lasted 30 seconds after a 10-second warm-up. The full report is in results/2026-10-11.json.

Scenario	Clients	Per second	p50 ms	p95 ms	p99 ms
`record`	1	257.7	3.47	6.10	9.82
`record`	16	331.5	47.10	73.95	94.35
`record`	64	322.5	192.83	326.58	397.73
`decide`	1	218.4	4.20	6.60	9.98
`decide`	16	245.3	46.73	174.01	299.44
`decide`	64	233.3	252.02	490.54	621.70
`gateway`	1	87.5	10.65	16.55	21.58
`gateway`	16	122.8	106.40	271.50	387.00
`gateway`	64	121.4	506.77	779.24	916.62

Package entries	Median ms	Size
100	229	0.3 MB
1,000	1,845	3.5 MB
10,000	18,107	36.3 MB

Every entry takes the lock on the chain head, so appends run one after another and the ledger reached about 330 entries a second on this machine whatever the number of clients; more clients only wait longer. A decision is one entry, and a gateway call is two, the decision and its outcome, so their ceilings follow from the same limit. One client sees the cost of a single call: about 3.5 ms to record, 4 ms to decide and 11 ms through the gateway. Several instances, as in High

Availability, share one chain and therefore the same append limit; they add availability, not write throughput. A package costs about 1.8 ms and 3.6 KB per disclosed entry, because each one carries its inclusion proof, its SCITT statement and its receipt.

Dependencies

Depends on	Used by
none	none

3.8 · ledger/conformance/README.md

Nexusphere Ledger Conformance Vectors

Fixed inputs and outputs for every format the ledger produces.

Purpose

A verifier written in another language, or another ledger, can prove it reads and writes the same formats by running these vectors. The ledger runs them too: `ConformanceVectorsTest` in `ledger/verifier` rebuilds every vector and fails when its output differs, so a format never changes by accident.

Keys

The vectors are signed with the published RFC 8032 Ed25519 test keys: test 1 is the ledger key and test 2 the witness key. Ed25519 signatures are deterministic, so the same input always gives the same bytes. The keys are public; never use them for real evidence.

Vectors

File	Contents
`keys.json`	Seeds, PKCS#8 and X.509 keys, the key ID and the C2SP verifier keys of the log and the witness
`canonical-json.json`	An object and its canonical JSON with sorted keys, no whitespace and escaped text, and its SHA-256
`evidence-chain.json`	Three entries with values, salts, commitments, canonical content and hashes, and a signed head checkpoint
`evidence-model.json`	An entry that names its model, version, prompt hash and config hash, with the two texts and its hashes
`a2a-reconciliation.json`	Signed A2A statements of two ledgers with one matched, one undelivered and one never sent exchange
`agent-track-record.json`	An agent track record that matches the agent's index proof, and a presentation that discloses only the outcomes
`index-proof.json`	A signed index root over the three entries with proofs that two terms occur and two never do
`merkle-tree.json`	RFC 9162 roots, inclusion proofs and consistency proofs for every size up to 8
`signed-note.json`	The C2SP signed-note log checkpoint over the three entries and its `tlog-cosignature/v1` cosignature
`key-history.json`	A rotation from test key 1 to test key 2, and the revocation of key 1 signed by key 2
`scitt.json`	COSE_Sign1 SCITT statements and RFC 9942 receipts for the three entries, in hex
`mandate.json`	An SD-JWT VC mandate, the same mandate presented with only the grant disclosed, and its disclosed claims
`mandate-key-binding.json`	A mandate bound to RFC 8032 test key 3 with `cnf`, its KB-JWT presentation, nonce and `sd_hash`

File	Contents
`http-signature.json`	An RFC 9421 signature of a POST with its `Content-Digest`, `Signature-Input`, `Signature` and signature base
`compliance-checkpoint.json`	A compliance profile with its canonical JSON and digest, and a `checkpoint/v2` that signs it with another profile
`timestamp.json`	An RFC 3161 response and token over the evidence-chain checkpoint, with the authority's chain and time
`rekor.json`	An Ed25519ph signature of the evidence-chain checkpoint, its Rekor v2 request and the entry the log returned
`report.json`	A dispute report over the three entries, its canonical signed bytes, entries digest and signature by test key 1

SD-JWT disclosures carry random salts, so `mandate.json` and the token in `mandate-key-binding.json` are checked rather than rebuilt: their signatures, digests and claims must verify. The KB-JWT is deterministic, so the SDKs rebuild the key-bound presentation from the agent seed and compare it. `timestamp.json` comes from a Sigstore timestamp authority whose keys are not kept, so it is never rebuilt: the token must be for the SHA-256 of `signedBytes`, verify against `certificates` and give `genTime` and `authority`, and only the root of the chain is not enough to trust it. `rekor.json` comes from a Rekor v2 log whose key is not kept: the signature and request are rebuilt from test key 1, and the entry must verify against `rekorVerifierKey`.

Checked Elsewhere

The vectors were also checked with other implementations: the Merkle roots, canonical JSON, content hashes and checkpoint signature with Python's `hashlib` and `cryptography`; the signed note with Go's `golang.org/x/mod/sumdb/note`; the cosignature, COSE signatures and SD-JWT digests with Python's `cryptography` and `cbor2`; the timestamp response with `openssl ts -verify`; the Rekor entry by the Rekor v2 log that accepted it. The Python SDK and the TypeScript SDK rebuild every vector except the mandate in their own test suites and check the mandate.

Regenerate

Change a vector only together with a deliberate format change, and record it in the changelog.

```
mvn -pl ledger/verifier -am test -Dtest=ConformanceVectorsTest -Dconformance.write=true -
Dsurefire.failIfNoSpecifiedTests=false
```

3.9 · ledger/CHANGELOG.md

Nexusphere Ledger Changelog

1.0.0 (not released yet)

First release of the Nexusphere Ledger, a self-hosted evidence layer for AI agent actions that sits beside existing agent platforms and gateways.

- Evidence entries in a SHA-256 hash chain with append-only PostgreSQL storage and Ed25519-signed checkpoints
- Agents with their own API keys, grants from principals to agents, and ALLOW/DENY decisions with reason codes and outcome reports, all recorded as evidence
- MCP gateway that decides every `tools/call`, forwards only allowed calls and records their outcome
- Evidence packages with selective disclosure and the offline verifier command-line tool
- Signed mandates (EdDSA JWS) issued from grants, a signed revocation status list and the ledger keys as a JWK set on public endpoints, with issue and revoke recorded as evidence
- `MandateVerifier` SDK that checks a mandate's issuer, key, signature, time, audience, coverage and revocation, and the `mandate` command of the verifier jar
- A2A gateway between two ledgers: outbound requests are decided and carry a mandate and a signed request proof, inbound requests are verified, recorded and answered with a signed receipt, so both sides hold matching evidence
- Signing key rotation at startup with a rotation record signed by the old and the new key; retired keys stay published, so everything they signed still verifies, and the offline verifier follows rotations from a pinned key
- Grant uses are returned when a gateway sees that the call never ran, and a receiving ledger counts the uses of each mandate on its own
- Streaming in both gateways: MCP answers, server requests and the `GET` stream, and A2A `message/stream` and `tasks/resubscribe` are relayed live, with the A2A receipt sent as the last event and covering every event
- Principal login with any OpenID Connect provider: principals approve, deny, create and revoke their own grants, grants from the operator or an agent wait for that consent, and every consent is evidence tied to the sign-in
- Web UI (React and TypeScript) for operators and principals: evidence, agents, grants, mandates, packages, A2A exchanges, ledger verification, and principal sign-in with PKCE to approve, deny and revoke grants
- Evidence batches: up to 500 entries recorded in order in one call, all or none
- Python SDK (`nexusphere-ledger`): a client for evidence, decisions, grants, mandates and packages, and an offline verifier for packages, statements, receipts and mandates that passes every conformance vector, with the `nexusphere-ledger-verify` command
- TypeScript SDK (`@nexusphere/ledger`): the same client and offline verifier for Node 20 and browsers on Web Crypto, with no runtime dependencies and the same `nexusphere-ledger-verify` command
- OpenTelemetry Collector exporter `nexusphere` : agent tool call and agent invocation spans that follow the GenAI semantic conventions become evidence in batches, with retries and a

sending queue, plus a ready Collector image; MCP spans with a JSON-RPC error are recorded as `FAILED`

- Adapters that record or decide tool calls: a LangGraph callback handler and `govern` wrapper, OpenAI Agents SDK run hooks with a tool input guardrail, an Amazon Bedrock AgentCore Gateway interceptor for Lambda, and an agentgateway tracing configuration with an end-to-end check
- Signing providers: the ledger signs through a local key or a HashiCorp Vault transit key that never leaves Vault; a Vault key rotation becomes an endorsed ledger rotation, and a move from a local key to Vault is endorsed too
- Key revocation with a compromise time, signed by the active key and recorded as evidence; the verifiers in Java, Python, TypeScript and the browser accept a revoked key only where witness cosignatures prove the log checkpoint came before the compromise, and a new `key-history.json` conformance vector covers rotation and revocation
- Agent signing keys: an agent registers its own Ed25519 key, mandates name it in `cnf`, and the agent presents them with an SD-JWT key binding (KB-JWT) for the audience and the request body hash; the A2A gateway and the receiving ledger refuse a bound mandate without a valid key binding, and the verifiers in Java, Python and TypeScript check it, with a `mandate-key-binding.json` conformance vector
- RFC 9421 HTTP message signatures with an RFC 9530 `Content-Digest` on every request the ledger sends to MCP servers, peer ledgers and A2A agents; the receiving ledger requires them, and the Java, Python and TypeScript SDKs sign and verify them, with an `http-signature.json` conformance vector
- Compliance profiles: retention, erasure, region, required evidence fields, accepted timestamp authorities and report templates per jurisdiction in JSON files, with `baseline`, `eu` and `us` built in and your own added from a directory; the strictest rule of the active profiles wins, required fields are enforced, a disallowed region stops the ledger, checkpoints sign the active profiles in format `checkpoint/v2`, and the verifiers report them, with a `compliance-checkpoint.json` conformance vector
- Evidence format v2 with crypto-shredding: the hash covers salted commitments of the personal fields instead of their values, which the ledger keeps encrypted under a key per principal; `POST /api/v1/principals/{id}/erasure` deletes them past the retention of the active profiles and pseudonymizes the principal in grants, decisions, mandates and exchanges, while the chain, log, checkpoints and packages stay valid; format v2 replaces v1, so a ledger database with v1 evidence must be recreated
- Retention and legal holds: a profile's retention rule can set a `maximum`, the ledger erases personal data past the shortest maximum on a schedule and completes pending erasures once their retention ends, and a legal hold on a principal stops both until it is released; holds, expiries and erasures are recorded as evidence, and the web UI has a Privacy page for them
- RFC 3161 checkpoint timestamps: with `LEDGER_TIMESTAMP_URL` set, a timestamp authority timestamps every signed checkpoint, later when it is down; packages carry the token, the verifiers in Java, Python, TypeScript and the browser check it offline against a pinned chain with `--timestamp-certificates` and `--require-timestamp`, the profiles' `timestampAuthorities` decide which authority is accepted, and a `timestamp.json` conformance vector holds a token from a Sigstore timestamp authority
- Rekor anchoring: with `LEDGER_REKOR_URL` and `LEDGER_REKOR_KEY` set, the ledger writes every signed checkpoint to a Rekor v2 log as a `hashedrekord` entry signed with Ed25519ph, later when the log is down, and publishes the log at `GET /public/v1/rekor`; packages carry the

entry, the verifiers in Java, Python, TypeScript and the browser check its inclusion proof offline against a pinned log with `--rekor-key` and `--require-rekor`, the web UI shows the entry, and a `rekor.json` conformance vector holds an entry from a real Rekor v2 log

- Reports and dispute packages: each compliance profile's report templates (`dispute`, `eu-ai-act-logging`, `dora-ict-records`, `finra-supervision`, `sec-17a-4-records`) map the evidence of a package to requirements as a signed `nexusphere-ledger/report/v1` report in JSON or HTML, `POST /api/v1/disputes` answers a ZIP with the package, the dispute report, keys and profiles, the verifiers in Java, Python and TypeScript check reports and dispute packages with `report` and `dispute`, the web UI has a Reports page, and a `report.json` conformance vector holds a signed report
- Model and prompt in evidence: an entry names the model, version, prompt hash and config hash the agent ran, covered by the content hash, sent through the gateways as `X-Ledger-Model*` headers and through the OpenTelemetry exporter, and required by a profile when it lists them; `PUT /api/v1/prompts` stores prompt texts by hash, packages, reports and disputes carry them with `includePrompts`, every verifier checks them, the EU AI Act and dispute reports have a model section, and an `evidence-model.json` conformance vector holds an entry with its model
- Index proofs: a sparse Merkle tree over agents, actions, inputs and the actions each grant allowed, with a root the ledger signs per checkpoint; `POST /api/v1/index/proofs` proves that a term occurs, with its count, or that it never did up to a checkpoint, the verifiers in Java, Python and TypeScript check proofs with `index`, and an `index-proof.json` conformance vector holds proofs of presence and absence
- Agent track records: `POST /api/v1/agents/{agentId}/track-record` issues an SD-JWT VC that counts the agent's evidence up to a checkpoint, with outcomes, principals and actions the agent may withhold, bound to the agent's key and tied to the signed index root; `TrackRecordVerifier`, the verifiers in Python and TypeScript and the `track-record` command check it and cross-check it with an index proof, and an `agent-track-record.json` conformance vector holds one
- A2A reconciliation: each ledger signs a statement of its exchanges with a peer over a window, and `POST /api/v1/a2a/reconciliations` compares the peer's statement with its own request by request, reporting requests one side never received, never sent or recorded differently, and records the result as evidence; the `reconcile` command and the Python and TypeScript SDKs compare two statements offline, and an `a2a-reconciliation.json` conformance vector holds a pair
- Cedar policies: the operator activates a policy set that every decision must also pass, with a default effect of permit or deny; a denial is `POLICY_DENIED`, each decision records the policy set hash and the deciding policies, and packages carry the policy texts that the verifier and SDKs check against their hashes
- Human approval: a forbid policy annotated `@approval` turns its denial into a request that the principal or the operator answers in the ledger, the principal also on their device through OpenID Connect CIBA, announced by a signed webhook; an approved request is allowed once, and a request nobody answers in time is denied
- Anomaly alerts: each agent's volume, denial share and failure share are compared with its own baseline, new anomalies are recorded as evidence and exported as `ledger_anomaly_score`, and Alertmanager delivers `LedgerAgentAnomaly`
- High availability: several instances share one PostgreSQL and one chain, and the scheduled work runs only on the leader, the instance holding a PostgreSQL advisory lock, which another

instance takes over when it stops
- Log archives: the transparency log is served as C2SP `tlog-tiles` with immutable caching, the verifier's `archive` command mirrors it incrementally and its `tiles` command checks the copy and a package against it offline
- Published benchmarks: load scenarios for recording, decisions, the MCP gateway and package export, with the results of a reference machine
- The demo MCP server sends its answers without the 40 ms Nagle delay that slowed every call through the gateway
- Browser verifier: one self-contained HTML page in React and TypeScript that verifies a package with the TypeScript SDK, pinned to the ledger key and optional witnesses, with a Content Security Policy that blocks every connection
- Secrets and signing keys from files written by a secret manager, Prometheus metrics with alert rules, and a tested backup and restore procedure
- Rate limit per caller with 429 and `Retry-After`, and alert delivery by email, Slack, Telegram or webhook, each turned on by setting its address
- Transparency log: every entry is a leaf of an RFC 9162 Merkle tree, log checkpoints are C2SP signed notes, and inclusion and consistency proofs are served by the API and carried in evidence packages
- Witness cosigning with the C2SP `tlog-witness` protocol: the ledger asks configured witnesses to cosign each log checkpoint, any ledger can witness other logs, and the verifier can require cosignatures from given witnesses
- IETF SCITT signed statements (COSE hash envelopes) for every entry and RFC 9942 receipts from the transparency log, served by the API, carried in evidence packages and checked by the verifier's `statement` command
- Mandates are IETF SD-JWT VCs (`dc+sd-jwt`): the principal, grant and terms hash are selectively disclosable, the limits are not, and the documentation maps them to AP2 and Verifiable Intent
- Conformance vectors for every format, signed with the RFC 8032 test keys and checked by a test
- Dockerfiles, compose file with Adminer, Keycloak, Prometheus, Alertmanager and the web UI, Kubernetes manifest, demo MCP server and demo script

PART 4

SDKs and Integrations

Client libraries and offline verifiers for other languages, checked against the same conformance vectors, the OpenTelemetry Collector exporter, the adapters for agent frameworks and gateways, and the browser verifier.

4.1 · ledger/sdk/python/README.md

Nexusphere Ledger Python SDK

Record evidence from a Python agent and verify everything the ledger produces, offline.

Purpose

`nexusphere-ledger` lets a Python agent, a gateway hook or a platform record evidence in the Nexusphere Ledger, and lets an auditor or a counterparty check what a ledger handed them without Java and without the server. The verifier is a second implementation of the formats, not a wrapper: it passes every Conformance Vector and rebuilds the signed checkpoints, notes, cosignatures, COSE statements and receipts byte for byte. Its only dependency is `cryptography`.

Install

Step 1. Check Python; the version must be 3.10 or later:

```
python3 --version
```

Step 2. Install the SDK from the repository:

```
python3 -m pip install ledger/sdk/python
```

Step 3. Check the install; it must print `1.0.0`:

```
python3 -c "import nexusphere_ledger; print(nexusphere_ledger.__version__)"
```

Record Evidence

These steps need a running ledger, as in the Quick Start, and an agent API key from `POST /api/v1/agents`. `input` and `output` are hashed on the client with SHA-256; only the hash reaches the ledger.

Step 1. Register an agent with the operator key and keep its API key:

```
from nexusphere_ledger import LedgerClient

operator = LedgerClient("http://localhost:8090", "nexusphere-ledger-development-key-change-me")
agent_key = operator.register_agent("invoice-agent", "Invoice agent", "acme")["apiKey"]
```

Step 2. Record what the agent did; the answer carries the `sequence` and `hash` of the new entry:

```
agent = LedgerClient("http://localhost:8090", agent_key)
entry = agent.record("invoice-agent", "acme", "tools/call", "SUCCEEDED", target="read_invoice",
                     decision="ALLOW", input={"invoice": 7}, output="invoice text")
print(entry["sequence"], entry["hash"])
```

Step 3. Record many entries in one call, all or none:

```
agent.record_batch([
    {"agentId": "invoice-agent", "principalId": "acme", "action": "tools/call", "outcome":
"SUCCEEDED"},
    {"agentId": "invoice-agent", "principalId": "acme", "action": "tools/call", "outcome":
"FAILED"},
])
```

Decide and Report

When the ledger decides, `act` asks for a decision, raises `Denied` when no grant covers the action, and reports the outcome when the block ends: `SUCCEEDED`, or `FAILED` with the exception name.

Step 1. Grant the agent the tools, as the operator:

```
from datetime import datetime, timedelta, timezone

operator.create_grant("alice", "invoice-agent", ["tools/call"], ["read_*"],
                      datetime.now(timezone.utc) + timedelta(days=30), max_uses=100)
```

Step 2. Act under the grant:

```
from nexusphere_ledger import Denied

try:
    with agent.act("alice", "tools/call", "read_invoice", input={"invoice": 7}) as action:
        text = "invoice text"
        action.output(text)
except Denied as denied:
    print(denied.decision["reasonCode"])
```

Step 3. Check the outcome; it must print `SUCCEEDED`:

```
print(action.reported["outcome"])
```

When a policy annotated `@approval` stops the action, `act` raises `Denied` with reason code `APPROVAL_PENDING` and the request in `denied.approval_id`; `approvals`, `approval`, `approve` and `deny_approval` list, read and resolve approval requests, and once approved the identical action is allowed once with reason code `ALLOWED_BY_APPROVAL`. `anomalies`, `anomaly` and `detect_anomalies` list, read and trigger the operator's anomaly checks on agent volume, denials and failures.

Record the Model and Prompt

An entry can name the model and prompt the agent acted with: `model` is a dict with any of `name`, `version`, `promptHash` and `configHash`, the two hashes being lowercase hex SHA-256. The model is part of the content hash of the entry. The prompt text itself stays out of the chain; the ledger keeps it apart, and a package carries it only when asked for.

Step 1. Store the system prompt; the answer is its hash, the same for the same text:

```
prompt_hash = agent.store_prompt("You read the invoices of Acme. Ask before paying anything.")
```

Step 2. Record evidence and act under a grant with the model:

```
model = {"name": "claude-sonnet-5-5", "version": "2026-09-01", "promptHash": prompt_hash}
agent.record("invoice-agent", "acme", "tools/call", "SUCCEEDED", target="read_invoice",
model=model)
with agent.act("alice", "tools/call", "read_invoice", model=model) as action:
    action.output("invoice text")
```

Step 3. Read the prompt back, as the operator; it must print the stored text:

```
print(operator.prompt(prompt_hash)["text"])
```

Step 4. Export a package with the prompt texts and verify it; `disclosed_prompts` must be `1`:

```
from nexusphere_ledger import verify_package

package = operator.export_package(agent_id="invoice-agent", include_prompts=True)
report = verify_package(package, operator.active_public_key())
print(report.valid, report.disclosed_prompts)
```

`decide` and `record_batch` items take `model` too, and `create_report` and `create_dispute` take `include_prompts=True`. The verifier checks that each prompt text in a package hashes to its hash and that a disclosed entry names that hash as its `promptHash` or `configHash`. The operator activates a Cedar policy set with `client.activate_policies(text)` and reads it back with `client.active_policies()` and `client.policy_set(hash)`; the verifier checks each policy set text in a package the same way, against the `policy.hash` attribute of a disclosed entry.

Verify a Package

Step 1. Export a package and read the ledger's public key, as the operator:

```
package = operator.export_package(agent_id="invoice-agent")
public_key = operator.active_public_key()
```

Step 2. Verify it pinned to that key; `valid` must be `True`:

```
from nexusphere_ledger import verify_package

report = verify_package(package, public_key)
print(report.valid, report.disclosed_entries, report.problems)
```

Pass witness verifier keys as the third argument and the number of cosignatures needed as the fourth, as with the Java verifier. A key in the package is trusted only when it is the pinned key or signed key rotations lead to it from the pinned key. Pass `key_list` with the list from `GET /api/v1/keys` to apply key revocations, as the Ledger README describes; the command takes it as `--keys keys.json`. `report.to_dict()` has the same fields as the Java `PackageReport`. `report.compliance_profiles` lists the id and digest of each compliance profile the checkpoint signs, and `client.compliance()` returns the active profiles and their combined rules from the ledger. An entry whose personal fields were erased still verifies from its commitments, and `client.erase_principal(principal_id, reason)` asks the ledger to erase a principal. `place_legal_hold`, `release_legal_hold`, `legal_holds` and `sweep_retention` manage legal holds and the retention pass. `timestamp_certificates`, a list of certificates or PEM text, pins the authority of the checkpoint timestamp, `require_timestamp=True` refuses a checkpoint without one, and

`report.timestamped_at` and `report.timestamp_authority` say when and by whom it was timestamped. `rekor_keys`, a list of verifier keys, pins the log of the Rekor entry, `require_rekor=True` refuses a checkpoint without one, and `report.rekor_log_index` and `report.rekor_origin` say where it was anchored. `report.disclosed_prompts` counts the prompt texts that match the hashes their entries name. `report.disclosed_policies` counts the policy set texts that match the hashes their entries name.

`client.reports()` lists the reports the ledger offers, `client.create_report(report_id, agent_id=...)` returns one signed by the ledger, or its HTML with `html=True`, and `client.create_dispute(title, description, agent_id=...)` returns the bytes of a dispute package zip. `verify_report(report, package, public_key)` checks the signature of a report with the keys the package is verified with, and that it was made for the checkpoint, scope and disclosed entries of that package; `check.valid`, `check.statuses` and `check.problems` give the result.

Verify a Statement and Receipt

```
from nexusphere_ledger import EvidenceStatement, LogReceipt, PublicKey

key = PublicKey.from_base64(public_key)
statement = EvidenceStatement.parse(agent.statement(entry["id"]))
receipt = LogReceipt.parse(agent.receipt(entry["id"]))
print(statement.verify(key), receipt.verify(statement.leaf_hash, key))
```

Prove That Something Never Happened

An index proof shows how often a term occurs in the evidence up to a checkpoint, or that it never occurs. A term is an `agent` by `agentId`, an `action` by `agentId` and `action`, an `input` by `inputHash`, or a `grant` by `delegationId`. `verify_index_proof` follows the path of the proof to the root of the index and checks the ledger's signature on that root with the keys the proof carries, trusted from the pinned key as in a package. Pass `key_list` to apply key revocations and `package` to check that the index belongs to the checkpoint of that package.

Step 1. Record an action with a known input and sign a checkpoint, as in Record Evidence:

```
from nexusphere_ledger import hash_of

input_hash = hash_of({"invoice": 7})
agent.record("invoice-agent", "acme", "tools/call", "SUCCEEDED", target="pay_invoice",
input_hash=input_hash)
checkpoint = operator.create_checkpoint()
```

Step 2. Ask, as the operator, for a proof that the input was recorded and for a proof that the agent never deleted anything; `sequence` picks the checkpoint and defaults to the latest:

```
recorded = operator.index_proof({"type": "input", "inputHash": input_hash},
checkpoint["sequence"])
never = operator.index_proof({"type": "action", "agentId": "invoice-agent", "action":
"files/delete"})
```

Step 3. Verify both pinned to the ledger's key; it must print `True True` with a count of at least 1, then `True False`:

```
from nexusphere_ledger import verify_index_proof

public_key = operator.active_public_key()
present = verify_index_proof(recorded, public_key)
absent = verify_index_proof(never, public_key)
print(present.valid, present.present, present.count, present.first_sequence,
present.last_sequence)
print(absent.valid, absent.present, absent.sequence, absent.problems)
```

Verify a Mandate

`MandateVerifier` reads the issuer's keys from `/public/v1/keys` and its revocation status list, and caches both for five minutes. Problems carry the same codes as the Java SDK: `MALFORMED`, `WRONG_TYPE`, `UNTRUSTED_ISSUER`, `UNKNOWN_KEY`, `BAD_SIGNATURE`, `NOT_YET_VALID`, `EXPIRED`, `WRONG_AUDIENCE`, `NOT_COVERED`, `REVOKED`, `STATUS_UNAVAILABLE`, `KEY_BINDING_MISSING`, `KEY_BINDING_INVALID` and `KEY_NOT_BOUND`.

```
from nexusphere_ledger import MandateVerifier

verifier = MandateVerifier("http://localhost:8090", audience="https://supplier.example")
check = verifier.verify_action(token, "a2a/send", "supplier/sales")
print(check.valid, check.claims.agent_id, [p.code for p in check.problems])
```

Present a Key-Bound Mandate

An agent with its own key registers the public half with `set_signing_key`; mandates issued afterwards name it in `cnf`. `present_bound` adds a KB-JWT signed by the agent for an audience and a nonce, and `verify_bound` checks it. For the A2A gateway the nonce is the hex SHA-256 of the request body.

```
from nexusphere_ledger import MandateVerifier, PrivateKey, hash_of, present_bound

key = PrivateKey.generate()
agent.set_signing_key("invoice-agent", key.public_key)
token = agent.issue_mandate(grant_id, "https://supplier.example")["token"]
presented = present_bound(token, key, "https://supplier.example", hash_of(body))
check = MandateVerifier("http://localhost:8090",
audience="https://supplier.example").verify_bound(presented, hash_of(body))
print(check.valid, check.claims.holder_key.key_id)
```

Verify an Agent Track Record

An agent track record is an SD-JWT VC that counts the evidence naming an agent up to a signed checkpoint, with its outcomes, principals and actions disclosed selectively. `TrackRecordVerifier` checks it like a mandate, with the same problem codes, but has no status list. `match_track_record` checks it against an index proof of the `agent` term at the same checkpoint and returns what does not match.

Step 1. Sign a checkpoint and let the agent ask for its track record for a supplier, as in Record Evidence:

```
operator.create_checkpoint()
issued = agent.issue_track_record("invoice-agent", "https://supplier.example", "P7D")
```

Step 2. Present only the outcomes, and ask, as the operator, for the index proof of the agent at the same checkpoint:

```
from nexusphere_ledger import SdJwt

presented = SdJwt.parse(issued["credential"]).present({"outcomes"})
proof = operator.index_proof({"type": "agent", "agentId": "invoice-agent"}, issued["checkpoint"])
```

Step 3. Verify both pinned to the ledger's key; it must print `True`, the total, the outcome counts and `[]`, then `None` and `[]`:

```
from nexusphere_ledger import PublicKey, StaticKeys, TrackRecordVerifier, match_track_record,
verify_index_proof

public_key = operator.active_public_key()
keys = StaticKeys.fixed("http://localhost:8090", PublicKey.from_base64(public_key))
check = TrackRecordVerifier("http://localhost:8090", keys=keys,
audience="https://supplier.example").verify(presented)
print(check.valid, check.claims.total, check.claims.outcomes, check.problems)
print(check.claims.actions)
print(match_track_record(check.claims, proof, verify_index_proof(proof, public_key)))
```

Reconcile Two Ledgers

Two ledgers that exchange A2A messages can reconcile what each recorded. `a2a_statement` asks a ledger, as the operator, for its signed statement of the exchanges with a peer in a window of at most 31 days, and `reconcile_a2a` has the peer's ledger compare it with its own and record the result. `verify_statement` checks a statement against the issuer's key, and `reconcile` compares two statements offline with the same rules: an exchange is `NOT_RECEIVED`, `NOT_SENT` or a `MISMATCH` with the fields that differ.

Step 1. Run the two ledgers of Reconciling Two Ledgers through its Step 4, so that the ledger on port 8090 has sent two orders to its peer `supplier` on port 8092, and connect to both as the operator:

```
from nexusphere_ledger import LedgerClient

buyer = LedgerClient("http://localhost:8090", "nexusphere-ledger-development-key-change-me")
supplier = LedgerClient("http://localhost:8092", "nexusphere-ledger-development-key-change-me")
```

Step 2. Let the buyer's ledger sign its statement of the exchanges with the supplier from an hour ago to an hour from now; it must print `2`:

```
from datetime import datetime, timedelta, timezone

now = datetime.now(timezone.utc).replace(microsecond=0)
signed = buyer.a2a_statement("supplier", now - timedelta(hours=1), now + timedelta(hours=1))
print(signed["exchanges"])
```

Step 3. Let the supplier's ledger reconcile it with its own; it must print `True 2 []`:

```
done = supplier.reconcile_a2a(signed["statement"])
print(done["reconciled"], done["matched"], done["discrepancies"])
```

Step 4. Verify both statements pinned to each ledger's key and reconcile them offline; it must print `True 2 0 []`:

```
from nexusphere_ledger import PublicKey, StaticKeys, reconcile, verify_statement

ours = verify_statement(signed["statement"], StaticKeys.fixed("http://localhost:8090",

PublicKey.from_base64(buyer.active_public_key())))
theirs = verify_statement(done["statement"], StaticKeys.fixed("http://localhost:8092",

PublicKey.from_base64(supplier.active_public_key())))
r = reconcile(ours, theirs)
print(r.reconciled, r.matched, r.undelivered, r.discrepancies)
```

Check a Signed Request

Every request the ledger forwards to an MCP server or an A2A agent carries an RFC 9421 HTTP message signature. `verify_request` checks it with the ledger's published keys and returns the key ID, `created` and `nonce`; it raises `InvalidHttpSignature` when the signature is missing, stale or does not match the method, address or body. `sign_request` makes the same headers.

```
from nexusphere_ledger import JwksKeys, verify_request
from nexusphere_ledger.mandate import http_fetcher

keys = JwksKeys(http_fetcher())
verified = verify_request("POST", "http://files-mcp:3000/mcp", request_headers, body,
                          lambda kid: keys.resolve("http://localhost:8090", kid))
print(verified.key_id, verified.nonce)
```

Command Line

The install adds `nexusphere-ledger-verify` with the same arguments, output and exit codes (0 valid, 1 invalid, 2 usage error) as the Java verifier jar.

Step 1. Verify a package pinned to the ledger's key; the last line must be `Result: VALID`:

```
nexusphere-ledger-verify --public-key "$(curl -s http://localhost:8090/api/v1/keys -H
'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[] | select(.status
== "ACTIVE") | .publicKey')" package.json
```

Step 2. Verify a statement with its receipt:

```
nexusphere-ledger-verify statement --public-key "{publicKey}" --receipt receipt.cose
statement.cose
```

Step 3. Verify a mandate for an action:

```
nexusphere-ledger-verify mandate --issuer http://localhost:8090 --action a2a/send --target
supplier/sales "{token}"
```

Step 4. Verify a key-bound presentation against its nonce; the output must show `Agent key` and end with `Result: VALID`:

```
nexusphere-ledger-verify mandate --issuer http://localhost:8090 --audience
https://supplier.example --nonce "{nonce}" "{presentation}"
```

Step 5. Verify a package from a ledger with checkpoint timestamps, pinned to the authority's chain; the output must show `Timestamp : ... (pinned)` and `Result: VALID`:

```
nexusphere-ledger-verify --timestamp-certificates tsa-chain.pem --require-timestamp package.json
```

Step 6. Verify a package from a ledger with Rekor anchoring, pinned to the log's verifier key; the output must show `Rekor : entry ... (pinned)` and `Result: VALID`:

```
nexusphere-ledger-verify --rekor-key "$REKOR_KEY" --require-rekor package.json
```

Step 7. Verify a dispute package with its signed report; the output must show `Nexusphere Ledger report` and `Result: VALID`:

```
nexusphere-ledger-verify dispute --public-key "$LEDGER_PUBLIC_KEY" dispute.zip
```

Step 8. Verify the package of Record the Model and Prompt, saved as `package.json`; the output must show `Prompts : 1 prompt texts match the hashes their entries name` and `Result: VALID`:

```
nexusphere-ledger-verify --public-key "$LEDGER_PUBLIC_KEY" package.json
```

Step 9. Save the proof of absence from Prove That Something Never Happened with `json.dump(never, open("proof.json", "w"))` and verify it; the output must show `Finding : no entry up to sequence` and `Result: VALID`:

```
nexusphere-ledger-verify index --public-key "$LEDGER_PUBLIC_KEY" proof.json
```

Step 10. Save the track record of Verify an Agent Track Record as `track-record.jwt` and the index proof as `proof.json`, and verify them together; the output must show `Index proof : ... entries at checkpoint ...` and `Result: VALID`:

```
nexusphere-ledger-verify track-record --issuer http://localhost:8090 --public-key
"$LEDGER_PUBLIC_KEY" --audience https://supplier.example --index-proof proof.json track-record.jwt
```

Step 11. Save both statements of Reconcile Two Ledgers with `open("buyer.jwt", "w").write(signed["statement"])` and `open("supplier.jwt", "w").write(done["statement"])`, and compare them pinned to both keys; the output must show `Matched : 2` and `Result: RECONCILED`:

```
nexusphere-ledger-verify reconcile --public-key "$(curl -s http://localhost:8090/api/v1/keys -H
'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[] | select(.status
== "ACTIVE") | .publicKey')" --peer-public-key "$(curl -s http://localhost:8092/api/v1/keys -H
'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[] | select(.status
== "ACTIVE") | .publicKey')" buyer.jwt supplier.jwt
```

Reference

Module	Contents
`client`	`LedgerClient` for evidence, batches, decisions, `act`, prompts, grants, mandates, packages, keys
`package`	`verify_package` and `PackageReport`
`timestamp`	RFC 3161 `TimestampToken`, its verification against a pinned chain, and `read_certificates`
`rekor`	Rekor v2 `hashedrekord` requests and `verify_rekor_entry` against a pinned log
`report`	Signed reports: `verify_report`, `ReportCheck`, `report_signed_bytes` and `entries_digest`
`index`	Index proofs of presence and absence: `verify_index_proof`, `IndexCheck`, `index_signed_bytes`
`evidence`	Content and link hashes, commitments, `ChainVerifier`, `Checkpoint` with its compliance profiles
`merkle`	RFC 9162 roots, inclusion and consistency proofs
`note`	C2SP signed notes, note keys and `tlog-cosignature/v1` cosignatures
`cose`	`CoseSign1`, SCITT `EvidenceStatement` and RFC 9942 `LogReceipt`
`mandate`	SD-JWT VC mandates, `MandateVerifier`, status lists and JWKs
`trackrecord`	SD-JWT VC agent track records, `TrackRecordVerifier` and `match_track_record`
`reconcile`	Signed A2A statements: `ExchangeStatement`, `verify_statement`, `reconcile` and `Reconciliation`
`httpsig`	RFC 9421 HTTP message signatures: `sign_request`, `verify_request`, `content_digest`
`keys`	Ed25519 keys, key IDs, key rotations and trusted keys from a pinned key
`canonical`	Canonical JSON
`cbor`	Deterministic CBOR

Test

Step 1. Run the unit tests and the conformance vectors; every test must pass:

```
cd ledger/sdk/python
PYTHONPATH=src python3 -m unittest discover -s tests -t .
```

Step 2. Run the same tests against a running ledger, as in the Quick Start; the seven `LiveLedger` tests must pass instead of being skipped:

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 PYTHONPATH=src python3 -m unittest discover -s tests -
t . -v
```

4.2 · ledger/sdk/typescript/README.md

Nexusphere Ledger TypeScript SDK

Record evidence from a Node or browser agent and verify everything the ledger produces, offline.

Purpose

`@nexusphere/ledger` is the TypeScript counterpart of the Python SDK: a client for agents, gateways and platforms that record evidence, and an offline verifier for packages, SCITT statements, receipts and mandates. It has no runtime dependencies. Signatures and hashes use Web Crypto and status lists use `DecompressionStream`, so the same code runs in Node 20 or later and in current browsers. It passes every Conformance Vector byte for byte. Every call that hashes or checks a signature returns a promise.

Install

Step 1. Check Node; the version must be 20 or later:

```
node --version
```

Step 2. Build the SDK and add it to your project:

```
cd ledger/sdk/typescript
npm install
npm run build
cd -
npm install ./ledger/sdk/typescript
```

Step 3. Check the install; it must print `1.0.0`:

```
node --input-type=module -e "import { VERSION } from '@nexusphere/ledger'; console.log(VERSION)"
```

Record Evidence

These steps need a running ledger, as in the Quick Start. `input` and `output` are hashed on the client with SHA-256; only the hash reaches the ledger.

Step 1. Register an agent with the operator key and keep its API key:

```
import { LedgerClient } from "@nexusphere/ledger";

const operator = new LedgerClient("http://localhost:8090", { apiKey: "nexusphere-ledger-
development-key-change-me" });
const { apiKey } = await operator.registerAgent("invoice-agent", "Invoice agent", "acme");
```

Step 2. Record what the agent did; the answer carries the `sequence` and `hash` of the new entry:

```
const agent = new LedgerClient("http://localhost:8090", { apiKey: apiKey as string });
const entry = await agent.record({
  agentId: "invoice-agent", principalId: "acme", action: "tools/call", outcome: "SUCCEEDED",
  target: "read_invoice", decision: "ALLOW", input: { invoice: 7 }, output: "invoice text",
});
console.log(entry.sequence, entry.hash);
```

Step 3. Record many entries in one call, all or none:

```
await agent.recordBatch([
  { agentId: "invoice-agent", principalId: "acme", action: "tools/call", outcome: "SUCCEEDED" },
  { agentId: "invoice-agent", principalId: "acme", action: "tools/call", outcome: "FAILED" },
]);
```

Record the Model and Prompt

Evidence and decisions may name the model the agent ran in `model`: its `name`, `version`, and the lowercase SHA-256 `promptHash` and `configHash` of its system prompt and configuration. The content hash covers the model; entries without one hash as before. `client.storePrompt(text)` stores a prompt or configuration text and resolves to its hash, and `client.prompt(hash)` reads it back, as the operator. `client.activatePolicies(text)` activates a Cedar policy set, and `client.activePolicies()` and `client.policySet(hash)` read it back, as the operator.

Step 1. Store the system prompt and record an action with its model; the answer must show the model:

```
const promptHash = await agent.storePrompt("You pay approved invoices of Acme up to 500 EUR.");
const modelled = await agent.record({
  agentId: "invoice-agent", principalId: "acme", action: "tools/call", outcome: "SUCCEEDED",
  model: { name: "claude-sonnet-5-5", version: "2026-09-01", promptHash },
});
console.log(modelled.model);
```

Step 2. Export a package with the prompt texts its entries name; `disclosedPrompts` must be `1`:

```
import { verifyPackage } from "@nexusphere/ledger";

const pkg = await operator.exportPackage({ agentId: "invoice-agent", includePrompts: true });
console.log((await verifyPackage(pkg, await operator.activePublicKey())).disclosedPrompts);
```

Decide and Report

`act` asks for a decision, throws `Denied` when no grant covers the action, runs the work, and reports `SUCCEEDED`, or `FAILED` with the error name when the work throws.

Step 1. Grant the agent the tools, as the operator:

```
await operator.createGrant({
  principalId: "alice", agentId: "invoice-agent", actions: ["tools/call"], targets: ["read_*"],
  expiresAt: new Date(Date.now() + 30 * 86_400_000), maxUses: 100,
});
```

Step 2. Act under the grant; it must print `SUCCEEDED`:

```
import { Denied } from "@nexusphere/ledger";

try {
  await agent.act({ principalId: "alice", action: "tools/call", target: "read_invoice", input: {
invoice: 7 } },
    async (action) => {
      await action.output("invoice text");
      console.log("running");
    });
} catch (e) {
  if (e instanceof Denied) console.log(e.decision.reasonCode);
}
```

When a policy annotated `@approval` stops the action, `act` throws `Denied` with reason code `APPROVAL_PENDING` and the request in `e.approvalId`; `approvals`, `approval`, `approve` and `denyApproval` list, read and resolve approval requests, and once approved the identical action is allowed once with reason code `ALLOWED_BY_APPROVAL`. `anomalies`, `anomaly` and `detectAnomalies` list, read and trigger the operator's anomaly checks on agent volume, denials and failures.

Verify a Package

Step 1. Export a package and read the ledger's public key, as the operator:

```
const pkg = await operator.exportPackage({ agentId: "invoice-agent" });
const publicKey = await operator.activePublicKey();
```

Step 2. Verify it pinned to that key; `valid` must be `true`:

```
import { verifyPackage } from "@nexusphere/ledger";

const report = await verifyPackage(pkg, publicKey);
console.log(report.valid, report.disclosedEntries, report.problems);
```

Pass `{ witnesses: [verifierKey], requiredWitnesses: 1 }` as the third argument to require witness cosignatures, and `keys` with the list from `GET /api/v1/keys` to apply key revocations, as the Ledger README describes; the command takes it as `--keys keys.json`. The report has the same fields as the Java `PackageReport`. `report.complianceProfiles` lists the id and digest of each compliance profile the checkpoint signs, and `client.compliance()` returns the active profiles and their combined rules from the ledger. An entry whose personal fields were erased still verifies from its commitments, and `client.erasePrincipal(principalId, reason)` asks the ledger to erase a principal. A package exported with `includePrompts: true` carries `prompts`; each text must match its hash and be named by a disclosed entry, and `report.disclosedPrompts` counts those that do. A package carries `policies` the same way; each policy set text must match its hash and be named by the `policy.hash` attribute of a disclosed entry, and `report.disclosedPolicies` counts those that do. `placeLegalHold`, `releaseLegalHold`, `legalHolds` and `sweepRetention` manage legal holds and the retention pass.

A checkpoint may carry an RFC 3161 token from a timestamp authority, as the Ledger README describes. Pass `timestampCertificates` with the authority's chain from `readCertificates(pem)` to pin the authority, and `requireTimestamp: true` to reject a checkpoint without a token; the command

takes them as `--timestamp-certificates chain.pem` and `--require-timestamp`. The report gives `timestampedAt` and `timestampAuthority`, and `verifyTimestamp(token, sha256, certificates)` checks a token on its own.

```
import { readFileSync } from "node:fs";
import { readCertificates, verifyPackage } from "@nexusphere/ledger";

const timestampCertificates = readCertificates(readFileSync("tsa-chain.pem", "utf8"));
const report = await verifyPackage(pkg, publicKey, { timestampCertificates, requireTimestamp: true
});
console.log(report.timestampedAt, report.timestampAuthority);
```

A checkpoint may also carry a Rekor v2 `hashedrekord` entry, as the Ledger README describes. Pass `rekorKeys` with the Rekor log's C2SP verifier keys to pin the log, and `requireRekor: true` to reject a checkpoint without an entry; the command takes them as `--rekor-key <verifier key>` and `--require-rekor`. The report gives `rekorLogIndex` and `rekorOrigin`, and `verifyRekorEntry(entry, signedBytes, ledgerKey, rekorKeys)` checks an entry on its own.

```
import { verifyPackage } from "@nexusphere/ledger";

const report = await verifyPackage(pkg, publicKey, { rekorKeys: [rekorVerifierKey], requireRekor:
true });
console.log(report.rekorLogIndex, report.rekorOrigin);
```

Verify a Report

The ledger signs reports and dispute packages that map the evidence of a package to requirements. `client.reports()` lists the reports the ledger offers, `client.createReport(id, scope)` and `client.createReportHtml(id, scope)` make one, and `client.createDispute({ title, description, ...scope })` returns the dispute package as zip bytes; each takes `includePrompts: true` in its scope to add the prompt texts. `verifyReport(report, pkg, { publicKey, keys })` checks the signature with the keys the package is trusted with, and that the report names the package's checkpoint, scope and disclosed entries; verify the package itself with `verifyPackage`.

```
import { verifyPackage, verifyReport } from "@nexusphere/ledger";

const pkg = await operator.exportPackage({ agentId: "invoice-agent" });
const report = await operator.createReport("dispute", { agentId: "invoice-agent" });
const check = await verifyReport(report, pkg, { publicKey });
console.log((await verifyPackage(pkg, publicKey)).valid && check.valid, check.statuses,
check.problems);
```

Verify an Index Proof

An index proof shows that a term occurs in the evidence up to a checkpoint, with how often and its first and last sequence, or that it never did. A term is an `agent` with `agentId`, an `action` with `agentId` and `action`, an `input` with `inputHash`, or a `grant` with `delegationId`. `client.indexProof(term, sequence)` asks the ledger for one, as the operator, at the latest checkpoint or at the checkpoint `sequence`. `verifyIndexProof(proof, { pinnedPublicKey, keyList, package })` checks the signature of the index with the keys the proof carries, reached from the

pinned key through signed key rotations when one is given, refuses a revoked key, checks that the path leads to the signed root, and with `package` that the index is for the checkpoint of that package.

Step 1. Record an action and sign a checkpoint:

```
await agent.record({ agentId: "invoice-agent", principalId: "acme", action: "files/read", outcome:
"SUCCEEDED" });
await operator.createCheckpoint();
```

Step 2. Ask for a proof that the agent read files and one that it never deleted any:

```
const read = await operator.indexProof({ type: "action", agentId: "invoice-agent", action:
"files/read" });
const deleted = await operator.indexProof({ type: "action", agentId: "invoice-agent", action:
"files/delete" });
```

Step 3. Verify both pinned to the ledger's key; it must print `true true 1`, then `true false`:

```
import { verifyIndexProof } from "@nexusphere/ledger";

const publicKey = await operator.activePublicKey();
const found = await verifyIndexProof(read, { pinnedPublicKey: publicKey });
const missing = await verifyIndexProof(deleted, { pinnedPublicKey: publicKey });
console.log(found.valid, found.present, found.count);
console.log(missing.valid, missing.present);
```

Verify a Statement and Receipt

```
import { EvidenceStatement, LogReceipt, PublicKey } from "@nexusphere/ledger";

const key = await PublicKey.fromBase64(publicKey);
const statement = EvidenceStatement.parse(await agent.statement(entry.id as string));
const receipt = LogReceipt.parse(await agent.receipt(entry.id as string));
console.log(await statement.verify(key), await receipt.verify(await statement.leafHash(), key));
```

Verify a Mandate

`MandateVerifier` reads the issuer's keys from `/public/v1/keys` and its revocation status list, caches both for five minutes, and reports the same problem codes as the Java and Python SDKs.

```
import { MandateVerifier } from "@nexusphere/ledger";

const verifier = new MandateVerifier("http://localhost:8090", { audience:
"https://supplier.example" });
const check = await verifier.verifyAction(token, "a2a/send", "supplier/sales");
console.log(check.valid, check.claims?.agentId, check.problems.map((p) => p.code));
```

Present a Key-Bound Mandate

An agent with its own key registers the public half with `setSigningKey`; mandates issued afterwards name it in `cnf`. `presentBound` adds a KB-JWT signed by the agent for an audience and a nonce, and `verifyBound` checks it. For the A2A gateway the nonce is the hex SHA-256 of the request body.

```
import { MandateVerifier, PrivateKey, hashOf, presentBound } from "@nexusphere/ledger";

const key = await PrivateKey.generate();
await agent.setSigningKey("invoice-agent", key.publicKey);
const token = (await agent.issueMandate(grantId, "https://supplier.example")).token as string;
const presented = await presentBound(token, key, "https://supplier.example", await hashOf(body));
const check = await new MandateVerifier("http://localhost:8090", { audience:
"https://supplier.example" })
  .verifyBound(presented, await hashOf(body));
console.log(check.valid, check.claims?.holderKey);
```

Verify an Agent Track Record

An agent track record is an SD-JWT VC that counts the evidence naming an agent up to a signed checkpoint, with its outcomes, principals and actions disclosed selectively. `client.issueTrackRecord(agentId, audience, validFor)` asks for one as the operator or the agent itself. `TrackRecordVerifier` checks it like a mandate, with the same problem codes, but has no status list. `matchTrackRecord(claims, proof, check)` checks it against an index proof of the `agent` term at the same checkpoint and returns what does not match.

Step 1. Sign a checkpoint and let the agent ask for its track record for a supplier, as in Record Evidence:

```
await operator.createCheckpoint();
const issued = await agent.issueTrackRecord("invoice-agent", "https://supplier.example", "P7D");
```

Step 2. Present only the outcomes, and ask, as the operator, for the index proof of the agent at the same checkpoint:

```
import { SdJwt } from "@nexusphere/ledger";

const presented = SdJwt.parse(issued.credential as string).present(["outcomes"]);
const proof = await operator.indexProof({ type: "agent", agentId: "invoice-agent" },
issued.checkpoint as number);
```

Step 3. Verify both pinned to the ledger's key; it must print `true`, the total, the outcome counts and `[]`, then `null` and `[]`:

```
import { PublicKey, StaticKeys, TrackRecordVerifier, matchTrackRecord, verifyIndexProof } from
"@nexusphere/ledger";

const publicKey = await operator.activePublicKey();
const keys = StaticKeys.fixed("http://localhost:8090", await PublicKey.fromBase64(publicKey));
const check = await new TrackRecordVerifier("http://localhost:8090", { keys, audience:
"https://supplier.example" })
  .verify(presented);
console.log(check.valid, check.claims?.total, check.claims?.outcomes, check.problems);
const index = await verifyIndexProof(proof, { pinnedPublicKey: publicKey });
console.log(check.claims?.actions, matchTrackRecord(check.claims!, proof, index));
```

Reconcile Two Ledgers

Two ledgers that exchange A2A messages can reconcile what each recorded. `client.a2aStatement(peer, from, to)` asks a ledger, as the operator, for its signed statement of the exchanges with a peer in a window of at most 31 days, and `client.reconcileA2a(statement)` has the peer's ledger compare it with its own and record the result. `verifyStatement` checks a statement against the issuer's key, and `reconcile` compares two statements offline with the same rules: an exchange is `NOT_RECEIVED`, `NOT_SENT` or a `MISMATCH` with the fields that differ.

Step 1. Run the two ledgers of Reconciling Two Ledgers through its Step 4, so that the ledger on port 8090 has sent two orders to its peer `supplier` on port 8092, and connect to both as the operator:

```
import { LedgerClient } from "@nexusphere/ledger";

const apiKey = "nexusphere-ledger-development-key-change-me";
const buyer = new LedgerClient("http://localhost:8090", { apiKey });
const supplier = new LedgerClient("http://localhost:8092", { apiKey });
```

Step 2. Let the buyer's ledger sign its statement of the exchanges with the supplier from an hour ago to an hour from now; it must print `2`:

```
const now = Math.floor(Date.now() / 1000) * 1000;
const signed = await buyer.a2aStatement("supplier", new Date(now - 3_600_000), new Date(now +
3_600_000));
console.log(signed.exchanges);
```

Step 3. Let the supplier's ledger reconcile it with its own; it must print `true 2 []`:

```
const done = await supplier.reconcileA2a(signed.statement as string);
console.log(done.reconciled, done.matched, done.discrepancies);
```

Step 4. Verify both statements pinned to each ledger's key and reconcile them offline; it must print `true 2 0 []`:

```
import { PublicKey, StaticKeys, reconcile, verifyStatement } from "@nexusphere/ledger";

const ours = await verifyStatement(signed.statement as string,
  StaticKeys.fixed("http://localhost:8090", await PublicKey.fromBase64(await
buyer.activePublicKey())));
const theirs = await verifyStatement(done.statement as string,
  StaticKeys.fixed("http://localhost:8092", await PublicKey.fromBase64(await
supplier.activePublicKey())));
const r = reconcile(ours, theirs);
console.log(r.reconciled, r.matched, r.undelivered, r.discrepancies);
```

Check a Signed Request

Every request the ledger forwards to an MCP server or an A2A agent carries an RFC 9421 HTTP message signature. `verifyRequest` checks it with the ledger's published keys and resolves to the key ID, `created` and `nonce`; it rejects with `InvalidHttpSignature` when the signature is missing, stale or does not match the method, address or body. `signRequest` makes the same headers.

```
import { JwksKeys, httpFetcher, verifyRequest } from "@nexusphere/ledger";

const keys = new JwksKeys(httpFetcher());
const verified = await verifyRequest("POST", "http://files-mcp:3000/mcp", request.headers, body,
  (kid) => keys.resolve("http://localhost:8090", kid));
console.log(verified.keyId, verified.nonce);
```

Command Line

The package adds `nexusphere-ledger-verify` with the same arguments, output and exit codes (0 valid, 1 invalid, 2 usage error) as the Java verifier jar and the Python SDK.

Step 1. Verify a package pinned to the ledger's key; the last line must be `Result: VALID`:

```
npx nexusphere-ledger-verify --public-key "$(curl -s http://localhost:8090/api/v1/keys -H
'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[] | select(.status
== "ACTIVE") | .publicKey')" package.json
```

Step 2. Verify a package whose checkpoint must carry a timestamp from the pinned authority; the output must show `Timestamp : ... (pinned)`:

```
npx nexusphere-ledger-verify --timestamp-certificates tsa-chain.pem --require-timestamp
package.json
```

Step 3. Verify a package whose checkpoint must be anchored in the pinned Rekor log; the output must show `Rekor : entry ... (pinned)`:

```
npx nexusphere-ledger-verify --rekor-key "{rekorVerifierKey}" --require-rekor package.json
```

Step 4. Verify a statement with its receipt, and a mandate for an action:

```
npx nexusphere-ledger-verify statement --public-key "{publicKey}" --receipt receipt.cose
statement.cose
npx nexusphere-ledger-verify mandate --issuer http://localhost:8090 --action a2a/send --target
supplier/sales "{token}"
```

Step 5. Verify a dispute package, and a report against its package; the last line must be `Result: VALID`:

```
npx nexusphere-ledger-verify dispute --public-key "$LEDGER_PUBLIC_KEY" dispute.zip
npx nexusphere-ledger-verify report --public-key "$LEDGER_PUBLIC_KEY" --package package.json
report.json
```

Step 6. Verify an index proof, optionally against the package of the same checkpoint; the last line must be `Result: VALID`:

```
npx nexusphere-ledger-verify index --public-key "$LEDGER_PUBLIC_KEY" --package package.json
proof.json
```

Step 7. Save the track record of Verify an Agent Track Record as `track-record.jwt` and its index proof as `proof.json`, and verify them together; the output must show `Index proof : ... entries at checkpoint ...` and `Result: VALID`:

```
npx nexusphere-ledger-verify track-record --issuer http://localhost:8090 --public-key
"$LEDGER_PUBLIC_KEY" --audience https://supplier.example --index-proof proof.json track-record.jwt
```

Step 8. Save both statements of Reconcile Two Ledgers as `buyer.jwt` and `supplier.jwt` with `writeFileSync`, and compare them pinned to both keys; the output must show `Matched : 2` and `Result: RECONCILED`:

```
npx nexusphere-ledger-verify reconcile --public-key "$(curl -s http://localhost:8090/api/v1/keys -
H 'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[] |
select(.status == "ACTIVE") | .publicKey')" --peer-public-key "$(curl -s
http://localhost:8092/api/v1/keys -H 'Authorization: Bearer nexusphere-ledger-development-key-
change-me' | jq -r '.[] | select(.status == "ACTIVE") | .publicKey')" buyer.jwt supplier.jwt
```

`dispute` takes `keys.json`, the timestamp authority chain, the Rekor log key and the witness keys from the archive when no option gives them, and lists the last three on the `Archive` line; compare them with copies of your own. It requires no witness cosignature unless `--witness` or `--witnesses-required` is given.

Reference

Module	Contents
`client`	`LedgerClient` for evidence, decisions, `act`, grants, mandates, prompts, packages, keys, proofs
`package`	`verifyPackage` and `PackageReport`
`report`	Signed reports: `verifyReport`, `ReportCheck`, `reportSignedBytes` and `entriesDigest`
`indexproof`	Index proofs: `verifyIndexProof`, `IndexCheck`, `indexTerm`, `indexKey` and `indexSignedBytes`
`evidence`	Content and link hashes, commitments, `ChainVerifier`, `Checkpoint` with its compliance profiles
`timestamp`	RFC 3161 tokens, X.509 certificates, `readCertificates` and `verifyTimestamp`
`rekor`	Rekor v2 `hashedrekord` entries: `verifyRekorEntry` and `rekorRequest`
`merkle`	RFC 9162 roots, inclusion and consistency proofs, exported as `merkle`
`note`	C2SP signed notes, note keys and `tlog-cosignature/v1` cosignatures

Module	Contents
`cose`	`CoseSign1`, SCITT `EvidenceStatement` and RFC 9942 `LogReceipt`
`mandate`	SD-JWT VC mandates, `MandateVerifier`, status lists and JWKs
`trackrecord`	SD-JWT VC agent track records, `TrackRecordVerifier` and `matchTrackRecord`
`reconcile`	Signed A2A statements: `ExchangeStatement`, `verifyStatement`, `reconcile` and `Reconciliation`
`httpsig`	RFC 9421 HTTP message signatures: `signRequest`, `verifyRequest`, `contentDigest`
`keys`	Ed25519 keys, key IDs, key rotations and trusted keys from a pinned key
`canonical`	Canonical JSON and SHA-256
`cbor`	Deterministic CBOR, exported as `cbor`

Test

Step 1. Install the tools, type-check and run the unit tests and the conformance vectors; every test must pass:

```
cd ledger/sdk/typescript
npm install
npm run lint
npm test
```

Step 2. Run the same tests against a running ledger, as in the Quick Start; the three tests of `a running ledger` must pass instead of being skipped:

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 npm test
```

4.3 · ledger/otel-exporter/README.md

Nexusphere Ledger OpenTelemetry Exporter

Turn the GenAI spans agents already emit into ledger evidence, through the OpenTelemetry Collector.

Purpose

Most agent frameworks and gateways already emit OpenTelemetry spans that follow the GenAI and MCP semantic conventions. The `nexusphere` exporter is a Collector component that picks the spans of tool calls and agent invocations and records each one as evidence through `POST /api/v1/evidence/batch`. An agent needs no code change and no SDK: point its OTLP exporter at a Collector that has this exporter in its traces pipeline. The resulting entries get the same hash chain, transparency log, witnesses, statements and packages as any other evidence. A ready configuration for agentgateway is in Adapters.

Tool arguments and results never reach the ledger. When a span carries `gen_ai.tool.call.arguments` or `gen_ai.tool.call.result`, the exporter sends only their SHA-256 as `inputHash` and `outputHash`, so the ledger can later prove which input and output belonged to the call without holding them.

Mapping

Evidence field	Taken from
selected spans	`gen_ai.operation.name` or `mcp.method.name` listed in `operations` (default `execute_tool`, `invoke_agent`, `tools/call`)
`agentId`	The first of `agent_attributes` (default `gen_ai.agent.id`, `gen_ai.agent.name`), else `default_agent`
`principalId`	The first of `principal_attributes` (default `nexusphere.principal.id`, `user.id`, `enduser.id`), else `default_principal`
`action`	`mcp.method.name`; `tools/call` for `execute_tool`; `agents/invoke` for `invoke_agent`; else `gen_ai/{operation}`
`target`	`gen_ai.tool.name`; the invoked `gen_ai.agent.name` for `invoke_agent`; else the span name
`occurredAt`	The span start time
`decision`	`nexusphere.decision` (`ALLOW` or `DENY`) when the gateway sets it
`outcome`	`DENIED` after a `DENY`, `FAILED` when the span status is an error or `mcp.error.code` is set, else `SUCCEEDED`
`reason`	`nexusphere.reason`, else `error.type` or the status message of a failed span
`delegationId`	`nexusphere.delegation.id`
`inputHash`	`nexusphere.input.hash`, else the SHA-256 of `gen_ai.tool.call.arguments`
`outputHash`	`nexusphere.output.hash`, else the SHA-256 of `gen_ai.tool.call.result`
`correlationId`	`gen_ai.conversation.id`, else the trace ID

Evidence field	Taken from
`model.name`	`gen_ai.response.model`, else `gen_ai.request.model`
`model.version`	`nexusphere.model.version`
`model.promptHash`	`nexusphere.prompt.hash`, lowercased, only when it is a hex SHA-256
`model.configHash`	`nexusphere.config.hash`, lowercased, only when it is a hex SHA-256
`attributes`	`otel.trace_id`, `otel.span_id`, `otel.span_name`, `otel.duration_ms`, the model, provider, tool call ID, `mcp.session.id`, `mcp.target` and `service.name`, and the keys in `attributes`

Span attributes win over resource attributes. A selected span without an agent or a principal is skipped and logged at debug level. Text is cut to the ledger's limits and control characters are replaced, so one odd span never makes the ledger refuse a whole batch. `model` is sent only when one of its fields is present, and a prompt or config hash that is not a hex SHA-256 is left out. A batch the ledger still refuses (HTTP 4xx) is dropped and logged; network errors, 429 and 5xx are retried with the Collector's retry and queue settings. A batch that reached the ledger but whose answer was lost is sent again, so the same span can be recorded twice; `otel.span_id` tells such entries apart.

Configuration

Setting	Default	Meaning
`endpoint`	none, required	Base URL of the ledger, such as `http://localhost:8090`
`api_key`	none, required	An agent API key, which records only that agent, or the operator key
`operations`	`execute_tool`, `invoke_agent`, `tools/call`	Values of `gen_ai.operation.name` or `mcp.method.name` that become evidence
`agent_attributes`	`gen_ai.agent.id`, `gen_ai.agent.name`	Attributes that name the agent, first match wins
`default_agent`	empty	Agent for spans without one
`principal_attributes`	`nexusphere.principal.id`, `user.id`, `enduser.id`	Attributes that name the principal, first match wins
`default_principal`	empty	Principal for spans without one
`attributes`	empty	More span or resource attributes to copy into the evidence attributes
`hash_content`	`true`	Hash tool arguments and results into `inputHash` and `outputHash`
`batch_size`	`500`	Entries per request, at most 500
`timeout`, `headers`, `tls`, `sending_queue`, `retry_on_failure`	Collector defaults, 30 s timeout	The standard Collector HTTP client, queue and retry settings

```
exporters:
  nexusphere:
    endpoint: http://localhost:8090
    api_key: ${env:NEXUSPHERE_LEDGER_API_KEY}
    default_principal: acme

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [nexusphere]
```

Run with Docker

The compose file of the ledger starts the Collector as `otel-collector` with the operator key of the `dev` profile, and `ledger/kube-dev.yaml` runs it in the `dev` namespace. Agents send OTLP to it on `4317` (gRPC) or `4318` (HTTP).

Step 1. Start the ledger as in the Quick Start; `otel-collector` starts with it.

Step 2. Send one tool call span over OTLP/HTTP; the answer must be `{"partialSuccess":{}}`:

```
curl -X POST http://localhost:4318/v1/traces -H "Content-Type: application/json" -d
'{"resourceSpans":[{"resource":{"attributes":[{"key":"service.name","value":
{"stringValue":"invoice-service"}}]},"scopeSpans":[{"spans":
[{"traceId":"5b8efff798038103d269b633813fc60c","spanId":"eee19b7ec3c1b174","name":"execute_tool
read_invoice","kind":1,"startTimeUnixNano":"'$(date +%s)000000000'","endTimeUnixNano":"'$(date
+%s)100000000'","attributes":[{"key":"gen_ai.operation.name","value":
{"stringValue":"execute_tool"}},{"key":"gen_ai.tool.name","value":{"stringValue":"read_invoice"}},
{"key":"gen_ai.agent.id","value":{"stringValue":"otel-agent"}},{"key":"user.id","value":
{"stringValue":"alice"}}]}]}]}]}'
```

Step 3. Check the evidence; one entry with `tools/call` on `read_invoice` for `alice` must appear:

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=otel-agent" -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me" | jq '.items[] | {action, target, principalId,
outcome}'
```

Build the Collector

`collector/builder-config.yaml` describes a Collector distribution, `nexusphere-otelcol`, with the OTLP receiver, the batch processor, the debug exporter and this exporter, built with the OpenTelemetry Collector Builder.

Step 1. Check Go; the version must be 1.26 or later:

```
go version
```

Step 2. Install the builder:

```
go install go.opentelemetry.io/collector/cmd/builder@v0.162.0
```

Step 3. Build the distribution; the last line names `./_build/nexusphere-otelcol`:

```
cd ledger/otel-exporter/collector
$(go env GOPATH)/bin/builder --config builder-config.yaml
```

Step 4. Run it against a ledger with an agent key; it must log `Everything is ready`:

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 NEXUSPHERE_LEDGER_API_KEY={agentApiKey}
NEXUSPHERE_DEFAULT_PRINCIPAL=acme ./_build/nexusphere-otelcol --config config.yaml
```

Add It to Your Collector

A Collector built with the builder takes the exporter as one more entry under `exporters` in its own builder configuration; the module path is `github.com/oss-community/nexusphere/ledger/otel-exporter` and its package name is `nexusphereexporter`.

```
exporters:
  - gomod: github.com/oss-community/nexusphere/ledger/otel-exporter v1.0.0
    name: nexusphereexporter
```

Test

Step 1. Run the unit tests; they must end with `ok`:

```
cd ledger/otel-exporter
go test ./...
```

Step 2. Run them against a running ledger; `TestLiveLedger` must pass instead of being skipped:

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 go test -count=1 -v -run TestLiveLedger ./...
```

4.4 · ledger/adapters/README.md

Nexusphere Ledger Adapters

Ready-made connections that put the tool calls of existing agent frameworks and gateways into the ledger.

Purpose

An adapter sits beside an agent framework or a gateway that already runs the tool calls and turns each call into an evidence entry, so nothing about the agent has to change but one line of setup. Every adapter records the same shape of entry as the ledger's own MCP gateway: action `tools/call`, the tool name as the target, the agent, the principal, SHA-256 hashes of the arguments and the result, and the outcome.

Adapters

Adapter	Connects	How
LangGraph	LangGraph and LangChain agents	A callback handler, or tools wrapped with `govern`
OpenAI Agents SDK	OpenAI Agents SDK agents	Run hooks, and a tool input guardrail to decide
AgentCore	Amazon Bedrock AgentCore Gateway	A Lambda function as REQUEST and RESPONSE interceptor
agentgateway	agentgateway	Its OpenTelemetry traces through the Collector exporter

Record or Decide

Each adapter can only record what already happened, which leaves the decision with the framework or gateway, or ask the ledger before the tool runs, which makes the ledger's grants the authority. agentgateway only records; its own authorization policies decide.

Mode	Before the tool runs	After the tool runs	A denied call
Record	Nothing	One entry with the outcome	Not possible
Decide	`POST /api/v1/decisions` against the grants	`POST /api/v1/decisions/{id}/outcome`	Never runs; entry `DENIED`

What an Entry Holds

Field	LangGraph	OpenAI Agents SDK	AgentCore	agentgateway
`agentId`	Set in the adapter	Set in the adapter	JWT claim or a fixed id	Request header or JWT claim
`principalId`	Run metadata or a default	Run context or a default	JWT claim or a default	Request header or JWT claim

Field	LangGraph	OpenAI Agents SDK	AgentCore	agentgateway
`target`	Tool name	Tool name	Tool name with its target	`gen_ai.tool.name`
`correlationId`	LangGraph `thread_id`	Agents SDK trace id	`Mcp-Session-Id`	Trace id
`inputHash`	Tool arguments	Tool arguments	Tool arguments	Not available in the span
`outputHash`	Tool result	Tool result	MCP `result`	Not available in the span

Test

Step 1. Start the ledger as in the Quick Start.

Step 2. Run the tests of each adapter as its README describes; every Python adapter has a live test that runs when `NEXUSPHERE_LEDGER_URL` is set, and agentgateway has `e2e.sh`. Each must end with `OK` or `Result: PASSED`.

4.5 · ledger/adapters/langgraph/README.md

Nexusphere Ledger for LangGraph

Every tool call of a LangGraph or LangChain agent as evidence in the Nexusphere Ledger.

Purpose

`nexusphere-langgraph` connects the ledger to LangGraph without changing the graph. `NexusphereCallbackHandler` records each tool call after it ran, with its outcome. `govern` wraps the tools so the ledger decides each call against the principal's grants first, and a denied call never runs: the model gets the denial as the tool's error message. Use one of the two for a tool, not both, or the call is recorded twice. Both work with `ToolNode`, `create_react_agent` and any LangChain tool, synchronous or asynchronous.

Install

Step 1. Install the SDK and the adapter from the repository; Python 3.10 or later is needed:

```
python3 -m pip install ledger/sdk/python ledger/adapters/langgraph
```

Step 2. Check the install; it must print `1.0.0`:

```
python3 -c "import nexusphere_langgraph; print(nexusphere_langgraph.__version__)"
```

Record Tool Calls

These steps need a running ledger, as in the Quick Start, and an agent registered with `POST /api/v1/agents`, as in the Python SDK.

Step 1. Create the handler with the agent's own key:

```
from nexusphere_ledger import LedgerClient
from nexusphere_langgraph import NexusphereCallbackHandler

ledger = LedgerClient("http://localhost:8090", agent_key)
handler = NexusphereCallbackHandler(ledger, "invoice-agent")
```

Step 2. Pass it with every run, and name the principal in the run metadata:

```
graph.invoke({"messages": [("user", "Read invoice 7")]},
             {"callbacks": [handler], "metadata": {"nexusphere_principal": "alice"},
              "configurable": {"thread_id": "conversation-42"}})
```

Step 3. Check the evidence; one `tools/call` entry per tool call must appear, with `SUCCEEDED` or `FAILED`:

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=invoice-agent" -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me" | jq '.items[] | {action, target, principalId,
outcome, correlationId}'
```

A failure to reach the ledger is logged and does not stop the agent. With `strict=True` the tool call fails instead, so no call runs without its evidence.

Let the Ledger Decide

Step 1. Create a grant for the agent, as in the Ledger Guide.

Step 2. Wrap the tools before building the graph:

```
from langgraph.prebuilt import ToolNode
from nexusphere_langgraph import govern

tools = govern([read_invoice, send_email], ledger, principal_key="nexusphere_principal")
tool_node = ToolNode(tools)
```

Step 3. Run the graph with the principal in the metadata or the `configurable` part of the config:

```
graph.invoke(inputs, {"configurable": {"nexusphere_principal": "alice", "thread_id":
"conversation-42"}})
```

Step 4. Check the evidence; an allowed call shows `ALLOW` with its outcome, and a call outside the grant shows `DENY` with `DENIED`, while the model reads `Denied by the Nexusphere Ledger: NOT_COVERED`.

Reference

Name	Purpose
`NexusphereCallbackHandler(client, agent_id, principal_id=None, principal_key="nexusphere_principal", attributes=None, strict=False)`	Records each tool call after it ran
`govern(tools, client, agent_id=None, principal_id=None, principal_key="nexusphere_principal", attributes=None)`	Returns tools whose calls the ledger decides first
`principal_id`	The principal when the run names none
`principal_key`	The key in the run metadata or `configurable` that names the principal
`attributes`	Extra evidence attributes; `framework` is always `langgraph`

The input hash is the SHA-256 of the tool arguments as canonical JSON, the output hash that of the result text, and the correlation id the LangGraph `thread_id`. A `ToolMessage` with status `error` is recorded as `FAILED`.

Test

Step 1. Install the adapter with LangGraph:

```
python3 -m pip install ./ledger/sdk/python "./ledger/adapters/langgraph[test]"
```

Step 2. Run the tests; they must end with `OK`:

```
cd ledger/adapters/langgraph && python3 -m unittest -v
```

Step 3. Run them against a running ledger; the two `LiveLedger` tests must pass instead of being skipped:

```
cd ledger/adapters/langgraph && NEXUSPHERE_LEDGER_URL=http://localhost:8090 python3 -m unittest -v
tests.test_live_ledger
```

4.6 · ledger/adapters/openai-agents/README.md

Nexusphere Ledger for the OpenAI Agents SDK

Every tool call of an OpenAI Agents SDK agent as evidence in the Nexusphere Ledger.

Purpose

`nexusphere-openai-agents` connects the ledger to the OpenAI Agents SDK through its own extension points. `NexusphereHooks` is a set of run hooks that records each function tool call with its outcome. `hooks.govern(tools)` adds a tool input guardrail, so the ledger decides each call against the principal's grants before it runs; a denied call never runs and the model reads the denial instead of a result. The same hooks then report the outcome of the allowed calls.

Install

Step 1. Install the SDK and the adapter from the repository; Python 3.10 or later is needed:

```
python3 -m pip install ledger/sdk/python ledger/adapters/openai-agents
```

Step 2. Check the install; it must print `1.0.0`:

```
python3 -c "import nexusphere_openai_agents; print(nexusphere_openai_agents.__version__)"
```

Record Tool Calls

These steps need a running ledger, as in the Quick Start, and an agent registered with `POST /api/v1/agents`, as in the Python SDK.

Step 1. Create the hooks with the agent's own key, and let them see tool failures:

```
from agents import Agent, Runner
from nexusphere_ledger import LedgerClient
from nexusphere_openai_agents import NexusphereHooks

hooks = NexusphereHooks(LedgerClient("http://localhost:8090", agent_key), "invoice-agent")
agent = Agent(name="Invoices", instructions="Handle invoices.",
              tools=hooks.govern([read_invoice, send_email], decide=False))
```

Step 2. Run the agent with the hooks and the principal in the run context:

```
result = await Runner.run(agent, "Read invoice 7", context={"principal_id": "alice"}, hooks=hooks)
```

Step 3. Check the evidence; one `tools/call` entry per tool call must appear, with `SUCCEEDED` or `FAILED`:

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=invoice-agent" -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me" | jq '.items[] | {action, target, principalId,
outcome, correlationId}'
```

Without `govern` the hooks still record every call, but a tool that raised is recorded as `SUCCEEDED`, because the SDK hands the hooks the error message as an ordinary result. A failure to reach the ledger is logged and does not stop the agent; with `strict=True` the run fails instead.

Let the Ledger Decide

Step 1. Create a grant for the agent, as in the Ledger Guide.

Step 2. Govern the tools; `decide` is on by default:

```
agent = Agent(name="Invoices", instructions="Handle invoices.", tools=hooks.govern([read_invoice,
send_email]))
```

Step 3. Run the agent with the same hooks, as in Record Tool Calls Step 2.

Step 4. Check the evidence; an allowed call shows `ALLOW` with its outcome, and a call outside the grant shows `DENY` with `DENIED`, while the model reads `Denied by the Nexusphere Ledger: NOT_COVERED`.

Reference

Name	Purpose
`NexusphereHooks(client, agent_id=None, principal_id=None, principal=None, attributes=None, strict=False)`	Run hooks that record or report each tool call
`hooks.govern(tools, decide=True)`	Adds the ledger guardrail and failure tracking to function tools
`hooks.guardrail`	The tool input guardrail, for tools built by hand
`principal`	A function from the run context to the principal
`principal_id`	The principal when the context names none

Without `principal`, the principal is the `principal_id` key of a dictionary context or the `principal_id` attribute of any other context. The input hash is the SHA-256 of the tool arguments as canonical JSON, the output hash that of the result, and the correlation id the Agents SDK trace id.

Test

Step 1. Run the tests; they must end with `OK`:

```
cd ledger/adapters/openai-agents && python3 -m unittest -v
```

Step 2. Run them against a running ledger; the two `LiveLedger` tests must pass instead of being skipped:

```
cd ledger/adapters/openai-agents && NEXUSPHERE_LEDGER_URL=http://localhost:8090 python3 -m
unittest -v tests.test_live_ledger
```

The tests run real agents on the SDK's own `ScriptedModel`, so no model and no OpenAI key are needed.

4.7 · ledger/adapters/agentcore/README.md

Nexusphere Ledger for Amazon Bedrock AgentCore

An AgentCore Gateway interceptor that records or decides every MCP tool call in the Nexusphere Ledger.

Purpose

`nexusphere-agentcore` is an AWS Lambda function for the REQUEST and RESPONSE interceptors of an Amazon Bedrock AgentCore Gateway with MCP targets. In `record` mode it lets every request through and records each `tools/call` with its outcome when the response passes. In `decide` mode the REQUEST interceptor asks the ledger first: a denied call is answered at once with a JSON-RPC error `-32003`, the same answer the ledger's own MCP gateway gives, and never reaches the target; an allowed call carries its decision id in `params._meta`, and the RESPONSE interceptor reports its outcome. Other MCP methods pass unchanged.

Build the Function

Step 1. Install the dependencies for the Lambda runtime into a build folder; the runtime here is Python 3.13 on x86_64:

```
python3 -m pip install --target build/agentcore --platform manylinux2014_x86_64 --python-version
3.13 --only-binary=:all: "cryptography>=42"
python3 -m pip install --target build/agentcore --no-deps ledger/sdk/python
ledger/adapters/agentcore
```

Step 2. Pack the folder:

```
cd build/agentcore && zip -qr ../nexusphere-agentcore.zip . && cd ../..
```

Step 3. Check the package; the list must contain `nexusphere_agentcore/interceptor.py`:

```
unzip -l build/nexusphere-agentcore.zip | grep nexusphere_agentcore/
```

Deploy

The ledger must be reachable from the Lambda function, for example through a VPC or a public HTTPS address. The commands use the AWS CLI; `{account}`, `{region}` and the role names are your own.

Step 1. Create the function with the handler `nexusphere_agentcore.lambda_handler`:

```
aws lambda create-function --function-name nexusphere-interceptor --runtime python3.13 --
architectures x86_64 --handler nexusphere_agentcore.lambda_handler --zip-file
fileb://build/nexusphere-agentcore.zip --role arn:aws:iam::{account}:role/nexusphere-interceptor-
role --timeout 10 --environment "Variables=
{NEXUSPHERE_LEDGER_URL=https://ledger.example.com,NEXUSPHERE_LEDGER_API_KEY=
{key},NEXUSPHERE_MODE=record}"
```

Step 2. Allow the gateway service role to invoke the function, as in the AgentCore guide on interceptor permissions.

Step 3. Add the function to the gateway as both interceptors, with the request headers, so the interceptor sees the caller's token:

```
aws bedrock-agentcore-control update-gateway --gateway-identifier {gatewayId} --name {gatewayName}
--role-arn arn:aws:iam::{account}:role/{gatewayRole} --protocol-type MCP --authorizer-type
CUSTOM_JWT --authorizer-configuration file://authorizer.json --interceptor-configurations
'[{"interceptor":{"lambda":{"arn":"arn:aws:lambda:{region}:{account}:function:nexusphere-
interceptor"}},"interceptionPoints":["REQUEST","RESPONSE"],"inputConfiguration":
{"passRequestHeaders":true}}]'
```

Step 4. Call a tool through the gateway, then check the evidence; one `tools/call` entry with the tool name as target, such as `invoices___read_invoice`, must appear:

```
curl -s "https://ledger.example.com/api/v1/evidence?agentId={agentId}" -H "Authorization: Bearer
{operatorKey}" | jq '.items[] | {action, target, principalId, outcome, correlationId}'
```

Settings

Variable	Default	Meaning
`NEXUSPHERE_LEDGER_URL`	(required)	Address of the ledger
`NEXUSPHERE_LEDGER_API_KEY`	(required)	The operator key for many agents, or one agent's key with `NEXUSPHERE_AGENT_ID`
`NEXUSPHERE_MODE`	`record`	`record`, or `decide` to let the ledger allow or deny each call
`NEXUSPHERE_AGENT_ID`	(none)	A fixed agent id for every call
`NEXUSPHERE_AGENT_CLAIM`	`client_id`	The token claim that names the agent when no fixed id is set
`NEXUSPHERE_PRINCIPAL_CLAIM`	`sub`	The token claim that names the principal
`NEXUSPHERE_DEFAULT_PRINCIPAL`	(none)	The principal when the token has none

Keep the key out of plain environment variables in production: encrypt it with a customer managed KMS key, or read it from AWS Secrets Manager through the Lambda extension and set the variable at cold start.

Identity

The gateway has already validated the caller's JWT before the interceptor runs, so the interceptor reads its claims without checking the signature again. A call without an agent or a principal is not recorded in `record` mode and is denied with `UNKNOWN_IDENTITY` in `decide` mode. When the ledger cannot be reached, `record` mode logs the error and lets the call through, while `decide` mode denies it with `LEDGER_UNAVAILABLE`.

Evidence field	Source
`agentId`	`NEXUSPHERE_AGENT_ID`, or the agent claim of the token
`principalId`	The principal claim of the token, or the default principal
`target`	`params.name` of the `tools/call`
`inputHash`	SHA-256 of `params.arguments` as canonical JSON
`outputHash`	SHA-256 of the JSON-RPC `result` as canonical JSON
`outcome`	`FAILED` for a JSON-RPC error, `isError` or an HTTP error
`correlationId`	The `Mcp-Session-Id` header

Test

The tests feed the function the interceptor events in the shapes the AgentCore documentation specifies; they do not call AWS.

Step 1. Run the tests; they must end with `OK`:

```
cd ledger/adapters/agentcore && python3 -m unittest -v
```

Step 2. Run them against a running ledger; the two `LiveLedger` tests must pass instead of being skipped:

```
cd ledger/adapters/agentcore && NEXUSPHERE_LEDGER_URL=http://localhost:8090 python3 -m unittest -v
tests.test_live_ledger
```

4.8 · ledger/adapters/agentgateway/README.md

Nexusphere Ledger for agentgateway

Every MCP tool call that passes agentgateway as evidence in the Nexusphere Ledger.

Purpose

agentgateway already traces every MCP request with OpenTelemetry, including `mcp.method.name`, `gen_ai.tool.name` and the JSON-RPC error of a failed call. `config.yaml` sends those traces to the Collector of the OpenTelemetry Exporter, which turns each `tools/call` span into an evidence entry. Two trace fields name the agent and the principal; here they come from request headers. agentgateway keeps deciding with its own authorization policies; the ledger records.

Run

Step 1. Start the ledger and the Collector as in the Quick Start; the compose file runs the Collector as `otel-collector` on 4317 and the demo MCP server on 8091.

Step 2. Install agentgateway as its documentation describes, and check it; the command must print the version:

```
agentgateway --version
```

Step 3. Start agentgateway with the configuration; the log must end with `started bind`:

```
agentgateway -f ledger/adapters/agentgateway/config.yaml
```

Step 4. Run the end-to-end check in another terminal; it must end with `Result: PASSED`:

```
ledger/adapters/agentgateway/e2e.sh
```

To proxy your own MCP servers, replace the `demo` target in `config.yaml`; the `config.tracing` part stays as it is. On a host without IPv6, start agentgateway with `IPV6_ENABLED=false`.

Identity from a Token

With `mcpAuthentication` on the route, agentgateway validates the caller's JWT, and the trace fields can name the agent and the principal from its claims instead of headers:

```
config:
  tracing:
    otlpEndpoint: http://localhost:4317
    randomSampling: true
    fields:
      add:
        gen_ai.agent.id: jwt.azp
        nexusphere.principal.id: jwt.sub
```

What Is Recorded

Evidence field	From the agentgateway span
`agentId`	`gen_ai.agent.id`, set by the trace field
`principalId`	`nexusphere.principal.id`, set by the trace field
`target`	`gen_ai.tool.name`
`outcome`	`FAILED` with `JSON-RPC {code} {message}` when `mcp.error.code` is set
`correlationId`	The trace id
`attributes`	`mcp.target`, `mcp.session.id`, `service.name` and the span ids

agentgateway does not put tool arguments or results into its spans, so the entries carry no input or output hash, and a tool result with `isError` is recorded as `SUCCEEDED`. Where those matter, use the ledger's own MCP gateway or one of the framework adapters.

Test

`e2e.sh` opens an MCP session through agentgateway as a new agent for `alice`, calls `read_file` and a tool that does not exist, waits for the Collector, and checks that the ledger holds a `SUCCEEDED` and a `FAILED` entry. It reads `AGENTGATEWAY_URL`, `NEXUSPHERE_LEDGER_URL` and `NEXUSPHERE_LEDGER_API_KEY`, and defaults to the local setup above.

4.9 · ledger/verifier-web/README.md

Nexusphere Package Verifier for the Browser

One HTML file that verifies an evidence package, an index proof or an agent track record in the browser, with no server and no network.

Purpose

An auditor or a counterparty who receives an evidence package should not need Java, a command line or the ledger to check it. The verifier is a React and TypeScript page that runs the TypeScript SDK verifier on Web Crypto. The build makes one self-contained `index.html` with the script and styles inline. Its Content Security Policy forbids every connection (`connect-src 'none'`), so the browser itself guarantees that the package and the keys never leave the page. The file can be mailed, put on a USB stick or opened from disk.

Build

Step 1. Check Node; the version must be 20 or later:

```
node --version
```

Step 2. Install the dependencies:

```
cd ledger/verifier-web && npm ci
```

Step 3. Build the page:

```
npm run build
```

Step 4. Check the result; `dist` must hold only `index.html`:

```
ls dist
```

Verify a Package

Step 1. Export a package and read the ledger's public key, as the operator:

```
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' > package.json
curl -s http://localhost:8090/api/v1/keys -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" | jq -r '.[] | select(.status == "ACTIVE") | .publicKey'
```

Step 2. Open `ledger/verifier-web/dist/index.html` in a browser, from disk or any web server.

Step 3. Choose or drop `package.json`, paste the public key, and add witness keys if the log must carry their cosignatures. To apply key revocations, also choose the key list saved from `GET /api/v1/keys`. When the ledger timestamps its checkpoints, choose the authority's `tsa-chain.pem` and tick Require a timestamp. When it anchors them in Rekor, paste the log's verifier key in Rekor log keys and tick Require a Rekor entry.

Step 4. Press Verify; the page must show `VALID` with the signing key, the checkpoint, its timestamp and Rekor entry, the log, the proofs and the disclosed entries, with `erased` in place of the principal and target of an entry whose personal fields were erased. When the entries name their model, the table adds a Model column with its name and version, and a package exported with `"includePrompts": true` adds a Prompts line with the number of prompt texts that match the hashes their entries name; a package that carries `policies` adds a Policies line with the number of Cedar policy set texts that match the hashes their entries name in their `policy.hash` attribute. Change one character of an entry in the file and verify again; the page must show `INVALID` and name the entry.

Without a public key, the page checks the package only against the keys it carries and says so: anyone can build such a package, so pin the ledger's key whenever the result matters. The page needs a browser with Ed25519 in Web Crypto: Chrome or Edge 137, Firefox 129, Safari 17 or later.

Verify an Index Proof

The page also takes an index proof, which it recognises by its format `nexusphere-ledger/index-proof/v1`: that a term occurs in the evidence up to a checkpoint, with how often, or that it never did. It runs the same checks as `nexusphere-ledger-verify index`, without `--package`: the term and its key, the signature of the index with the keys the proof carries, reached from the pinned key, a revoked key from the key list, and the path from the leaf to the signed root.

Step 1. Ask for a proof that the payments agent never deleted a file, as the operator:

```
curl -s -X POST http://localhost:8090/api/v1/index/proofs -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d '{"term": {"type":"action","agentId":"payments-agent","action":"files/delete"}}' > proof.json
```

Step 2. Open `ledger/verifier-web/dist/index.html`, choose or drop `proof.json` and paste the ledger's public key.

Step 3. Press Verify; the page must show `VALID` with the Term `action payments-agent files/delete`, the Checkpoint with its head hash, Signed by with the key ID, and the Finding `no entry up to sequence N`, or the count with the first and last sequence when the term occurs. Change `files/delete` to `files/remove` in the file and verify again; the page must show `INVALID` and `the key does not belong to the term`.

Verify a Track Record

The page also takes an agent track record, which it recognises as an SD-JWT VC whose `vct` is `urn:nexusphere:vct:agent-track-record:1`. It checks the `dc+sd-jwt` type, the disclosures, the signature with the pinned ledger key or a key it endorsed in the key list, the validity period and, when the record names an agent key in `cnf`, its key binding. With an index proof for the `agent`

term, it also checks that the proof verifies and gives the same checkpoint, index root, count and sequences, as `nexusphere-ledger-verify track-record --index-proof` does. The page has no issuer list: the pinned key is the trust anchor.

Step 1. Get a track record and an index proof for the agent at the same checkpoint, as in Agent Track Records.

Step 2. Open `ledger/verifier-web/dist/index.html`, choose or drop `track-record.jwt` or paste its text in Or paste a track record, paste the ledger's public key and choose `agent-proof.json` as the index proof.

Step 3. Press Verify; the page must show `VALID` with the Credential, Issuer, Agent, Audience, Valid, Checkpoint, Index root, Entries, Outcomes, Principals and Actions, `withheld` for each claim the presentation does not disclose, and Index proof `N entries at checkpoint S, matches the track record`. Verify a proof for another agent; the page must show `INVALID` and `The index proof is not about the agent quote-agent`.

What It Checks

Check	Shown as
Package format and the signature of the checkpoint	Signing key and checkpoint
The pinned key, or a key the pinned key endorsed by rotation	Pinned key
Every link of the hash chain from the anchor to the checkpoint	Checked links
Disclosed entries against their content hash and the scope	Disclosed entries and scope
Merkle inclusion proofs and SCITT receipts of disclosed entries	Inclusion proofs and receipts
Witness cosignatures on the log checkpoint	Witnesses
Revoked keys, valid only where witnesses predate the compromise	Revoked keys
The RFC 3161 timestamp of the checkpoint and the authority chain	Timestamp
The Rekor entry of the checkpoint and its inclusion proof	Rekor
Prompt texts against their hash and the entries that name them	Prompts
Policy set texts against their hash and the entries naming them	Policies
The term, the signed index and the path of an index proof	Term, Checkpoint, Signed by
The count and sequences of a term, or that it never occurred	Finding
Type, disclosures, signature and validity of a track record	Credential, Signed by, Valid
The key binding of a track record bound to an agent key	Agent key
A track record against an index proof for its agent	Index proof

The checks are the same as those of the Offline Verifier, so a package that is valid on the command line is valid here.

Develop

Step 1. Start the development server; the page opens on http://localhost:5174:

```
cd ledger/verifier-web && npm run dev
```

Test

Step 1. Run the unit tests; they must all pass:

```
cd ledger/verifier-web && npm test
```

Step 2. Build the page and run the browser tests, which open the built file from disk, verify a package, check that no request leaves the page, that the prompt texts and models show, that an index proof verifies and that a track record matches its index proof; they must report `4 passed`:

```
npm run build && npx playwright install chromium && npm run e2e
```

When Chromium is already installed elsewhere, skip `playwright install` and set `CHROMIUM_PATH` to its executable.

PART 5

Web UI

The React and TypeScript interface for operators and principals.

5.1 · frontend/README.md

Nexusphere Frontend

The web UI of Nexusphere, built with React and TypeScript.

Purpose

The ledger's API is enough for agents and scripts, but operators and principals need to see evidence, grants and consent without curl. This module is a single-page application that talks to the Nexusphere Ledger API through the same origin, so it needs no CORS on the ledger.

Responsibilities

- Operator pages: the ledger head, checkpoints and signing keys, full verification, evidence with filters and entry details (including the model name, version, prompt hash and config hash when recorded), erased entries marked with their commitments, agents, grants with their mandates, evidence package export and verification, an Approvals page that lists the approval requests of `@approval` policies by status and approves or denies them with an optional reason, an Anomalies page that lists the unusual volume, denial and failure rates the ledger detected per agent by status and checks for new ones, a Reports page that creates the signed reports of the active compliance profiles and dispute packages, A2A exchanges, and a Privacy page to erase principals, place and release legal holds and run the retention pass
- Principal pages: sign-in with the ledger's OpenID Connect provider (authorization code with PKCE), the grants waiting for approval, approve, deny, revoke and create grants, the approval requests of the principal's agents to approve or deny, and the evidence about the principal

Dependencies

Depends on	Used by
the Nexusphere Ledger API (`/api` , `/public`)	none

Getting Started

Prerequisites

- Node.js 22

Development

Step 1. Start the ledger, for example with the Quick Start, and stop its `ledger-ui` container so port 5173 is free:

```
docker stop ledger-ui
```

Step 2. Install the dependencies:

```
cd frontend
npm install
```

Step 3. Start the development server:

```
npm run dev
```

Step 4. Open http://localhost:5173 and sign in with the operator key `nexusphere-ledger-development-key-change-me`.

The development server forwards `/api` and `/public` to the ledger at http://localhost:8090, or to `LEDGER_URL` when it is set.

Test

```
npm test
```

Build

```
npm run build
```

Dockerized

The image builds the application and serves it with nginx on port 80, forwarding `/api` and `/public` to `LEDGER_URL` (default `http://ledger:8090`). The ledger compose file runs it as `ledger-ui` on http://localhost:5173.

```
docker build -t samanalishiri/nexusphere-frontend:latest frontend
docker run -p 5173:80 -e LEDGER_URL=http://host.docker.internal:8090 samanalishiri/nexusphere-frontend:latest
```

Sign-in

The operator signs in with the ledger API key; the key stays in the browser's session storage until sign-out or the tab closes, and an agent key is refused. A principal signs in with the provider the ledger trusts: the UI reads the issuer and client ID from `/public/v1/oidc` and uses the authorization code flow with PKCE, so the provider needs a public client that allows `http://localhost:5173/*` as redirect URI. The Keycloak realm in the ledger compose file has one, `nexusphere-ledger`, with the principals `alice` and `bob`.

PART 6

Operations

Running Nexusphere in production and through the delivery pipeline.

6.1 · docs/operations.md

Operations

Secrets, signing keys, backup and restore, monitoring, alerts and rate limits for the ledger.

Secrets

The ledger reads secrets from environment variables or from files. Every file in `LEDGER_SECRETS_DIR`, `/run/secrets/` by default, becomes a setting named like the file, so a file `LEDGER_API_KEY` holding the key works exactly like the environment variable. A missing directory is ignored. This is how a secret manager hands secrets over without putting them in the environment: Docker secrets, Kubernetes secrets mounted as a volume, the Vault agent, the External Secrets Operator or the Secrets Store CSI driver all write files.

Secret file	Holds
`LEDGER_API_KEY`	The operator key
`LEDGER_SIGNING_PRIVATE_KEY`	The base64 PKCS#8 Ed25519 signing key
`LEDGER_SIGNING_PUBLIC_KEY`	The base64 X.509 public key
`LEDGER_DATABASE_PASSWORD`	The PostgreSQL password

Docker Compose

```
services:
  ledger:
    secrets:
      - LEDGER_API_KEY
      - LEDGER_SIGNING_PRIVATE_KEY
      - LEDGER_SIGNING_PUBLIC_KEY
secrets:
  LEDGER_API_KEY:
    file: ./secrets/ledger-api-key
  LEDGER_SIGNING_PRIVATE_KEY:
    file: ./secrets/ledger-signing-private-key
  LEDGER_SIGNING_PUBLIC_KEY:
    file: ./secrets/ledger-signing-public-key
```

Kubernetes

```
apiVersion: v1
kind: Secret
metadata:
  name: ledger-secrets
  namespace: production
type: Opaque
stringData:
  LEDGER_API_KEY: "{operator key}"
  LEDGER_SIGNING_PRIVATE_KEY: "{base64 PKCS#8 Ed25519 key}"
  LEDGER_SIGNING_PUBLIC_KEY: "{base64 X.509 public key}"
```

```
          env:
            - name: LEDGER_SECRETS_DIR
              value: /run/secrets/ledger/
          volumeMounts:
            - name: ledger-secrets
              mountPath: /run/secrets/ledger
              readOnly: true
      volumes:
        - name: ledger-secrets
          secret:
            secretName: ledger-secrets
```

Mount the secrets in their own directory, not on `/run/secrets` itself, so the service account token Kubernetes mounts there stays in place. With Vault or a cloud secret manager, let the External Secrets Operator create this Secret, or let the CSI driver mount the same files; the application side does not change.

Signing Keys

The signing key is the ledger's identity: it signs checkpoints, mandates and receipts. The private key is never stored in the database, only the public keys and their rotation history are. Generate it once, keep it in the secret manager, and keep a second copy offline; a backup of the database without the key cannot sign anything new.

Step 1. Generate an Ed25519 key:

```
openssl genpkey -algorithm ed25519 -outform DER -out ledger-signing.der
```

Step 2. Write the private and the public key in the form the ledger reads:

```
base64 -w0 ledger-signing.der > ledger-signing-private-key
openssl pkey -inform DER -in ledger-signing.der -pubout -outform DER | base64 -w0 > ledger-
signing-public-key
```

Step 3. Put both files in the secret manager as `LEDGER_SIGNING_PRIVATE_KEY` and `LEDGER_SIGNING_PUBLIC_KEY`, as in Secrets, keep one copy offline, and delete `ledger-signing.der`.

Step 4. Start the ledger and check that `/api/v1/keys` shows the new key as `ACTIVE`.

Situation	What to do
Planned rotation	Put the new key in `LEDGER_SIGNING_PRIVATE_KEY` / `PUBLIC_KEY` and the old one in `LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY`, then restart
Lost key	Start with a new key and `LEDGER_SIGNING_UNENDORSED_ROTATION=true`; tell verifiers to pin the new public key
Leaked key	Rotate, then revoke the leaked key with its compromise time; tell verifiers to pin the new key and pass `--keys`
After rotation	Old evidence and checkpoints still verify: retired public keys stay published in `/api/v1/keys` and in packages

The details of the rotation record are in Key Rotation, and those of revocation in Key Revocation. Run witnesses before a leak happens: after it, only their cosignatures prove which checkpoints of the leaked key are genuine.

Keys in Vault

In production the key is better kept in HashiCorp Vault, where it cannot be copied out. The ledger signs through the transit engine, as described in Keys in Vault.

Step 1. Create the transit key; `exportable` stays false, so nobody can read the private key:

```
vault secrets enable transit
vault write -f transit/keys/nexusphere-ledger type=ed25519
```

Step 2. Give the ledger a policy with only what it needs, and a token or an auth role bound to it:

```
vault policy write nexusphere-ledger - <<'POLICY'
path "transit/keys/nexusphere-ledger" { capabilities = ["read"] }
path "transit/sign/nexusphere-ledger" { capabilities = ["update"] }
POLICY
```

Step 3. Start the ledger with `LEDGER_SIGNING_PROVIDER=vault`, `LEDGER_SIGNING_VAULT_ADDRESS` and the token in `LEDGER_SIGNING_VAULT_TOKEN_FILE`, written by the Vault agent; when the ledger signed with a local key before, add that key as `LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY` for this one start.

Step 4. Check that `/api/v1/keys` shows the Vault key as `ACTIVE`.

To rotate, rotate the transit key in Vault and restart every instance; keep older versions in Vault, since the ledger needs the previous version once to endorse the new one.

Backup and Restore

What to Back Up

Data	Where	How
Ledger evidence, checkpoints, grants, mandates, agents, key history	PostgreSQL schema `ledger`	Database backup
Ledger signing private key	Secret manager	The secret manager's own backup, plus one offline copy
Signed checkpoints	Outside the ledger, with auditors	Copy `/api/v1/checkpoints/latest` regularly
Log checkpoints	With independent witnesses	`LEDGER_LOG_WITNESSES_{NAME}_URL` and `_KEY`

Logical Backup

Step 1. Dump the `ledger` schema:

```
docker exec ledger-postgresql pg_dump -U ledger -d ledger --schema=ledger --format=custom >
ledger-$(date +%F).dump
```

Restore

Step 1. Restore the dump into a new database:

```
docker exec ledger-postgresql createdb -U ledger ledger_restored
docker exec -i ledger-postgresql pg_restore -U ledger -d ledger_restored --no-owner < ledger-2026-10-09.dump
```

Step 2. Start the ledger on the restored database (`LEDGER_DATABASE_DB=ledger_restored`) with the same signing key. The restore keeps the triggers that make evidence append-only.

Step 3. Check the whole chain:

```
curl -s http://localhost:8090/api/v1/verification -H "Authorization: Bearer {operator key}"
```

The answer must be `"valid": true` with the `headSequence` of the backup. This was checked end to end: evidence and a signed checkpoint were recorded, dumped, restored into a new PostgreSQL, and the restored ledger verified the whole chain and checkpoint and still refused changes to evidence.

A backup also holds the encrypted personal data and the principal keys that an erasure destroys in the live database. Keep backups and WAL archives no longer than the shortest erasure deadline of your compliance profiles, one month with `eu`, so an erased principal is gone from every copy in time. A restore from an older backup brings back principals erased since then: erase them again before the ledger serves anyone.

No Lost Evidence

A nightly dump loses what was recorded after it. For evidence, run PostgreSQL with continuous WAL archiving and point-in-time recovery (for example pgBackRest, or a managed PostgreSQL with PITR) and keep the dump as a second line. A restore can never hide a loss: a signed checkpoint held outside the ledger with a higher sequence than the restored head shows that entries are missing, and any entry that differs fails verification against that checkpoint. Witnesses give the same guarantee without anyone copying checkpoints by hand: each keeps the last log checkpoint it cosigned and refuses a smaller or forked one, so a restored ledger that lost entries cannot get new cosignatures until it is back on the history the witnesses saw. An independent copy of the whole log, kept current by the verifier's `archive` command on a schedule outside the ledger's network, as in Log Archives, also shows which entries a restore lost.

Monitoring

Endpoints

Health	Metrics
`/actuator/health` , `/actuator/health/liveness` , `/actuator/health/readiness`	`/actuator/prometheus`

By default these share the application port. Set `LEDGER_MANAGEMENT_PORT` to serve them on a separate port that only the cluster can reach, and point probes and Prometheus at it.

Metrics

Metric	Meaning
`ledger_head_sequence`	Sequence of the last evidence entry

Metric	Meaning
`ledger_checkpoint_sequence`	Sequence covered by the latest signed checkpoint
`ledger_checkpoint_lag`	Entries not yet covered by a signed checkpoint
`ledger_checkpoint_age_seconds`	Seconds since the latest signed checkpoint
`ledger_evidence_recorded_total`	Entries appended, by `decision` and `outcome`
`ledger_leader`	1 on the instance that runs the scheduled work
`ledger_anomaly_score`	Score of each open anomaly, by `agent` and `kind`
`http_server_requests_seconds`	Requests, latency and status
`jvm_*`, `hikaricp_*`	Memory, threads and database connection pools

Prometheus

The ledger compose file runs Prometheus on http://localhost:9090. It scrapes the ledger with the configuration in `ledger/prometheus/prometheus.yml`. In Kubernetes the ledger pod carries the `prometheus.io/scrape`, `prometheus.io/path` and `prometheus.io/port` annotations.

Alerts

Alert	Fires when
`LedgerDown`	The ledger has not answered for 1 minute
`LedgerCheckpointStale`	New evidence has waited more than 10 minutes for a checkpoint
`LedgerHighErrorRate`	The ledger answers more than one 5xx every ten seconds
`LedgerAgentAnomaly`	An agent's volume, denials or failures are far above its baseline, as in Anomaly Alerts

The rules are in `ledger/prometheus/alerts.yml`. Prometheus evaluates them and sends them to Alertmanager, which delivers them on the channels below. Run a full chain verification (`/api/v1/verification`) on a schedule as well; it reads the whole chain, so run it at a quiet hour rather than on every scrape.

Alert Channels

The ledger compose file runs Alertmanager on http://localhost:9093. No channel is on by default: alerts are visible there and in Prometheus but go nowhere until a channel's settings are given. Any number of channels can be on at once; each alert goes to all of them, again when it is resolved, and every 4 hours while it keeps firing.

Channel	Turned on by	Optional
Email	`ALERT_EMAIL_TO` and `ALERT_SMTP_HOST`	`ALERT_SMTP_PORT` (587), `ALERT_SMTP_FROM`, `ALERT_SMTP_USERNAME`, `ALERT_SMTP_PASSWORD`, `ALERT_SMTP_TLS` (true)
Slack	`ALERT_SLACK_WEBHOOK_URL`	`ALERT_SLACK_CHANNEL`
Telegram	`ALERT_TELEGRAM_BOT_TOKEN` and `ALERT_TELEGRAM_CHAT_ID`	
Webhook	`ALERT_WEBHOOK_URL`	

```
ALERT_EMAIL_TO=ops@example.com ALERT_SMTP_HOST=smtp.example.com ALERT_SMTP_USERNAME=alerts
ALERT_SMTP_PASSWORD=secret \
  docker compose --file ledger/compose.yaml --project-name ledger up -d alertmanager
docker logs alertmanager 2>&1 | head -1
```

The first log line names the channels that are on, such as `Alert channels: email`. The settings can also live in a `ledger/.env` file, and each one can come from a file of the same name in `/run/secrets`, so passwords and tokens stay out of the environment. A Slack webhook URL comes from an incoming webhook of a Slack app. A Telegram bot token comes from @BotFather, and the chat id is the user's or group's id after they send the bot a message. The webhook channel posts the Alertmanager JSON to any URL, such as a ticketing system or an on-call service. `ALERT_GROUP_WAIT` (30s), `ALERT_GROUP_INTERVAL` (5m) and `ALERT_REPEAT_INTERVAL` (4h) tune how often messages are sent.

Delivery was checked end to end: with email and webhook on, Prometheus fired an alert, and the message reached a test mail server and the webhook. In Kubernetes, give Alertmanager the same entrypoint `ledger/alertmanager/entrypoint.sh` with these variables, or use the Alertmanager configuration of the cluster's own monitoring stack.

Capacity

Every entry is appended under the lock of the chain head, so a deployment has one write ceiling however many instances it runs. On the reference machine of the benchmarks (4 vCPU, PostgreSQL on the same host) that ceiling was about 330 entries a second; a decision is one entry and a call through a gateway two. Run the benchmark on your own hardware before sizing, watch `ledger_evidence_recorded_total` against it, and raise the per-caller rate limits so that the sum over busy agents stays below it.

Rate Limits

The ledger limits requests per caller. A caller is its token or API key, or its address when it sends none. Each caller has a bucket that holds the burst and refills at the per-minute rate; a request over it is answered 429 `RATE_LIMIT_EXCEEDED` with `Retry-After` in seconds. Health and metrics under `/actuator/` are never limited. The limit is per instance and kept in memory, so with several replicas a caller can reach the limit once on each.

Requests a minute	Burst
`LEDGER_RATE_LIMIT_PER_MINUTE` (1200)	`LEDGER_RATE_LIMIT_BURST` (200)

`0` turns the limit off. Behind a reverse proxy the address comes from `X-Forwarded-For`, so anonymous callers are told apart; let only the proxy set that header. Keep a limit at the proxy as well when the ledger is exposed to the internet, since it can also stop floods before they reach Java.

Production Checklist

- Profiles without `dev`: the ledger refuses to start with the published development secrets

- The operator key, the database password and the signing key from the secret manager, as files
- A signing key generated for this ledger, with an offline copy, or a non-exportable Vault transit key
- TLS at a reverse proxy or ingress in front of the ledger; it speaks plain HTTP
- Management port reachable only inside the cluster
- WAL archiving with point-in-time recovery, plus a regular logical dump, and a restore tested once, kept no longer than the erasure deadline of your profiles
- Signed checkpoints copied outside the ledger, and at least one independent witness cosigning the log
- A scheduled `archive` of the log's tiles kept by someone outside the operating team
- `LEDGER_TIMESTAMP_URL` and `LEDGER_TIMESTAMP_CERTIFICATES` set to an RFC 3161 timestamp authority your auditors accept, as in Checkpoint Timestamps
- `LEDGER_REKOR_URL` and `LEDGER_REKOR_KEY` set to a Rekor v2 log outside your control, with the local signing provider, as in Rekor Anchoring
- Prometheus scraping the ledger, and at least one alert channel on
- At least two ledger instances behind the load balancer, each with its own `LEDGER_INSTANCE_ID`, as in High Availability
- Rate limits sized for the expected traffic, plus a limit at the reverse proxy for internet-facing deployments
- `LEDGER_OIDC_*` set so principals approve grants with their own sign-in
- `LEDGER_COMPLIANCE_PROFILES` and `LEDGER_COMPLIANCE_REGION` set for the jurisdictions you answer to, reviewed with your legal team, as in Compliance Profiles, and a legal hold placed for every principal in a dispute before any erasure or retention pass, as in Retention and Legal Holds

6.2 · docs/devops-guide.md

DevOps Step by Step

Running the Nexusphere pipeline with the mvn-devops ready-made pipelines.

Before You Start

Every step runs in the root of the repository with Docker running. On Windows use Git Bash or Cygwin and `mvn-devops\devops.bat` instead of `mvn-devops/devops.sh`; machine settings are in Local Environment Setup.

Step 1. Create a GitHub personal access token for `oss-community/nexusphere` with the `repo` scope, and `write:packages` if GitHub Packages will be used. The pipeline clones the repository with it, so a private repository needs it in every pipeline. See mvn-devops/docs/github-setup.md.

Step 2. Check the prerequisites (Bash 4, Docker with compose, curl, jq, git, ssh-keygen, Java 21 and Maven):

```
mvn-devops/devops.sh doctor
```

Step 3. List the ready-made pipelines:

```
mvn-devops/devops.sh pipelines
```

Maven with SonarQube and Nexus

The simplest pipeline. Maven runs on this machine; SonarQube and Nexus run in Docker. The pipeline builds and tests the project, sends it to SonarQube and publishes the artifacts to Nexus. Nothing is deployed.

Step 1. Set it up:

```
mvn-devops/devops.sh setup --pipeline maven-sonarqube-nexus
```

Question	Answer
GitHub username	your GitHub user (asked only if git lacks it)
GitHub personal access token	the token from step 1
Accept the Nexus Community Edition EULA?	`yes`
Any other question	Enter, to keep the default

Step 2. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 3. Open SonarQube and Nexus; the URLs and admin users are printed, passwords are in `.devops/`:

```
mvn-devops/devops.sh urls
```

Step 4. Commit the settings so a teammate gets the same tools (`.devops/` stays out of git):

```
git add devops.conf
git commit -m "Add the maven-sonarqube-nexus pipeline"
```

Jenkins Complete

The heaviest pipeline: Jenkins with SonarQube, Nexus, a Docker registry, Trivy, Syft, Cosign, Argo CD on Kubernetes, Vault, PostgreSQL, Prometheus, Loki and k6. The image goes through `dev`, `test` and `staging` on every run, is load-tested in `staging`, and goes to `prod` only when approved. Every tool starts at once, so give Docker plenty of memory (16 GB or more) and disk. Jenkins checks the project out of GitHub, so push your commits first.

Step 1. Push the branch the pipeline will build:

```
git push
```

Step 2. Set it up:

```
mvn-devops/devops.sh setup --pipeline jenkins-complete
```

Question	Answer
GitHub username	your GitHub user (asked only if git lacks it)
GitHub personal access token	the token from step 1 of Before You Start
Accept the Nexus Community Edition EULA?	`yes`
Any other question	Enter, to keep the default

Step 3. Run ci, cd, dev, test and staging:

```
mvn-devops/devops.sh run
```

Step 4. Approve production when staging looks right:

```
mvn-devops/devops.sh run --phase prod
```

Step 5. Open Jenkins, SonarQube, Nexus, Argo CD, Vault, Grafana and the other tools:

```
mvn-devops/devops.sh urls
```

Step 6. Put the previous image back if a release goes wrong:

```
mvn-devops/devops.sh rollback prod
```

Step 7. Commit the settings:

```
git add devops.conf
git commit -m "Add the jenkins-complete pipeline"
```

Notes for This Repository

`devops.sh stages` shows what a pipeline will run before anything starts. For `jenkins-complete` two stages assume a single-module project and have not been run against this multi-module one yet: `migrate` points Flyway at `src/main/resources/db/migration` in the root, while the migrations of this repository live in `ledger/server/src/main/resources/db/migration/ledger`, and `image` runs Jib from the root, but the executable jar is built by `ledger/server`. Expect to adjust these two stages on the first run.

Every Day

Command	Does
`mvn-devops/devops.sh status`	Shows the selected tools and whether they are running
`mvn-devops/devops.sh stages`	Prints the stages and the exact commands they run
`mvn-devops/devops.sh run`	Runs the pipeline
`mvn-devops/devops.sh urls`	Prints the tool URLs and users
`mvn-devops/devops.sh secrets --reconfigure`	Answers every question again

Stop and Clean Up

```
mvn-devops/devops.sh down
mvn-devops/devops.sh destroy
```

`down` stops the tools and keeps their data; `destroy` asks before removing their containers and volumes, and again before deleting the stored values in `.devops/`.

Upgrade

```
mvn-devops/devops.sh upgrade --check
mvn-devops/devops.sh upgrade
```

`upgrade` replaces the `mvn-devops` folder with the latest release after checking it against the release's `SHA256SUMS`. Commit the folder afterwards.

PART 7

Testing

The end-to-end test suite and the scenarios it proves.

7.1 · ledger/ledger-e2e-tests/README.md

Nexusphere Ledger End-to-End Tests

Getting Started

Prerequisites

- Java 21
- Maven 3
- Docker

Run Only This Module

```
mvn -pl ledger/ledger-e2e-tests -am verify
```

Run One Test Class

```
mvn -pl ledger/ledger-e2e-tests -am verify -Dit.test=VerificationE2ETest -
Dfailsafe.failIfNoSpecifiedTests=false -Dsurefire.failIfNoSpecifiedTests=false
```

Test Harness

`LedgerE2ETestBase` starts the ledger on a free port, which is also its mandate issuer address, with the `postgresql` and `dev` profiles against a PostgreSQL 18 container. `LedgerClient` talks to it over HTTP with the development API key or a registered agent's key, like any outside caller, and rebuilds evidence entries from the JSON so they can be checked with `ledger/chain` without trusting the server. `FakeMcpServer` is a small MCP server on a random port, configured as the `files` server of the gateway; it answers `initialize`, `tools/list` and `tools/call`, and streams the answer of tools ending in `_sse`. `A2aE2ETest`, `AgentKeyBindingE2ETest` and `ReconciliationE2ETest` start a second ledger, `SupplierLedger`, with its own PostgreSQL container and signing key, which trusts the first one and forwards to `FakeA2aAgent`, which streams `message/stream` and `tasks/resubscribe`, so mandates, request proofs and receipts cross between two ledgers over HTTP. `PrincipalConsentE2ETest` starts a `StandaloneLedger` that trusts `FakeOidcProvider`, a small OpenID Connect provider on a random port that signs RS256 tokens for any principal. `KeyRotationE2ETest` restarts a `StandaloneLedger` against one PostgreSQL container with a new signing key, so the rotation happens at startup as it does in production. `VaultSigningE2ETest` adds a HashiCorp Vault container whose transit key signs, and `KeyRevocationE2ETest` adds a witness `StandaloneLedger` whose cosignatures prove which checkpoints of a revoked key came before its compromise. `ComplianceE2ETest` restarts a `StandaloneLedger` with different compliance profiles and regions against one PostgreSQL container, so profile changes are recorded once and a disallowed region stops the start. `ErasureE2ETest` erases principals on the shared ledger and on a `StandaloneLedger` with the `us` profile, whose retention keeps recent entries. `RetentionE2ETest` starts a `StandaloneLedger` with its own short profile, so entries expire and a pending erasure completes within the test. `TimestampE2ETest` starts `StandaloneLedger`s that use `FakeTimestampAuthority`, an RFC 3161 authority on a random port that signs real CMS tokens with

a test chain in `src/test/resources/tsa`, and can be told to fail. `RekorE2ETest` does the same with `FakeRekor`, a Rekor v2 log on a random port that returns entries with real inclusion proofs and signed log checkpoints. `ReportE2ETest` starts `StandaloneLedger`s with the `eu` and `us` profiles and with a profile of its own. `WitnessE2ETest` starts two `StandaloneLedger`s with their own PostgreSQL containers: one is the log and asks the other, which watches it, to cosign its log checkpoints over the C2SP witness protocol.

Test Classes

Class	Scenario
`EvidenceE2ETest`	Recorded evidence is sealed and linked to the previous entry
	Evidence is read back exactly as recorded, with microsecond timestamps
	Evidence is listed per agent and per principal in pages
	A denied action is recorded with its reason
	Invalid evidence is rejected with field errors
	Malformed requests, unknown evidence and bad paging are reported clearly
`VerificationE2ETest`	The ledger verifies its own chain and checkpoints
	A checkpoint is verified offline with the published key
	An outsider rebuilds and verifies the whole chain from the API
`ErasureE2ETest`	Erasing a principal removes its personal data and keeps the chain
	Open grants block the erasure and are then pseudonymized
	The retention of the profiles keeps recent entries
`LegalHoldE2ETest`	A legal hold keeps the personal data until it is released
	A legal hold needs a known principal, a reason and the operator
`PackageE2ETest`	A package discloses only the requested evidence and verifies offline
	A package starts at the last checkpoint before the evidence
	Any change to a package is detected
	Packages are for the operator and need matching evidence
	The operator can have the ledger verify a package
`ApiKeyE2ETest`	The API requires the ledger key
	Health is open for probes
`RateLimitE2ETest`	A caller over its rate limit is told when to retry
`AgentE2ETest`	A registered agent gets its own key once
	Registering the same agent twice is a conflict
	A disabled agent or an old key is rejected
	Agent lifecycle is recorded as evidence
	An agent cannot do what only the operator may
	An agent records and reads only its own evidence

Class	Scenario
`GrantE2ETest`	A grant is created and its terms can be recomputed from the evidence
	A revoked grant is recorded and shown as revoked
	Invalid grants are rejected with field errors
	Grants need an active agent
	An agent sees only its own grants
`DecisionE2ETest`	A covered action is allowed and its outcome is recorded
	Every denial is recorded with its reason
	The operator can ask for a decision on behalf of an agent
	Outcomes are reported once by the deciding agent for allowed actions only
	An agent cannot ask for another agent's decision and the chain stays valid
`McpGatewayE2ETest`	An allowed tool call is forwarded and recorded with its outcome
	The model headers are recorded with the decision and its outcome
	A denied tool call never reaches the server
	Tool errors are recorded as failed outcomes
	Streamed responses are returned as JSON
	A streamed answer is relayed live with the server requests inside
	The server stream is relayed on GET
	Other messages pass through with the session
	Bad requests are answered as JSON-RPC errors
	An unreachable server is recorded as a failed call
	The server can check that the ledger signed every request
`MandateE2ETest`	An agent obtains a mandate that anyone can verify with the published key
	Revoking a mandate sets its bit in the signed status list
	Revoking the grant revokes every mandate issued from it
	An agent cannot obtain or revoke mandates it does not own
	A mandate cannot outlive its grant
	The keys and status list are public and signed
`MandateVerifierE2ETest`	Another organization verifies a mandate from the public endpoints only
	An agent can present a mandate without revealing its principal
	The verifier sees a revocation as soon as the status list is refetched
	The verifier rejects what the mandate does not allow
`ReconciliationE2ETest`	Both ledgers reconcile the exchanges between them
	A request sent around the sending ledger is found
	Only the operator asks and only known peers are trusted
`A2aE2ETest`	An allowed message carries a mandate and both ledgers hold matching evidence

Class	Scenario
	A denied message never leaves the ledger
	A failed task is recorded as failed on both sides with a receipt
	The receiver rejects a revoked mandate and records the denial
	Replayed, tampered or forged requests are rejected without reaching the agent
	The receiver counts the uses of a mandate on its own
	An unreachable peer is answered without delivery
	A streamed task is relayed live and the receipt covers every event
	A stream that ends in a failed task is recorded as failed on both sides
	A request without a valid HTTP signature is rejected
`AgentKeyBindingE2ETest`	The agent registers its key and only an operator can replace it
	A mandate names the agent key in its confirmation claim
	A key-bound presentation is accepted on both ledgers
	An API key alone is not enough once the agent has a key
	A mandate bound to a replaced key is refused
	The receiving ledger rejects a bound mandate without key binding
`PrincipalConsentE2ETest`	A principal signs in with an OIDC token that only opens the principal API
	An operator grant waits for the principal's consent
	An agent asks for a grant and the principal denies it
	A principal grants and revokes on its own
	A principal cannot see or decide another principal's grants
`KeyRotationE2ETest`	An endorsed rotation keeps earlier checkpoints and mandates valid
	A new key without the previous key is refused unless the operator allows it
	A retired key cannot sign again
`VaultSigningE2ETest`	A local key moves to Vault and a Vault rotation is followed
	A wrong token or a key that is not Ed25519 is refused at startup
`KeyRevocationE2ETest`	A revoked key stays valid only where witnesses prove the time
`ComplianceE2ETest`	The active profiles are enforced, signed in checkpoints and recorded
	A ledger outside the allowed region does not start
`RetentionE2ETest`	Personal data past the maximum retention is erased unless held
	A pending erasure completes once the retention ends
`TransparencyLogE2ETest`	The log checkpoint is a signed note over the Merkle root
	An entry is proven in the signed tree
	A later tree is proven to extend an earlier one
	A package proves its entries in the log
	Verification checks the Merkle tree and log checkpoints

Class	Scenario
	This ledger witnesses no log unless configured
`WitnessE2ETest`	A witness cosigns every checkpoint that extends what it saw
	A witness refuses a forked or stale checkpoint
`ScittE2ETest`	Every entry has a signed statement and a receipt from the log
	A receipt can be asked for at an earlier log checkpoint
	Statements and receipts belong to the evidence owner
	A package carries a statement and receipt for every disclosed entry
`TimestampE2ETest`	Every checkpoint is timestamped and the package proves it
	A checkpoint is timestamped later when the authority is down
	The profiles decide which timestamp authority is accepted
`RekorE2ETest`	Every checkpoint is anchored in Rekor and the package proves it
	A checkpoint is anchored later when Rekor is down
	The ledger needs the verifier key of the Rekor log
`ReportE2ETest`	Every report of the profiles is signed and bound to its package
	Only the reports of the active profiles are offered
	A profile can name only the reports the ledger has templates for
	A dispute package holds everything needed to check it
`TrackRecordE2ETest`	Another organization checks the track record against the signed index
	The agent withholds counts and proves it holds the record
	Only the agent or the operator asks and the validity is bounded
`IndexE2ETest`	The ledger proves that a grant was used and that another never was
	An earlier checkpoint keeps its signed index
	Only the operator asks for known terms at known checkpoints
`ModelE2ETest`	The model and prompt are recorded, required and disclosed with their texts
`PolicyE2ETest`	Policies deny what a grant allows and their text is disclosed and verified
	A default deny ledger allows only what a policy permits
`ApprovalE2ETest`	The operator approves or denies once and silence denies
	The principal answers in the ledger or on their device through CIBA
`AnomalyE2ETest`	A burst of denials is detected, recorded, exported and resolved
	A new agent is not judged before it has a history
`ClusterE2ETest`	One instance leads, both write one chain and the other takes over
`TileE2ETest`	Tiles are public, immutable and match the signed checkpoint
	An archive is mirrored incrementally and checks a package offline

PART 8

Build and Delivery Toolkit

mvn-devops, the DevOps framework the project builds and ships with.

8.1 · mvn-devops/README.md

mvn-devops

A modular DevOps framework for Maven projects, written in plain Bash.

You pick the tools from a menu, mvn-devops starts them in Docker, configures them (admin passwords, tokens, repositories) and turns the stages they contribute into a pipeline for the orchestrator you chose: plain `mvn` on your machine, `mvn` in a container on a Docker machine, Jenkins or Concourse.

```
$ mvn-devops/devops.sh init

Pipeline orchestrator (choose one)
  1) concourse        Concourse CI in Docker: a ci job on every push, a manual cd job
  2) jenkins          Jenkins in Docker, configured as code (no setup wizard)
  3) maven            Run the stages with mvn directly on this machine
  4) maven-container  Run the stages with mvn in a container on the Docker machine, from a fresh
checkout of GitHub
  > [1]: 2

Code quality (choose any, comma separated, 0 for none)
  1) sonarqube        Static code analysis (SonarQube Community + PostgreSQL)
  > [0]: 1

Artifact repositories (choose any, comma separated, 0 for none)
  1) github-packages  Deploy to the Maven registry of the GitHub repository
  2) jfrog            JFrog Artifactory OSS + PostgreSQL
  3) nexus            Sonatype Nexus 3 for Maven releases and snapshots
  > [0]: 1,3
...
```

- **No changes to your pom.** Plugins are called by their coordinates and credentials come from the framework's own settings file.
- **Nothing to click.** Admin passwords, tokens, repositories, the Jenkins job and the Concourse pipeline are created for you.
- **Shared by the team.** The tools and answers are kept in a committed `devops.conf`; passwords and tokens stay on each machine.
- **Anywhere.** Each tool runs in Docker on your machine or a VM, or is an existing server with its own URL.

Prerequisites

Bash 4+ (Linux, macOS, or Git Bash on Windows), Docker with the compose plugin, `curl`, `jq`, `git` and `ssh-keygen`, plus Java and Maven when the pipeline runs on your machine. `devops.sh doctor` checks them; install commands per system are in docs/prerequisites.md.

Install

mvn-devops is meant to live inside the project, like the Maven wrapper. From the project root:

```
curl -fsSLO https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip
unzip -q mvn-devops-1.0.0.zip && mv mvn-devops-1.0.0 mvn-devops && rm mvn-devops-1.0.0.zip
git add mvn-devops && git commit -m "Add mvn-devops 1.0.0"
```

Later, `mvn-devops/devops.sh upgrade` replaces it with the latest release. Windows steps, the deb and rpm packages, and notes on line endings are in docs/installation.md.

Quick start

Step-by-step guides, one command per step, are in docs/getting-started.md.

Create the GitHub token first (docs/github-setup.md), then from the project root:

```
mvn-devops/devops.sh setup      # menu, questions, containers, tokens, pipeline
mvn-devops/devops.sh run        # run the pipeline
mvn-devops/devops.sh urls       # web consoles and how to log in
```

On Windows use `mvn-devops\devops.bat` instead. Commit the `devops.conf` that `setup` writes: a teammate who clones the project runs the same `setup` and is only asked for their own passwords, tokens and user names. Passwords, tokens and generated files go to `.devops/`, which keeps itself out of git.

`setup` is these five steps; each can also be run on its own:

Step	Command	What happens
1	`init`	Menu per category. Saves the choice in `devops.conf`.
2	`secrets`	Each selected module asks for the values it needs. Passwords default to random ones.
3	`up`	Modules prepare (config files, keys), then `docker compose up` with the selected tools.
4	`configure`	Modules finish their tool: replace default admin passwords, create tokens and repositories.
5	`publish`	The orchestrator renders the pipeline and installs it (Jenkins job, Concourse pipeline, local script).

Or start from a ready-made pipeline, a tested combination of tools, and answer only for your GitHub user and token (docs/pipelines.md):

```
mvn-devops/devops.sh pipelines
mvn-devops/devops.sh setup --pipeline jenkins-sonarqube-nexus
```

Without questions, for scripts and CI:

```
mvn-devops/devops.sh -y init --orchestrator jenkins --with sonarqube,nexus,github-pages
mvn-devops/devops.sh -y setup
```

examples/hello-maven is a small project to try it on.

Commands

Command	
`init [--orchestrator X --with a,b | --pipeline name]`	choose modules, or a ready-made pipeline
`pipelines`	list the ready-made pipelines
`setup [--pipeline name]`	init (if needed) + secrets + up + configure + publish
`secrets [--reconfigure]`	ask for values; `--reconfigure` asks again
`up`, `configure`, `publish`	single setup steps
`stages`	the ordered stages contributed by the selected modules
`render`	write pipeline files into `.devops/generated`
`run`	run the pipeline. maven, maven-container: `--dry-run`, `--from <stage>`, `--only <stage>`, `--phase ci|cd`. concourse: `--phase ci|cd`. Every orchestrator: `--phase <environment>` approves an environment
`rollback [environment] [--to TAG]`	put the previous image back, in the last environment by default
`release [--version X] [--next Y] [--dry-run] [--no-push]`	release the project (docs/releasing.md)
`status`, `urls`, `logs [service]`, `compose ...`	operations
`down`	stop containers, keep data
`destroy`	remove containers and volumes, the deploy key and webhook on GitHub, optionally the stored values
`export-compose [dir]`	write one `docker-compose.yml` and its `.env` for the selected tools
`env [--show|--windows]`	regenerate env files; `--show` masks secrets; `--windows` writes a `setx` script for IDEs
`get <KEY>`	print one stored value, e.g. `get SONAR_ADMIN_PASSWORD`
`modules`, `doctor [--fix]`	list modules, check prerequisites; `--fix` repairs CRLF line endings
`upgrade [--version X] [--check]`	replace the copy of mvn-devops inside the project with a release

Documentation

Getting started	step-by-step guides
Installation	packages, the zip inside a project, upgrades, line endings
Prerequisites	install commands for Windows, Linux and macOS
GitHub setup	tokens and their scopes, SSH key
Ready-made pipelines	tested combinations of tools, and your own
Modules and stages	every tool and the stages it adds
Project requirements	what the Maven project needs (almost nothing)

Orchestrators	maven, maven-container, Jenkins, Concourse, and the Docker images
Where the tools run	existing servers, a VM, the exported compose file
Project site	the Maven site on GitHub Pages
Deployment	environments, container images, registries and deploying the application
Releasing your project	`devops.sh release`
Troubleshooting	common errors and their fixes
IDE, ngrok	IntelliJ settings, exposing a local Jenkins to GitHub
Design	how the framework is built
Writing a module	adding a tool
Development	tests and releasing mvn-devops
Changelog	what changed in each version

License

Apache License 2.0

8.2 · mvn-devops/docs/installation.md

Installation

Every release on the releases page has:

File	For
`mvn-devops_<version>_all.deb`	Debian, Ubuntu: `sudo apt install ./mvn-devops_<version>_all.deb`
`mvn-devops-<version>-1.noarch.rpm`	Fedora, RHEL, Rocky: `sudo dnf install ./mvn-devops-<version>-1.noarch.rpm`
`mvn-devops-<version>.zip`	Windows: unzip, add the folder to `PATH`, run `devops.bat` (needs Git for Windows)
`mvn-devops-<version>.tar.gz`	macOS or any Linux: unpack and add the folder to `PATH`
`SHA256SUMS`	checksums of the files above

The Linux packages install into `/usr/share/mvn-devops` and add the command `mvn-devops`. With the zip or tar.gz the command is `devops.bat` or `devops.sh` in the unpacked folder. A `git clone` of this repository works the same way. `mvn-devops --version` prints the installed version.

Shipping it with the project (zip)

mvn-devops can live inside the project and be committed with it, so everyone who clones the project gets the same DevOps setup, like the Maven wrapper.

1. Download and extract `mvn-devops-<version>.zip` from the releases page into the project root, and rename the folder to `mvn-devops` so paths stay the same across upgrades.

Linux, macOS, Git Bash:

```
cd my-maven-project
curl -fsSLO https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip
unzip -q mvn-devops-1.0.0.zip && mv mvn-devops-1.0.0 mvn-devops && rm mvn-devops-1.0.0.zip
```

Windows PowerShell (or right-click the zip > Extract All, then rename the folder):

```
cd my-maven-project
Invoke-WebRequest https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip -OutFile mvn-devops.zip
Expand-Archive mvn-devops.zip -DestinationPath . ; Rename-Item mvn-devops-1.0.0 mvn-devops ; Remove-Item mvn-devops.zip
```

The project then looks like this:

```
my-maven-project/
  pom.xml
  mvn-devops/          committed: devops.sh, devops.bat, lib/, modules/, ...
  .devops/             created by setup, never committed (passwords, tokens)
```

2. Commit it:

```
git add mvn-devops && git commit -m "Add mvn-devops 1.0.0"
```

3. Use it from the project root:

```
mvn-devops/devops.sh setup        # Windows: mvn-devops\devops.bat setup
mvn-devops/devops.sh run
```

Commit `devops.conf` too. Everyone else clones the project and runs `mvn-devops/devops.sh setup`: the tools and answers come from `devops.conf`, so they are only asked for their own passwords, tokens and user names.

Upgrade with `mvn-devops/devops.sh upgrade`: it downloads the latest release, checks it against `SHA256SUMS` and replaces the folder; review the change with `git status` and commit it. `upgrade --check` only compares versions, `upgrade --version X` picks a release, and `--version` shows the version in use.

Notes:

- The folder has no `pom.xml`, so Maven, Sonar and the pipeline ignore it.
- Use the zip or tar.gz, not a `git clone`, which would put a repository inside the project's repository.
- On Linux and macOS keep the executable bit when committing (`unzip` and git keep it). If `devops.sh` lost it, run `git update-index --chmod=+x mvn-devops/devops.sh`.
- Line endings: Bash cannot run scripts with Windows (CRLF) line endings, and git on Windows (`core.autocrlf=true`) converts files to CRLF on checkout. The zip contains `mvn-devops/.gitattributes`, which keeps every file LF (and `devops.bat` CRLF) whatever `core.autocrlf` is, so commit it together with the folder. `devops.sh doctor` reports files that are already CRLF and prints the command that fixes them. A project that committed mvn-devops before this file existed adds it from the new zip, then fixes its checkout once with:

`bash git add --renormalize mvn-devops && git commit -m "Normalize mvn-devops line endings" rm -rf mvn-devops && git checkout -- mvn-devops`

8.3 · mvn-devops/docs/prerequisites.md

Installing the prerequisites

Tool	Needed for
Bash 4+, curl, jq, git, ssh-keygen	always
Docker with the compose plugin	tools that run in Docker
Java 21 and Maven 3.9	the `maven` orchestrator and `release` (Jenkins and Concourse bring their own)

`devops.sh doctor` (or `mvn-devops doctor`) checks all of them.

Debian, Ubuntu

```
sudo apt-get update
sudo apt-get install -y bash curl jq git openssh-client openjdk-21-jdk maven
## Docker Engine with the compose plugin: https://docs.docker.com/engine/install/ubuntu/
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"     # log out and in again
```

Fedora, RHEL, Rocky

```
sudo dnf install -y bash curl jq git openssh-clients java-21-openjdk-devel maven
## Docker Engine: https://docs.docker.com/engine/install/fedora/ (or /rhel/)
sudo dnf -y install dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/fedora/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker && sudo usermod -aG docker "$USER"
```

The Maven packages of Debian, Ubuntu and RHEL are often older than 3.9; check with `mvn -version` and install Maven by hand as described below if needed.

macOS

```
brew install bash jq git openjdk@21 maven
brew install --cask docker          # Docker Desktop, start it once
```

macOS ships Bash 3.2; Homebrew's Bash 4+ is used through `#!/usr/bin/env bash` when `/opt/homebrew/bin` (or `/usr/local/bin`) comes first in `PATH`.

Windows

1. Git for Windows: brings Git Bash, curl and ssh-keygen. `devops.bat` finds it.
2. jq: `winget install jqlang.jq`, or download `jq-windows-amd64.exe` from jqlang.github.io/jq, rename it to `jq.exe` and put it in a folder on `PATH`.
3. Docker Desktop with the WSL 2 backend.
4. Java 21: `winget install EclipseAdoptium.Temurin.21.JDK`.

5. Maven: download the binary zip from maven.apache.org, unpack it to e.g. `C:\sdk\maven`, then in a console run as administrator: `bat setx /M MAVEN_HOME C:\sdk\maven setx /M PATH "%PATH%;C:\sdk\maven\bin"`

Open a new console afterwards so `PATH` is reloaded.

Manual Maven install (any Linux, macOS)

```
curl -fsSLO https://archive.apache.org/dist/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-bin.tar.gz
sudo mkdir -p /opt/maven && sudo tar -xzf apache-maven-3.9.11-bin.tar.gz -C /opt/maven --strip-components=1
echo 'export PATH=/opt/maven/bin:$PATH' >> ~/.bashrc
```

Check

```
java -version && mvn -version && git --version && docker compose version && jq --version
```

8.4 · mvn-devops/docs/project-requirements.md

What your Maven project needs

A `pom.xml` that builds with `mvn package`. Nothing else.

The stages call every plugin by its full coordinates and configure it with `-D` properties, so the project needs no profiles, no `distributionManagement`, no plugin declarations and no settings file of its own:

```
org.apache.maven.plugins:maven-checkstyle-plugin:3.6.0:check -
Dcheckstyle.config.location=google_checks.xml
org.sonarsource.scanner.maven:sonar-maven-plugin:5.8.0.7211:sonar -Dsonar.host.url=$SONAR_URL -
Dsonar.token=$SONAR_TOKEN
package ... org.apache.maven.plugins:maven-deploy-plugin:3.2.0:deploy -
DaltSnapshotDeploymentRepository=nexus-snapshots::$NEXUS_ARTIFACTORY_SNAPSHOT_URL ...
```

`./devops.sh stages` prints the exact commands for the selected modules.

Why not a profiles.xml?

Maven 2 read profiles from a `profiles.xml` next to the pom; Maven 3 removed it. Profiles in `settings.xml` can only hold properties, repositories and activation rules, not plugins or `distributionManagement`. Calling the plugins directly is the way to keep all of that out of the project.

Settings and credentials

The framework passes templates/settings.xml as global settings (`-gs`). It declares one `<server>` per deploy target, with credentials read from environment variables the modules export:

Server id	Credentials
`github`	`GITHUB_USERNAME` , `GITHUB_PACKAGE_TOKEN`
`nexus-snapshots` , `nexus-releases`	`NEXUS_ARTIFACTORY_USERNAME` , `NEXUS_ARTIFACTORY_PASSWORD`
`jfrog-snapshots` , `jfrog-releases`	`JFROG_ARTIFACTORY_USERNAME` , `JFROG_ARTIFACTORY_ENCRYPTED_PASSWORD`

Jenkins and Concourse write the same file into the build container before the first stage.

Optional knobs

All of them are questions asked by `secrets` :

Value	Default	
`MAVEN_SETTINGS`	empty	the project's own settings file, added with `-s`
`MAVEN_PROFILES`	empty	project profiles to activate in every stage
`MAVEN_ATTACH_SOURCES`	yes	deploy sources and javadoc jars too
`MAVEN_CHECKSTYLE`	yes	run the checkstyle stage

Value	Default	
`MAVEN_CHECKSTYLE_CONFIG`	`google_checks.xml`	`sun_checks.xml`, a URL, or a file in the project such as `code-style/checkstyle.xml`
`SITE_BRANCH`	`site`	branch the site is published to

Plugin versions live in lib/maven.sh. Override one for a project by storing a value, for example `MVN_JAVADOC_VERSION`.

If the pom already configures one of these plugins (checkstyle rules, Sonar exclusions, site reports in `<reporting>`), that configuration still applies; the command line only fills in what the pom leaves open.

Site

The site stage builds the site with the reports in the pom's `<reporting>` section, or the default project information pages when there is none. `maven-site-plugin`'s own `stage` goal requires `<distributionManagement><site>` in the pom, so a short shell step (`stage-site`) assembles `target/staging` instead: the root site plus each first-level module's site in a sub directory. `publish-site` pushes it with `maven-scm-publish-plugin` to `git@github.com:<owner>/<repo>.git`, branch `site`. Pushing uses SSH: your own key when the `maven` orchestrator runs on your machine, and a deploy key (generated and registered on the repository by `configure`) when Jenkins or Concourse runs the build.

Create the `site` branch once and point GitHub Pages at it, as described in Project site on GitHub Pages.

Tokens

See github-setup.md for the GitHub tokens, their scopes and the SSH key.

8.5 · mvn-devops/docs/github-setup.md

GitHub tokens and SSH keys

Do this once before `setup`. `secrets` asks for the tokens; the SSH key is only needed when the `maven` orchestrator publishes the site from your machine.

Tokens

Create classic tokens under GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic) > Generate new token (direct link). On GitHub Enterprise it is the same menu on your server. Copy each token right away; GitHub shows it only once.

Token	Asked as	Scopes	Used for
Repository token	`GITHUB_TOKEN`	`repo`	Jenkins and Concourse clone the repository; `configure` checks the site branch and registers the deploy key
		`admin:repo_hook`	only with `JENKINS_TRIGGER=webhook`: `configure` registers the push webhook
Packages token (optional)	`GITHUB_PACKAGE_TOKEN`	`write:packages`, `read:packages`	the `deploy-github` stage. Empty: the repository token is used, so give it these scopes too
		`delete:packages`	only if you want to delete package versions later
		`write:packages` on `GITHUB_TOKEN`	the `image` stage of the `github-container` module pushes to ghcr.io with the repository token

With a public repository and no private dependencies, `public_repo` is enough instead of `repo`.

Fine-grained tokens work for the repository token: select the repository and give it *Contents: Read and write*, *Administration: Read and write* (deploy keys) and, for the webhook, *Webhooks: Read and write*. GitHub Packages does not accept fine-grained tokens yet, so the packages token must be a classic one.

Change a token later with `devops.sh secrets --reconfigure`, then `devops.sh publish` so the pipeline gets the new value.

SSH key (site publishing with the `maven` orchestrator)

`publish-site` pushes the site with `git@github.com:<owner>/<repo>.git`. With Jenkins or Concourse a deploy key is generated and registered for you. With the `maven` orchestrator your own key is used, so set it up once:

```
ssh-keygen -t ed25519 -C "my-laptop"                    # accept the default file
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519     # Windows: run in Git Bash
ssh-keyscan github.com >> ~/.ssh/known_hosts
cat ~/.ssh/id_ed25519.pub                               # add it at github.com/settings/ssh/new
ssh -T git@github.com                                   # "Hi <user>! You've successfully
authenticated"
```

Or add it with the GitHub CLI: `gh ssh-key add ~/.ssh/id_ed25519.pub`. `devops.sh doctor` runs the `ssh -T` check for the project's GitHub host.

8.6 · mvn-devops/docs/getting-started.md

Getting started, step by step

Every guide starts in the root of your Maven project, with Docker running. On Windows, use `mvn-devops\devops.bat` instead of `mvn-devops/devops.sh`.

- Guide 1: a ready-made pipeline
- Guide 2: your own choice of tools
- Guide 3: Maven in a container on another machine
- Guide 4: a ready-made pipeline file of your own
- Guide 5: a teammate joins

Before every guide

Step 1. Create a GitHub token with the `repo` scope (and `write:packages`). See github-setup.md.

Step 2. Add mvn-devops to the project:

```
curl -fsSLO https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip
unzip -q mvn-devops-1.0.0.zip && mv mvn-devops-1.0.0 mvn-devops && rm mvn-devops-1.0.0.zip
```

Step 3. Check the prerequisites:

```
mvn-devops/devops.sh doctor
```

Guide 1: a ready-made pipeline

Step 1. List the ready-made pipelines:

```
mvn-devops/devops.sh pipelines
```

Step 2. Set one up:

```
mvn-devops/devops.sh setup --pipeline jenkins-sonarqube-nexus-argocd
```

Questions:

Question	Answer
GitHub username	your user (asked only when git does not know it)
GitHub token	the token from *Before every guide*
Accept the Nexus Community Edition EULA?	`yes` (only with Nexus)

Step 3. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 4. Approve production when you want it deployed:

```
mvn-devops/devops.sh run --phase prod
```

Step 5. Open the tools:

```
mvn-devops/devops.sh urls
```

Step 6. Commit the settings:

```
git add devops.conf mvn-devops && git commit -m "Add mvn-devops"
```

Guide 2: your own choice of tools

Step 1. Choose the tools from the menu:

```
mvn-devops/devops.sh init
```

Questions (type the number, or several separated by commas):

Menu	Example answer
Pipeline orchestrator	`jenkins`
Code quality	`sonarqube`
Artifact repositories	`nexus`
Container image	`docker-registry`
Deployment	`kubernetes`
The other menus	`0` for none

Step 2. Answer the questions:

```
mvn-devops/devops.sh secrets
```

Press Enter to keep each default. Type only:

Question	Answer
GitHub token	the token from *Before every guide*
Deployment environments, in the order the image goes through them	Enter for `staging production`, or e.g. `dev test staging prod`
Approval before deploying to `<environment>`	Enter (only the last one needs it)
Accept the Nexus Community Edition EULA?	`yes` (only with Nexus)

Step 3. Start, configure and publish:

```
mvn-devops/devops.sh up
mvn-devops/devops.sh configure
mvn-devops/devops.sh publish
```

Step 4. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 5. Commit the settings:

```
git add devops.conf mvn-devops && git commit -m "Add mvn-devops"
```

Guide 3: Maven in a container on another machine

The pipeline runs in a container with Java and Maven, which checks the project out of GitHub. Nothing runs on your machine but `devops.sh`.

Step 1. Point Docker at the other machine (skip this step to use this one):

```
export DOCKER_HOST=ssh://user@build-vm
```

Step 2. Check that Docker answers:

```
mvn-devops/devops.sh doctor
```

Step 3. Push your commits; the container builds what is on GitHub:

```
git push
```

Step 4. Set it up:

```
mvn-devops/devops.sh setup --pipeline maven-container-kubernetes
```

Questions:

Question	Answer
GitHub username	your user (asked only when git does not know it)
GitHub token	the token from *Before every guide*

Step 5. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 6. Approve production:

```
mvn-devops/devops.sh run --phase prod
```

To choose the tools yourself instead of step 4, follow guide 2 and choose `maven-container` as the orchestrator.

Guide 4: a ready-made pipeline file of your own

Step 1. Write the file, for example `../team/jenkins-k8s.conf`:

```
## Jenkins and SonarQube with three environments.
ORCHESTRATOR=jenkins
TOOLS=sonarqube,docker-registry,kubernetes
ENVIRONMENTS=dev test prod
ENV_TEST_APPROVAL=yes
ENV_PROD_APPROVAL=yes
```

Step 2. Set it up:

```
mvn-devops/devops.sh setup --pipeline ../team/jenkins-k8s.conf
```

Questions: the GitHub token, and the Nexus EULA when Nexus is in `TOOLS`.

Step 3. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 4. Approve each environment that needs it, in order:

```
mvn-devops/devops.sh run --phase test
mvn-devops/devops.sh run --phase prod
```

Guide 5: a teammate joins

Step 1. Clone the project:

```
git clone https://github.com/acme/app.git && cd app
```

Step 2. Set it up; the tools come from the committed `devops.conf`:

```
mvn-devops/devops.sh setup
```

Questions: the GitHub username and token. Passwords are generated.

Step 3. Run the pipeline:

```
mvn-devops/devops.sh run
```

When something goes wrong

Problem	Command
A stage failed	`mvn-devops/devops.sh run --from <stage>`
See the stages	`mvn-devops/devops.sh stages`
Logs of a tool	`mvn-devops/devops.sh logs <service>`
Answer the questions again	`mvn-devops/devops.sh secrets --reconfigure`
Stop everything	`mvn-devops/devops.sh down`
Remove everything	`mvn-devops/devops.sh destroy`

More in troubleshooting.md.

8.7 · mvn-devops/docs/pipelines.md

Ready-made pipelines

A ready-made pipeline is a tested combination of tools with its settings, in the pipelines folder. Choose one instead of going through the menu and the questions:

```
mvn-devops/devops.sh pipelines                              # what there is
mvn-devops/devops.sh setup --pipeline jenkins-sonarqube-nexus
```

`setup` then asks only what cannot be chosen for you: your GitHub user and token, and whether you accept the Nexus licence when Nexus is part of it. Every password, token and key is generated for your project, so two projects set up from the same pipeline share no secrets. The images of the tools are downloaded when they start, as with any other setup.

Pipeline	Orchestrator	Tools	Environments
`maven-sonarqube-nexus`	maven	SonarQube, Nexus	
`jenkins-sonarqube-nexus`	Jenkins	SonarQube, Nexus	
`concourse-sonarqube-nexus`	Concourse	SonarQube, Nexus	
`maven-docker-host`	maven	registry, Docker Compose over SSH	dev test staging prod
`jenkins-kubernetes`	Jenkins	SonarQube, Nexus, registry, Trivy, Syft, Cosign, Kubernetes	dev test staging prod
`jenkins-observability`	Jenkins	registry, Docker Compose over SSH, PostgreSQL, Prometheus, Loki, k6	dev test staging prod
`concourse-argocd`	Concourse	registry, Argo CD, Vault, Sealed Secrets	dev test staging prod
`jenkins-sonarqube-nexus-argocd`	Jenkins	SonarQube, Nexus, registry, Argo CD	dev test staging prod
`maven-container-sonarqube-nexus`	maven-container	SonarQube, Nexus	
`maven-container-kubernetes`	maven-container	registry, Trivy, Kubernetes, PostgreSQL	dev staging prod
`maven-kubernetes`	maven	registry, Kubernetes	dev test staging prod
`concourse-kubernetes-vault`	Concourse	SonarQube, registry, Kubernetes, Vault, PostgreSQL	dev test staging prod
`jenkins-argocd-observability`	Jenkins	registry, Argo CD, Prometheus, Loki, k6	dev test staging prod
`concourse-docker-host-secure`	Concourse	registry, Trivy, Syft, Cosign, Docker Compose over SSH, PostgreSQL	dev staging prod
`maven-sonarqube-github`	maven	SonarQube, GitHub Packages, GitHub Pages	
`jenkins-complete`	Jenkins	SonarQube, Nexus, registry, Trivy, Syft, Cosign, Argo CD, Vault, PostgreSQL, Prometheus, Loki, k6	dev test staging prod

Only `prod` waits for approval, except in `concourse-kubernetes-vault`, where staging does too; see Environments. `concourse-argocd` releases prod as a canary. The end-to-end test runs every combination on GitHub's runners except `maven-sonarqube-github`, which needs a real GitHub repository. `jenkins-complete` is the heaviest: it starts every tool at once.

Step by step: getting-started.md.

What it writes

`init --pipeline` selects the pipeline's tools and writes its settings to the project's `devops.conf`, together with `PIPELINE=<name>`. From then on the project is like any other: commit `devops.conf`, and a teammate's `setup` gets the same tools and settings. Change a setting in `devops.conf`, or answer everything again with `devops.sh secrets --reconfigure`.

A pipeline of your own

A ready-made pipeline is a `devops.conf` without the project's name: the orchestrator, the tools as `--with` takes them, and any setting that is neither secret nor personal. The first comment line describes it.

```
## Jenkins and SonarQube with three environments.
ORCHESTRATOR=jenkins
TOOLS=sonarqube,docker-registry,kubernetes
ENVIRONMENTS=dev test prod
ENV_TEST_APPROVAL=yes
ENV_PROD_APPROVAL=yes
KUBERNETES_REPLICAS=1
```

Keep the file anywhere, for example in a repository your team shares, and pass its path:

```
mvn-devops/devops.sh setup --pipeline ../team/jenkins-k8s.conf
```

A file added to the `pipelines` folder of mvn-devops is listed by `devops.sh pipelines`.

8.8 · mvn-devops/docs/modules.md

Modules and stages

Category	Mode	Modules
Source control	required	`github`
Build	required	`maven` (validate, package, test, checkstyle, install)
Pipeline orchestrator	one	`maven` , `maven-container` , `jenkins` , `concourse`
Code quality	any	`sonarqube`
Artifact repositories	any	`jfrog` , `nexus` , `github-packages`
Project site	any	`github-pages`
Container image	one or none	`docker-registry` , `github-container`
Image security	any	`trivy` , `syft` , `cosign`
Deployment	one or none	`docker-host` , `kubernetes`
GitOps	one or none	`argocd` (adds `kubernetes`)
Database	one or none	`postgresql`
Secrets	any	`vault` , `sealed-secrets` (adds `kubernetes`)
Monitoring	any	`prometheus` (with Grafana), `loki` (adds `prometheus`)
Load test	one or none	`k6`

Artifactory OSS does not allow creating repositories through its API. After `configure` , open Artifactory, choose Quick Setup > Maven and enter the repository prefix you gave in `secrets` (`JFROG_ARTIFACTORY_REPOSITORY_PREFIX`), so `<prefix>-libs-release-local` and `<prefix>-libs-snapshot-local` exist. `configure` creates them itself on the Pro editions.

Default stages with every module selected (plugin coordinates shortened):

```
ORDER  PHASE      STAGE            MAVEN ARGUMENTS
10     ci   validate         validate
20     ci   build            clean package -DskipTests=true
30     ci   test             test
35     ci   migrate          flyway-maven-plugin:12.4.0:clean ...:migrate ...:validate -
Dflyway.url=$DATABASE_CI_URL ...
40     ci   checkstyle       maven-checkstyle-plugin:3.6.0:check -
Dcheckstyle.config.location=google_checks.xml
45     ci   sonar            sonar-maven-plugin:5.8.0.7211:sonar -Dsonar.host.url=$SONAR_URL -
Dsonar.token=$SONAR_TOKEN
50     ci   install          install -DskipTests=true
60     cd   site             maven-site-plugin:3.22.0:site
62     cd   stage-site       (shell) copy the root and module sites into target/staging
65     cd   publish-site     -N maven-scm-publish-plugin:3.3.0:publish-scm -
Dscmpublish.pubScmUrl=... -Dscmpublish.scmBranch=site
70     cd   deploy-jfrog     package source:jar-no-fork javadoc:jar maven-deploy-
plugin:3.2.0:deploy -DaltSnapshotDeploymentRepository=jfrog-
snapshots::$JFROG_ARTIFACTORY_SNAPSHOT_URL ...
71     cd   deploy-github    ... -DaltSnapshotDeploymentRepository=github::$GITHUB_PACKAGES_URL
...
72     cd   deploy-nexus     ... -DaltSnapshotDeploymentRepository=nexus-
snapshots::$NEXUS_ARTIFACTORY_SNAPSHOT_URL ...
75     cd   image            package -DskipTests=true jib-maven-plugin:3.5.2:build -
Djib.to.image=$IMAGE_REPOSITORY:$(git rev-parse --short=12 HEAD) ...
76     cd   sbom             (shell) sh "$DEVOPS_SCRIPTS/sbom.sh"
77     cd   scan-image       (shell) sh "$DEVOPS_SCRIPTS/scan-image.sh"
78     cd   sign-image       (shell) sh "$DEVOPS_SCRIPTS/sign-image.sh"
80     staging    deploy-staging          (shell) sh "$DEVOPS_SCRIPTS/deploy-compose.sh" staging
85     staging    load-test               (shell) sh "$DEVOPS_SCRIPTS/load-test.sh" staging
-      production approval                waits until someone approves production
79     production verify-image-production (shell) sh "$DEVOPS_SCRIPTS/verify-image.sh"
80     production deploy-production       (shell) sh "$DEVOPS_SCRIPTS/deploy-compose.sh"
production
```

The project's pom.xml needs no profiles, no distributionManagement and no settings file. Every plugin is called by its coordinates and configured with `-D` properties, and credentials come from the framework's own settings.xml, passed as global settings (`-gs`). Details and optional knobs (extra profiles, checkstyle rules, plugin versions) are in docs/project-requirements.md.

Container images and deployments are described in docs/deployment.md.

Adding a tool is one directory; see docs/module-guide.md.

8.9 · mvn-devops/docs/orchestrators.md

Orchestrators

maven runs each stage with `mvn` on your machine, with the pipeline variables exported only for that process. `render` also writes `.devops/generated/pipeline.sh` for IDE run configurations.

maven-container runs the same stages with `mvn` in a container from the official image `maven:<MAVEN_VERSION>-eclipse-temurin-<JAVA_VERSION>` (`MAVEN_IMAGE`), started by `run` on the Docker machine: this one, or the one `DOCKER_HOST` points to (where-tools-run.md). The container checks the branch (`GIT_BRANCH`) out of GitHub, so push your commits before `run`; it needs neither Java, Maven nor the checkout of this machine. It joins the network of the tools and reaches them by their service names, as Jenkins and Concourse do, and a volume keeps the Maven repository and the downloaded tools between runs. `run` takes the same options as with maven; `run --phase <environment>` deploys the commit that passed the part of the pipeline before it, not the newest one. `destroy` removes the volume.

jenkins builds an image from `jenkins/jenkins:lts-jdk21` with Maven 3.9 (copied from the official Maven image) and the needed plugins, skips the setup wizard and configures everything with Configuration as Code: the admin user, one secret-text credential per secret and a pipeline job generated from the stages. `run` triggers the job and streams its console. There is no login to the UI and no API token to create by hand: devops.sh talks to its own Jenkins with the admin password over the REST API.

Builds also start on every push, chosen with `JENKINS_TRIGGER` in `secrets`: `poll` (default) checks the repository every two minutes, `webhook` registers a GitHub webhook to `JENKINS_PUBLIC_URL/github-webhook/` (Jenkins must be reachable from GitHub, see docs/ngrok.md, and the token needs `admin:repo_hook`), `none` builds only on `run`. Either trigger starts working after the first build, which records the repository.

concourse runs `concourse quickstart` (web and worker in one container). The pipeline has a `ci` job, triggered by every push, a `cd` job that runs the ci and cd stages and deploys to the environments before the first approval, started by hand after `ci` passed, and a job per environment that needs approval (see below). Tasks run in `maven:3.9-eclipse-temurin-21`. `fly` is downloaded from the server.

Approvals

A deployment environment that needs approval (by default the last one, see Environments) runs its stages, and those of the environments after it up to the next approval, only after someone approves it, in the way each orchestrator offers:

Orchestrator	Waiting	Approve with
maven, maven-container	`run` stops before the environment	`devops.sh run --phase <environment>`
jenkins	the build stops at the `approve-<environment>` stage (an `input` step, kept for 7 days)	`devops.sh run --phase <environment>`, or *Deploy* in the build
concourse	the environment's job, after the job before it passed	`devops.sh run --phase <environment>`, or the job's + button

`prod` stands for the last environment. An approved environment gets the commit that passed the environments before it; it is not built again.

Stage scripts (`templates/scripts`) and the Helm chart (`templates/helm`) reach maven-container, Jenkins and Concourse as a compressed copy in the generated pipeline, unpacked into `.devops/` of the checkout before the first stage.

All orchestrators use the same Java and Maven, set with `JAVA_VERSION` (21) and `MAVEN_VERSION` (3.9) in `secrets` . Java 21 also builds projects whose pom targets an older release such as 17.

Docker images

Every tool runs from its official image, unchanged:

Tool	Image
SonarQube	`sonarqube:26.9.0.129388-community` (with `postgres:18`)
Nexus	`sonatype/nexus3`
Artifactory OSS	`releases-docker.jfrog.io/jfrog/artifactory-oss` (with `postgres:18`)
Concourse	`concourse/concourse` (with `postgres:18`)
Concourse build tasks, maven-container	`maven:<MAVEN_VERSION>-eclipse-temurin-<JAVA_VERSION>`
Docker registry	`registry:3`
Simulated deploy machine	built from `docker:cli` with openssh-server and curl
Kubernetes	`rancher/k3s:v1.37.1-k3s1` (`K3S_IMAGE_TAG`)
Argo CD, Argo Rollouts	the projects' release manifests (3.5.4, 1.10.0), applied to the cluster

Jenkins is the one exception. The official `jenkins/jenkins` image has Java but no Maven, and the pipeline stages are `mvn` commands run inside Jenkins. So `modules/orchestrator/jenkins/docker/Dockerfile` adds a thin layer built only from official images: it starts from `jenkins/jenkins:lts-jdk<JAVA_VERSION>` , copies Maven from `maven:<MAVEN_VERSION>-eclipse-temurin-<JAVA_VERSION>` , and adds git, ssh, curl, jq and the plugins in `plugins.txt` . `up` builds it locally; no image is published by mvn-devops. Java and Maven versions come from `JAVA_VERSION` (default 21) and `MAVEN_VERSION` (default 3.9), asked by the `maven` build module, so Jenkins, Concourse and your machine use the same versions.

Only stable releases are used. SonarQube Community Build has no long-term release and a new version each month, so it is pinned (`SONAR_IMAGE_TAG` overrides it); the other tools follow their stable `latest` or `lts` tag. SonarQube refuses to start when the disk holding Docker is more than 90% full.

8.10 · mvn-devops/docs/where-tools-run.md

Where the tools run

No tool has to run on `localhost`. Every tool with a server can be one of:

	How	**Asked in `secrets`**
started in Docker by devops.sh	on this machine, or on the machine `DOCKER_HOST` points to	`DEVOPS_HOST`: address of the Docker machine
an existing server	anywhere, with its own URL; nothing is started for it	`<TOOL>_SERVER_URL`, plus the credentials

Each tool is decided on its own, so any mix works: SonarQube on `https://sonar.acme.com`, Nexus in Docker on a VM, Jenkins on `https://jenkins.acme.com`, and the repository on GitHub Enterprise.

Tool	**Existing server**	**What devops.sh asks for it**
GitHub	`GITHUB_URL`, e.g. `https://github.acme.com` (detected from `origin`)	token as for github.com
SonarQube	`SONAR_SERVER_URL`	an analysis token (My Account > Security)
Nexus	`NEXUS_SERVER_URL`	a user that may deploy, its password, repository names
Artifactory	`JFROG_SERVER_URL`, ending in `/artifactory`	a user, its password, API key or identity token, repository names
GitHub Packages	`GITHUB_PACKAGES_REGISTRY` (`https://maven.<host>` on Enterprise)	
Jenkins	`JENKINS_SERVER_URL`	a user and its API token
Concourse	`CONCOURSE_SERVER_URL`	team, user and password

For an existing server, `configure` only checks the credentials and the repositories; it does not change passwords or create anything there. For Jenkins, `publish` creates or updates the job and its credentials through the REST API (credential ids get the prefix `<project>-`, because the server is shared). The agents need git, ssh, Java 21 and Maven 3.9, and the server the plugins workflow-aggregator, git, credentials-binding, plain-credentials and timestamper. For Concourse, `publish` downloads `fly` from the server and sets the pipeline in your team.

Leave a `*_SERVER_URL` empty to run that tool in Docker. Change your answers later with `devops.sh secrets --reconfigure`.

Tools in Docker on another machine

Clone mvn-devops and the project on the VM and run `devops.sh` there, or point Docker at the VM and run everything from your machine:

```
export DOCKER_HOST=ssh://user@build-vm      # or a docker context
../mvn-devops/devops.sh doctor              # docker daemon reachable on build-vm
../mvn-devops/devops.sh setup               # containers start on the VM
../mvn-devops/devops.sh run                 # mvn runs here and talks to build-vm:<port>
```

With the `maven-container` orchestrator mvn runs on the VM as well, in a container next to the tools, and your machine needs only Docker's client and `devops.sh` (getting-started.md).

`secrets` takes the default for `DEVOPS_HOST` from `DOCKER_HOST` or the docker context. The ports must be reachable from where the pipeline runs (firewall, security group). Jenkins in Docker mounts its configuration from `.devops/`, which a remote Docker daemon cannot see, so run `devops.sh` on the VM for it.

When an existing Jenkins or Concourse server runs the pipeline and some tools run in Docker, set `DEVOPS_HOST` to an address that server can reach; `publish` warns when it is `localhost`.

The compose file of your tools

`up` also writes the selected tools as one plain compose project, so you can read it, keep it and run it without devops.sh:

```
../mvn-devops/devops.sh export-compose           # .devops/compose/docker-compose.yml + .env
../mvn-devops/devops.sh export-compose ./docker  # or into a folder of the project
cd docker && docker compose up -d
```

`docker-compose.yml` merges the compose fragments of the selected modules; paths are written out and every other `${VAR}` (ports, passwords) is read from the `.env` next to it. `.env` holds the passwords, so it is `chmod 600` and, outside `.devops`, added to a `.gitignore` in that folder. The compose project name is `devops-<project>`, the same one devops.sh uses, so both manage the same containers. Tools on an existing server are not in the file.

8.11 · mvn-devops/docs/deployment.md

Container images and deployment

These modules turn the build into something that runs: an image in a registry, and later a running application. They are capabilities for your project; select the ones you want in `init`.

The hello-api example is a small web application with a health endpoint (`/actuator/health`) that the end-to-end tests deploy; hello-data adds a PostgreSQL database with Flyway migrations.

Environments

The image is built once and the same image goes through the deployment environments one after the other. `ENVIRONMENTS` in `devops.conf` lists them in that order; a project that does not set it has `staging production`. The names commonly used for four environments are:

```
ENVIRONMENTS=dev test staging prod
```

An environment either follows the one before it on its own, or waits until someone approves it (`ENV_<NAME>_APPROVAL=yes`; by default only the last one waits). `secrets` asks for each. The environments after an approved one follow it up to the next one that needs approval, so the pipeline falls into parts:

`ENVIRONMENTS`	Approval	What runs
`staging production`	production	`run`: ci, cd, staging. `run --phase production`: production
`dev test staging prod`	prod	`run`: ci, cd, dev, test, staging. `run --phase prod`: prod
`dev test staging prod`	staging, prod	`run`: ci, cd, dev, test. `run --phase staging`: staging. `run --phase prod`: prod

`prod` also stands for the last environment, whatever it is called, in `run --phase prod` and `rollback prod`. `devops.sh stages` shows each environment's stages and where an approval comes. How each orchestrator asks for approval is in orchestrators.md.

Everything that belongs to an environment is per environment: its machine or namespace, its port, its secrets in Vault, its database and password, its metrics and logs. Values follow the pattern `<PREFIX>_<NAME>_<SETTING>`, for example `DEPLOY_DEV_PORT` or `DATABASE_PROD_PASSWORD`. Default ports count up from the last environment: with `dev test staging prod` the simulated machine serves prod on 8180, staging on 8181, test on 8182 and dev on 8183 (k3s: 8280 and up). Names are lowercase letters and digits, starting with a letter.

Container image

Choose one module of the *Container image* category:

Module	Registry
`docker-registry`	The Distribution registry in Docker (port 5000), or an existing registry given as `REGISTRY_SERVER_URL`: `https://docker.io`, Harbor, a Docker repository of Nexus or Artifactory

Module	Registry
`github-container`	The GitHub Container Registry (`ghcr.io/<owner>/<image>`), with the GitHub user and token. The token needs `write:packages` (github-setup.md)

The `image` stage (cd phase, order 75) builds the application and pushes it with two tags:

- the commit, `git rev-parse --short=12 HEAD` . It never changes, and it is what deployments use, so a rollback is just the previous tag;
- `latest` , for people.

Question	Default	Meaning
`IMAGE_NAME`	the project name	repository name in the registry; add the namespace where the registry needs one (`acme/app` on Docker Hub)
`IMAGE_MODULE`	empty	Maven module that is the application, in a multi-module project
`IMAGE_BUILDER`	`jib`	`jib` , or `dockerfile` to build the project's own `Dockerfile`
`IMAGE_BASE`	`eclipse-temurin:<java>-jre`	base image (Jib)
`IMAGE_PORT`	`8080`	port the application listens on (Jib)

Jib (the default) builds the image from the Maven build without Docker and without a Dockerfile, so it works in every orchestrator. **A Dockerfile** is used by default when the project has one in its root and the pipeline runs on this machine (the `maven` orchestrator); Jenkins and Concourse run without a Docker daemon and need Jib.

The pipeline gets these variables:

Variable	Meaning
`IMAGE_REPOSITORY`	where the pipeline pushes, as it reaches the registry
`IMAGE_DEPLOY_REPOSITORY`	the same repository as the machines that run the image pull it
`IMAGE_REGISTRY_USERNAME` , `IMAGE_REGISTRY_PASSWORD`	credentials, empty for the local registry

The local registry has no TLS and no login. Docker accepts it without configuration only as `localhost:<port>` , which is why the deploy repository uses `localhost` . To pull from it on another machine, add it to `insecure-registries` in that machine's `/etc/docker/daemon.json` , or use a registry with TLS.

Check the result:

```
curl -s http://localhost:5000/v2/hello-api/tags/list
docker run --rm -p 8080:8080 localhost:5000/hello-api:latest
```

Image security

The *Image security* modules work on the pushed image, between the image stage and the deployment:

Module	Stage	What it does
`syft`	`sbom` (76, cd)	writes the software bill of materials of the image to `target/sbom.spdx.json` (SPDX) and `target/sbom.cdx.json` (CycloneDX)
`trivy`	`scan-image` (77, cd)	scans the operating system packages and the jars in the image, writes `target/trivy-report.json` and prints the findings of `TRIVY_SEVERITY` (HIGH,CRITICAL). Fails when there are vulnerabilities of `TRIVY_FAIL_ON` (CRITICAL) that have a fix; empty means report only
`cosign`	`sign-image` (78, cd)	signs the image by digest with the project's key and, with `syft`, attaches the SBOM as a signed attestation
`cosign`	`verify-image-<environment>` (79)	with a deployment module, in each environment that needs approval: it only gets an image signed with that key

The tools are not installed by hand: `templates/scripts/tool.sh` downloads the pinned release where the pipeline runs (your machine, the Jenkins container or the Concourse task), checks it against the release's checksum file and caches it in `~/.cache/mvn-devops/tools` (`DEVOPS_TOOLS`). Trivy's vulnerability database is cached there too. `TRIVY_VERSION`, `SYFT_VERSION` and `COSIGN_VERSION` override a version.

The Cosign key is generated by `setup` (`.devops/keys/cosign.key` and `cosign.pub`, protected by `COSIGN_PASSWORD`), or taken from `COSIGN_KEY_FILE`. The pipeline gets it as a secret. Keep a copy of the key: images signed with it can only be verified with its public key. Signatures go to the image's registry (OCI referrers); nothing is sent to the public Sigstore transparency log. Check a signature yourself with:

```
cosign verify --key .devops/keys/cosign.pub --insecure-ignore-tlog=true --allow-http-registry
localhost:5000/hello-api:latest
```

Deploying to a machine

The `docker-host` module (*Deployment* category) runs the image with Docker Compose on a machine it reaches over SSH:

Stage	Phase	What happens
`deploy-<environment>` (80)	the environment	deploys the image of the commit to the environment's machine, after its approval when it needs one

Each deployment:

1. writes `compose.yml` and the tag to `~/mvn-devops/<image name>-<environment>/` on the machine and runs `docker compose up`;
2. calls the health endpoint (`DEPLOY_HEALTH_PATH`, default `/actuator/health`) for up to two minutes;
3. when it does not answer, puts back the image that ran before and fails the stage.

The container gets `DEPLOY_ENVIRONMENT` (the environment's name), the environment's secrets from Vault (see Secrets), written to `secrets.yml` in that directory, and the settings in `app.env` there, which you keep on the machine; it is never overwritten.

The machines

Question	Default	Meaning
`DEPLOY_SERVER_URL`	empty	machine of the first environment, e.g. `ssh://deploy@staging.example.com:22`; empty: a simulated machine in Docker for all of them
`DEPLOY_<NAME>_SERVER_URL`	the first machine	machine of each other environment, e.g. `DEPLOY_PRODUCTION_SERVER_URL`
`DEPLOY_SSH_KEY_FILE`	empty	private key that may log in; empty: `setup` generates `.devops/keys/deploy`
`DEPLOY_<NAME>_PORT`	8180 for the last, counting up	port of the application on the machine, e.g. `DEPLOY_STAGING_PORT` 8181
`DEPLOY_HEALTH_PATH`	`/actuator/health`	health check

A real machine needs Docker with the compose plugin, curl or wget, and a user in the `docker` group whose `~/.ssh/authorized_keys` holds the public key (`devops.sh get DEPLOY_SSH_PUBLIC_KEY`). `configure` records the machines' SSH host keys, which the pipeline then checks, and tells you when the key cannot log in yet. The machine pulls `IMAGE_DEPLOY_REPOSITORY`, logging in with the registry user when there is one.

The simulated machine is a container with an SSH server and the Docker CLI, using the Docker daemon of your machine. The applications it starts are ordinary containers on your machine:

```
curl http://localhost:8181/hello     # staging
curl http://localhost:8180/hello     # production
```

Rollback

```
devops.sh rollback                     # the last environment: the image that ran before
devops.sh rollback staging
devops.sh rollback prod --to 3f2a9c1d4e5b
```

Rolling back twice returns to where you started. Every image stays in the registry under its commit tag, so `--to` can go further back.

Each deployment replaces the container, so the application is down for the few seconds it takes to start. Zero-downtime updates come with Kubernetes.

Deploying to Kubernetes

The `kubernetes` module (*Deployment* category) installs the application with Helm, one release per environment:

Stage	Phase	What happens
`deploy-<environment>` (80)	the environment	`helm upgrade --install` in the namespace `<app>-<environment>`, after its approval when it needs one

Helm waits until the new pods pass their readiness probe (the health path). The update is rolling: a new pod must be ready before an old one stops, so the application stays up. When the new version does not become ready within `KUBERNETES_TIMEOUT` (5m), Helm rolls the release back and

the stage fails.

Question	Default	Meaning
`KUBERNETES_SERVER_URL`	empty	an existing cluster; empty: k3s in Docker
`KUBERNETES_KUBECONFIG`	`~/.kube/config`	kubeconfig of the existing cluster, passed to the pipeline as a secret
`KUBERNETES_<NAME>_PORT`	8280 for the last, counting up	k3s: the application on this machine, e.g. `KUBERNETES_STAGING_PORT` 8281
`KUBERNETES_<NAME>_NODE_PORT`	30080 for the last, counting up, with k3s	node port of the service; empty: a ClusterIP service for your ingress
`KUBERNETES_CHART`	empty	the project's own chart, e.g. `deploy/chart`; empty: the generic chart
`KUBERNETES_REPLICAS`	2	pods per environment
`DEPLOY_HEALTH_PATH`	`/actuator/health`	readiness and liveness probe

The generic chart (templates/helm/app) has a Deployment with readiness and liveness probes, resource requests, a rolling update strategy and a Service. The container gets `DEPLOY_ENVIRONMENT`, and every key of the Secret `<app>-env` in its namespace. With Vault (see Secrets) the release creates that Secret from the environment's secrets (`secretEnv`) and the pods wait for it; without, it is used when it exists:

```
kubectl --namespace hello-api-production create secret generic hello-api-env --from-
literal=GREETING=hello
```

A private registry gets an image pull secret from the registry credentials. **A chart of your own** receives the same values: `image.repository`, `image.tag`, `registryAuth`, `environment`, `replicas`, `containerPort`, `healthPath`, `service.nodePort`, and `secretEnv` or `sealedSecretEnv`.

k3s in Docker is a one-node cluster on your machine. It pulls from the local registry through its service name, so the image reference stays `localhost:5000/<image>`. Use it from your machine with:

```
export KUBECONFIG=.devops/k3s/kubeconfig-host.yaml
kubectl get pods --namespace hello-api-staging
curl http://localhost:8281/hello
```

`devops.sh rollback` goes back to the previous Helm revision (`helm rollback`); `--to TAG` deploys an earlier image again.

GitOps with Argo CD

With the `argocd` module (*GitOps* category, it adds `kubernetes`) the pipeline does not deploy to the cluster itself. It writes the desired state to git, and Argo CD, running in the cluster, makes the cluster match it:

Stage	Phase	What happens
`deploy-<environment>` (80)	the environment	commits `environments/<environment>/` to the GitOps branch and waits until Argo CD reports the application synced and healthy

Each environment directory holds the chart and `environment.yaml` with the image tag and the environment's values, so the branch's history is the history of every release, and a pull request against it is a release proposal. When a release does not become healthy within `GITOPS_TIMEOUT` (600 seconds), the script commits the previous release again and the stage fails. `devops.sh rollback` is such a commit too.

Question	Default	Meaning
`GITOPS_BRANCH`	`gitops`	branch of the project's repository with the desired state; created by the first release
`GITOPS_CANARY`	`yes`	the last environment as a canary with Argo Rollouts
`ARGOCD_HOST_PORT`	8443	k3s: Argo CD's web console on this machine (`admin` , `devops.sh get ARGOCD_ADMIN_PASSWORD`)

`configure` installs Argo CD and Argo Rollouts from their release manifests, adds the repository with the GitHub token, and creates one Application `<app>-<environment>` per environment, which syncs automatically, prunes what was removed from git and undoes changes made by hand.

The canary replaces the Deployment with an Argo Rollouts `Rollout`: the new version first gets a quarter, then half of the pods, with a pause after each step, and then all of them. Its pods must become ready within three minutes, or Argo Rollouts aborts and keeps the stable version. Change the steps with `canary.steps` in a chart of your own.

Images from a private registry need an image pull secret in the namespace; GitOps never writes credentials to git. Create it once, or keep it in git encrypted with Sealed Secrets.

Secrets

The `vault` module (*Secrets* category) keeps the application's secrets in HashiCorp Vault, one KV secret per environment:

```
secret/<app>/staging      e.g. GREETING=..., SPRING_DATASOURCE_PASSWORD=...
secret/<app>/production
```

The deploy stage reads the secret of its environment (app-secrets.sh) and the application gets each key as an environment variable, which Spring Boot maps to its properties. Where they go depends on the deployment:

Deployment	Where the secrets are
`docker-host`	`secrets.yml` next to the compose file on the machine, readable only by the deploy user
`kubernetes`	the Secret `<app>-env` of the Helm release; new pods when it changes
`argocd` with `sealed-secrets`	a SealedSecret in `environments/<environment>/environment.yaml` of the GitOps branch, encrypted for the cluster
`argocd` alone	the Secret `<app>-env` , put in the cluster by the pipeline, not in git

A deployment picks up changed secrets the next time it runs; a rollback with Helm (`rollback` without `--to`) brings back the secrets of that revision.

Question	Default	Meaning
`VAULT_SERVER_URL`	empty	an existing Vault; empty: Vault in Docker

Question	Default	Meaning
`VAULT_TOKEN`		existing Vault: a token that may read `<mount>/data/<app>/*`; it should be periodic or long-lived
`VAULT_HOST_PORT`	8200	Vault in Docker: its port on this machine
`VAULT_KV_MOUNT`	`secret`	mount path of the KV version 2 secrets engine

Vault in Docker keeps its data in a volume. `configure` initialises it once (one unseal key; the key and the root token stay in `.devops/values`), enables the KV engine, and gives the pipeline a token that may only read the application's secrets and is renewed on every run. Vault starts sealed after every restart: run `devops.sh configure` to unseal it. Write secrets in the web console (`devops.sh urls`) or with the Vault CLI in the container:

```
./devops.sh compose exec -e VAULT_ADDR=http://127.0.0.1:8200 \
  -e VAULT_TOKEN="$(./devops.sh get VAULT_ROOT_TOKEN)" vault \
  vault kv put secret/hello-api/staging GREETING=Hi
```

Sealed Secrets (`sealed-secrets`, adds `kubernetes`): `configure` installs the controller in `kube-system`. With Argo CD the pipeline encrypts each secret with `kubeseal` for its namespace and name (strict scope), so the release in git carries it, and only the controller in the cluster can decrypt it. Seal secrets of your own the same way:

```
kubectl create secret generic db --dry-run=client --output yaml --from-literal=password=... \
  | kubeseal --controller-namespace kube-system --format yaml > sealed-db.yaml
```

Database

The `postgresql` module (*Database* category) gives the application a PostgreSQL database in each environment and checks its schema migrations before anything is deployed. The schema lives in Flyway migrations in the project (`src/main/resources/db/migration/V1__....sql`), and the application applies them when it starts, as Spring Boot does with `flyway-core` on the classpath.

Stage	Phase	What happens
`migrate` (35)	ci	empties the ci database and applies every migration from scratch with the Flyway Maven plugin, then validates them; a broken migration fails the build

Each deployment runs a PostgreSQL next to the application, its data kept across releases and rollbacks:

Deployment	The database
`docker-host`	a `db` container in the application's compose project on the machine, its data in a volume there
`kubernetes`, `argocd`	a StatefulSet `<app>-db` with a persistent volume in the environment's namespace

The application gets `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` and `SPRING_DATASOURCE_PASSWORD` with the environment's secrets, so a Spring Boot application needs no settings of its own. With Vault, the same names in the environment's secret win: set them there to use a managed database instead. When a release does not become healthy because a

migration fails, the previous version runs on: PostgreSQL rolls the failed migration back. Write migrations the previous release can live with (add a column first, remove the old one a release later), because a rollback does not undo them.

Question	Default	Meaning
`DATABASE_NAME`	the image name, e.g. `hello_data`	database and user name in every environment
`DATABASE_HOST_PORT`	5433	the ci database on this machine
`FLYWAY_VERSION`	12.4.0	the Flyway version of the project, which Spring Boot manages (4.1: 12.4.0)
`FLYWAY_LOCATIONS`	`filesystem:src/main/resources/db/migration`	where the migrations are

The passwords of the ci database and of each environment's database are generated (`devops.sh get DATABASE_PRODUCTION_PASSWORD`). A database keeps the password it was created with: after changing one, change it in the database too (`ALTER USER`).

Monitoring

The *Monitoring* modules watch the deployed application in each environment. They run in Docker next to the other tools.

Prometheus and Grafana (`prometheus`): Prometheus scrapes the application's metrics in each environment every 15 seconds, labelled `application` and `environment`, and keeps them for 15 days. Grafana opens on the dashboard *Application*: whether each environment is up, requests per second, failed requests, response times (95th percentile and average), JVM heap, CPU and database connections. Prometheus raises two alerts, visible at `/alerts` : *ApplicationDown* when an environment does not answer for two minutes, and *HighErrorRate* when more than 5% of its requests fail.

The application needs Micrometer's Prometheus registry, as in the examples:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
  <scope>runtime</scope>
</dependency>
```

The deployments expose its endpoint (`/actuator/prometheus`) and turn on the response time histograms through the application's environment (`MANAGEMENT_ENDPOINTS_WEB_EXPOSURE_INCLUDE`). Prometheus finds the environments from the deployment: the machines and ports of `docker-host` , or the ports of k3s. A cluster of your own is out of its reach; run Prometheus in the cluster there.

Question	Default	Meaning
`PROMETHEUS_HOST_PORT`	9090	Prometheus on this machine
`GRAFANA_HOST_PORT`	3000	Grafana on this machine (`admin` , `devops.sh get GRAFANA_ADMIN_PASSWORD`)
`METRICS_PATH`	`/actuator/prometheus`	where the application's metrics are

Loki (`loki` , adds `prometheus`): Loki keeps the application's logs and Grafana searches them, on the dashboard *Application logs* or in Explore with queries such as `{environment="production"} |= "ERROR"` . Grafana Alloy collects them with the labels `application` , `environment` , `container` (`app` , `db`) and `source` : from the containers of the simulated machine, which run on the same Docker, and from the pods of k3s. Machines and clusters of your own need an agent there, such as Alloy, that pushes to Loki's push URL (`devops.sh urls`).

Question	Default	Meaning
`LOKI_HOST_PORT`	3100	Loki on this machine

The configuration of Prometheus, Grafana and Alloy is written to `.devops/monitoring` by `devops.sh up` ; run `up` and `configure` again after changing the deployment.

Load test

The `k6` module (*Load test* category) load tests one environment (`LOAD_TEST_ENVIRONMENT` , by default the one before the last, e.g. staging) with k6 right after each deployment there:

Stage	Phase	What happens
`load-test` (85)	that environment	`LOAD_TEST_VUS` virtual users call it for `LOAD_TEST_DURATION` ; the stage fails when a threshold fails

Without a script of its own, the project gets load-test.js: each virtual user calls every path of `LOAD_TEST_PATHS` and pauses a second. It fails when more than `LOAD_TEST_MAX_ERROR_RATE` of the requests fail or the 95th percentile of the response times is above `LOAD_TEST_P95_MS` . A failed load test stops the release there: the environments after it keep the version they have.

Question	Default	Meaning
`LOAD_TEST_ENVIRONMENT`	the one before the last	environment the load test calls
`LOAD_TEST_URL`	found from the deployment	that environment as the pipeline reaches it; asked for a cluster of its own
`LOAD_TEST_SCRIPT`	empty	the project's k6 script, e.g. `src/test/k6/load.js` ; it gets `BASE_URL` and the values below as `__ENV`
`LOAD_TEST_PATHS`	`/actuator/health`	paths the generic script calls, comma separated, e.g. `/hello,/actuator/health`
`LOAD_TEST_VUS`	10	virtual users
`LOAD_TEST_DURATION`	`30s`	how long
`LOAD_TEST_P95_MS`	500	highest 95th percentile response time in milliseconds
`LOAD_TEST_MAX_ERROR_RATE`	0.01	highest share of failed requests

The summary is written to `target/load-test.json` . With the `prometheus` module k6 also sends its metrics to Prometheus (`k6_http_reqs_total` , `k6_http_req_duration_p95` and others, labelled `environment`), so a load test shows next to the application's own metrics in Grafana. k6 is downloaded where the pipeline runs, like the other tools.

8.12 · mvn-devops/docs/github-pages.md

Project site on GitHub Pages

The `github-pages` module builds the Maven site and pushes it to a `site` branch, which GitHub Pages serves. Two things have to be done once by hand.

1. Create the `site` branch in your project repository. It is an orphan branch that only holds the published site:

```
git checkout --orphan site
git rm -rf --cached . > /dev/null
echo "site" > index.html
git add index.html
git commit -m "Initialize site"
git push origin site
git checkout -f main
```

`git checkout -f main` restores your working tree; untracked files such as `.devops/` are left alone. Use another branch name by answering the `SITE_BRANCH` question in `secrets`.

2. Point GitHub Pages at it. In the GitHub repository go to Settings > Pages, and under Build and deployment set:

- Source: Deploy from a branch
- Branch: `site`, folder `/ (root)`

The site is then served at `https://<owner>.github.io/<repo>/`.

Publishing. The site stages run in the `cd` phase, after the ci stages:

```
./devops.sh run --phase cd            # site, stage-site, publish-site and the deploys
./devops.sh run --only publish-site   # push an already built and staged site again
```

Maven by hand. The same steps as plain Maven commands, run in the project root. No profiles or settings are needed:

```
## build the site, including module sites
mvn -B org.apache.maven.plugins:maven-site-plugin:3.22.0:site

## preview it at http://localhost:8000
mvn org.apache.maven.plugins:maven-site-plugin:3.22.0:run -Dport=8000

## publish target/staging to the site branch (after ./devops.sh run --only stage-site)
mvn -B -N org.apache.maven.plugins:maven-scm-publish-plugin:3.3.0:publish-scm \
  -Dscmpublish.pubScmUrl=scm:git:git@github.com:<owner>/<repo>.git \
  -Dscmpublish.scmBranch=site
```

`./devops.sh stages` prints the exact command of every stage for your selection. Pushing uses SSH: your own key with the `maven` orchestrator, and a deploy key that `configure` registers on the repository with Jenkins or Concourse.

8.13 · mvn-devops/docs/releasing.md

Releasing your project

`devops.sh release` releases the Maven project without maven-release-plugin, so the pom needs no `<scm>` and no `<distributionManagement>`:

1. sets the release version in every module (`1.2.0-SNAPSHOT` becomes `1.2.0`), commits "Release 1.2.0" and tags `v1.2.0`
2. runs the deploy stages of the selected artifact repositories
3. sets the next development version (`1.2.1-SNAPSHOT`) and commits it
4. pushes the branch and the tag

```
devops.sh release --dry-run                          # show what would happen
devops.sh release                                    # 1.2.0, then 1.2.1-SNAPSHOT
devops.sh release --version 2.0.0 --next 2.1.0-SNAPSHOT
devops.sh release --no-push                          # check locally, push yourself
```

It runs on your machine with `mvn` and git, needs a clean working tree and pushes with your own git credentials or SSH key. Nothing is pushed before step 4: if a step fails, the release commit and the tag are rolled back. To undo a release that was already pushed, delete the tag (`git push origin :refs/tags/v1.2.0`) and revert the two commits.

8.14 · mvn-devops/docs/ngrok.md

Reaching tools on your machine from the internet (ngrok)

Only needed when GitHub has to call a tool that runs on your machine or a VM without a public address, which is the Jenkins push webhook (`JENKINS_TRIGGER=webhook`). Polling (`poll` , the default) and Concourse need nothing from outside.

1. Create an account at ngrok.com and install ngrok: - Windows: unzip it into, e.g., `C:\sdk\ngrok` and add that folder to `PATH` . - Linux: `sudo snap install ngrok` , or unpack the tgz into `/opt/ngrok` and add it to `PATH` . - macOS: `brew install ngrok` .
2. Add your auth token from the ngrok dashboard: `bash ngrok config add-authtoken <token>`
3. Start a tunnel to Jenkins (use your `JENKINS_HOST_PORT` , 8080 by default): `bash ngrok http 8080` With a free account the URL changes on every start; a reserved domain (`ngrok http --url=<name>.ngrok-free.app 8080`) keeps it stable.
4. Give that URL to mvn-devops and register the webhook: `bash devops.sh secrets --reconfigure # JENKINS_TRIGGER=webhook, JENKINS_PUBLIC_URL=https://<name>.ngrok-free.app devops.sh publish # registers <url>/github-webhook/ on the repository` The GitHub token needs `admin:repo_hook` for this (github-setup.md).

Several tools at once can be described in the ngrok config file (`ngrok config edit`) and started with `ngrok start --all` :

```
tunnels:
  jenkins:   { addr: 8080, proto: http }
  sonarqube: { addr: 9000, proto: http }
  nexus:     { addr: 8084, proto: http }
```

When the URL changes, run `secrets --reconfigure` and `publish` again; the old webhook can be removed under Settings > Webhooks.

8.15 · mvn-devops/docs/ide.md

IDE settings (IntelliJ IDEA)

Checkstyle

The `checkstyle` stage checks the code with `MAVEN_CHECKSTYLE_CONFIG` (`google_checks.xml` unless you chose your own file). To see the same warnings while you type:

1. Install the CheckStyle-IDEA plugin.
2. Settings > Tools > Checkstyle: add a configuration file. Use the project's own file (e.g. `code-style/checkstyle.xml`) or the bundled Google checks, whichever the pipeline uses, and make it active.
3. To format code the same way: Settings > Editor > Code Style > Java > (gear icon) > Import Scheme > CheckStyle Configuration, and pick the same file.

Test coverage

If IntelliJ's own coverage runner reports wrong numbers or slows tests down, open Help > Find Action > `Registry...` and turn off:

- `idea.coverage.new.sampling.enable`
- `idea.coverage.test.tracking.enable`
- `idea.coverage.tracing.enable`

Pipeline variables in the IDE

Run configurations started from the IDE do not see the pipeline variables. `devops.sh render` writes `.devops/generated/pipeline.sh` (run it as a shell run configuration), and on Windows `devops.sh env --windows` writes `.devops/generated/set-env.bat`, which sets them with `setx` for the IDE.

8.16 · mvn-devops/docs/troubleshooting.md

Troubleshooting

Start with `devops.sh doctor`: it checks the tools, Docker, the Java and Maven versions, your SSH key on GitHub and the line endings of the framework files. `devops.sh logs <service>` shows a tool's own log, and `devops.sh status` its containers.

Running devops.sh

`mvn-devops needs Bash 4 or newer` on macOS, whose own Bash is 3.2. Install a newer one with `brew install bash` and open a new terminal; `#!/usr/bin/env bash` then finds it.

`$'\r': command not found`, `bad interpreter` or **`invalid option name`**: the scripts got Windows (CRLF) line endings, usually from git on Windows with `core.autocrlf=true`. `doctor` lists the files and prints the command that fixes them. Keep `mvn-devops/.gitattributes` committed so it does not happen again; installation.md shows how to repair a checkout.

`Cannot connect to the Docker daemon`: start Docker Desktop, or the docker service on Linux. With `DOCKER_HOST` set, check that the other machine is reachable (`docker info`).

A question is not asked any more, or a teammate's answer is used: answers in the committed `devops.conf` win over your own. Run `devops.sh secrets --reconfigure` to answer again (the new answer is written back to `devops.conf` for everyone), or edit `devops.conf` and commit it.

Starting the tools

`port is already allocated` during `up`: another program (or another project's tool) uses the port. Change it, e.g. `NEXUS_HOST_PORT`, with `devops.sh secrets --reconfigure` or in `devops.conf`, then run `up` again.

SonarQube never comes up and its log shows `high disk watermark` or `no_shard_available_action_exception`: SonarQube's search engine does not start when the disk holding Docker is more than 90% full. Free space, for example with `docker system prune`, then `devops.sh down` and `up`.

A tool keeps restarting after its version changed: the tools' data is kept in Docker volumes, and a newer version may not read an older one's data. `devops.sh destroy` removes the containers and volumes, then `up` and `configure` start them fresh.

Jenkins in Docker on another machine does not see its configuration: it mounts files from `.devops/`, which only exist on the machine running devops.sh. Run devops.sh on the Docker machine for Jenkins.

Running the pipeline

Deploy to Nexus fails with 400 or 403 although the password is right: Nexus Community Edition accepts uploads only after its EULA is accepted. Set `NEXUS_ACCEPT_EULA` to `yes` with `secrets --reconfigure` and run `configure`.

Deploy to Artifactory fails with 404: Artifactory OSS does not let devops.sh create repositories. Open Artifactory, choose Quick Setup > Maven and enter the prefix from `JFROG_ARTIFACTORY_REPOSITORY_PREFIX` (modules.md).

Checkstyle fails: the default rules are `google_checks.xml`. Point `MAVEN_CHECKSTYLE_CONFIG` at your own file, or set `MAVEN_CHECKSTYLE` to `no` (project-requirements.md).

Jenkins does not build on push: with `poll` or `webhook`, the trigger starts working after the first build, which records the repository. Run `devops.sh run` once. A webhook also needs Jenkins reachable from GitHub (ngrok.md).

`publish-site` is refused: the `site` branch must exist and your SSH key (with the `maven` orchestrator) or the deploy key must be allowed to push (github-pages.md).

Starting over

```
devops.sh destroy     # containers, volumes, deploy key and webhook; asks before deleting .devops/
devops.sh setup       # everything again; devops.conf keeps the team's answers
```

8.17 · mvn-devops/docs/design.md

Design

The idea

Each tool is a self-contained module with its own scripts. `devops.sh` asks which tool you want from each category, then runs only the scripts of those tools, in order, until everything is running, configured and wired into a pipeline that is ready to run.

Categories and modules

Category	Choice	Modules
Source control	required	github
Build	required	maven (validate, package, test, checkstyle, install)
Pipeline orchestrator	one	maven (on your machine), maven-container, jenkins, concourse
Code quality	any	sonarqube
Artifact repositories	any	jfrog, nexus, github-packages
Project site	any	github-pages

A new tool only needs a new directory under `modules/<category>/<tool>/` ; the menu finds it on its own (module-guide.md).

A module

```
modules/artifact/nexus/
  module.conf    name, description, modules it depends on
  compose.yml    the tool's containers
  module.sh      hooks
```

Every hook is one step of the life cycle, and every hook is optional:

Hook	Job
`module_secrets`	ask for the values the tool needs (port, passwords, ...)
`module_prepare`	prepare files before the containers start
`module_configure`	finish setup once the tool is up: admin password, tokens, repositories
`module_env`	declare the variables the pipeline needs
`module_stages`	add Maven stages to the pipeline
`module_destroy`	undo what `configure` did outside Docker
`module_render` , `module_publish` , `module_run`	orchestrators only: build, install and run the pipeline

Each hook runs in its own subshell, so modules cannot interfere with each other.

Stages come from the modules

Every module declares its stages with an order number and a phase (`ci`, `cd`, or a deployment environment such as `staging`). For example sonarqube says "stage 45 in phase ci: `sonar-maven-plugin:sonar`". The orchestrator takes the sorted list and turns it into its own format:

- **maven** runs every stage with `mvn` on your machine.
- **maven-container** writes a script that checks the commit out of GitHub and runs the stages with `mvn` in a container on the Docker machine.
- **jenkins** writes a Jenkinsfile and a `casc.yaml` that defines the admin user, the credentials and the job. No setup wizard and nothing to click.
- **concourse** writes a `pipeline.yml` with a `ci` job that runs on every push, a `cd` job you start by hand after `ci` passed, and a job per environment that needs approval.

So a new tool shows up in every orchestrator at once.

The project's pom.xml needs no profiles

`profiles.xml` only existed in Maven 2. Profiles in `settings.xml` only accept properties, repositories and activation, not plugins or `distributionManagement`.

So stages call every plugin by its full coordinates and configure it with `-D`, for example `maven-deploy-plugin:3.2.0:deploy -DaltSnapshotDeploymentRepository=nexus-snapshots::<url>`. Credentials come from the framework's own `templates/settings.xml`, passed with `-gs` (global settings). A project adds no profile, `distributionManagement` or settings to its pom. A project that has its own profiles or settings file can still use them through `MAVEN_PROFILES` and `MAVEN_SETTINGS`.

How it fits together

```
devops.sh ── lib/  (menu, value store, env files, stages, compose)
   │
   └── modules/<category>/<tool>/
         module.conf    title, description, requirements
         module.sh      hooks: module_secrets, module_prepare, module_configure,
                        module_env, module_stages, module_urls
                        (+ module_render, module_publish, module_run for orchestrators)
         compose.yml    containers of the tool (optional)
```

The selected tools and every answer that is not a secret or personal are kept in `<project>/devops.conf`, which you commit. Everything else is kept in `<project>/.devops/` (it git-ignores itself):

```
.devops/
  values/<KEY>          one file per value, chmod 600
  env/pipeline.env      variables handed to the pipeline (compose env_file format)
  env/pipeline.sh       the same as bash exports
  env/compose.env       everything, for ${VAR} substitution in compose files
  generated/            Jenkinsfile, jenkins/casc.yaml, concourse/*.yml, pipeline.sh, maven-
container/run.sh
  keys/                 generated SSH deploy key
```

Nothing is written to `~/.bashrc` or to system environment variables, and secrets are never printed: `env --show` masks them and the orchestrators bind them as masked credentials.

URLs handed to the pipeline depend on where it runs. A tool on an existing server is always reached at its own URL. A tool in Docker is reached at `DEVOPS_HOST:<port>` by the `maven` orchestrator and by an existing Jenkins or Concourse server, and at its compose service name (`http://sonarqube:9000`) by maven-container, and Jenkins or Concourse in Docker, which share its network. See Where the tools run.

Where the tools run

Every tool with a server (SonarQube, Nexus, Artifactory, Jenkins, Concourse) has two modes:

- **In Docker:** devops.sh starts it, on this machine or on the machine `DOCKER_HOST` points to. It is reached through `DEVOPS_HOST` .
- **Existing server:** anywhere, with its own URL. `secrets` asks for `<TOOL>_SERVER_URL` and credentials, no container is created, and `configure` only checks the credentials and repositories.

Each tool is decided on its own, so any mix works. Details in where-tools-run.md.

Life cycle

```
init  →  secrets  →  up      →  configure  →  publish          →  run
menu     questions   Docker      tokens        install pipeline    run it
```

`devops.sh setup` runs the first five steps in a row.

8.18 · mvn-devops/docs/module-guide.md

Writing a module

A module is a directory under `modules/<category>/<name>/`. The menu discovers it automatically; no other file has to change.

Files

`module.conf` (required)

```
MODULE_TITLE="Nexus Repository"
MODULE_DESCRIPTION="Sonatype Nexus 3 for Maven releases and snapshots"   # shown in the menu
MODULE_REQUIRES="scm/github"     # other modules that must be selected too (optional)
MODULE_RUNS_IN=docker            # orchestrators only: host or docker
MODULE_SERVER=NEXUS              # tools with a server: prefix of <PREFIX>_SERVER_URL (optional)
```

With `MODULE_SERVER`, the tool can also be an existing server: when `<PREFIX>_SERVER_URL` has a value, the module's `compose.yml` is left out and the hooks talk to that URL instead.

`compose.yml` (optional)

A docker compose fragment. The fragments of all selected modules are merged into one compose project (`devops-<project>`), so services reach each other by service name. Use named volumes for data. Every stored value is available for `${VAR}` substitution, plus `DEVOPS_HOME`, `DEVOPS_STATE`, `PROJECT_DIR` and `PROJECT_NAME`. Use them for paths, since relative paths are resolved against `.devops/`.

`module.sh` (optional)

Hook functions. Each hook runs in its own subshell with the library loaded, `MODULE_ID` and `MODULE_DIR` set and the module's `module.conf` sourced.

Hook	Called by	Purpose
`module_secrets`	`secrets`	ask for values with `ask` / `ask_secret`
`module_prepare`	`up`, before containers start	render config files, create keys
`module_configure`	`configure`, after containers start	admin passwords, tokens, repositories
`module_env`	every command	export pipeline variables with `pipeline_var` / `pipeline_secret`
`module_stages`	`stages`, `render`, `run`	contribute stages with `stage`
`module_urls`	`urls`, end of `setup`	print the web console and how to log in
`module_rollback`	`rollback`	put the previous release back (deployment modules)
`module_destroy`	`destroy`, before containers are removed	undo what `configure` did outside Docker (deploy keys, webhooks)
`module_render`	orchestrators: `render`, `publish`	write the pipeline definition
`module_publish`	orchestrators: `publish`	install the pipeline
`module_run`	orchestrators: `run`	run it

For orchestrators, `module_configure` runs during `publish` (after the tools are configured) instead of during `configure`.

Helpers

```
ask KEY "Question" [default]          # stored in .devops/values/KEY and shared through
devops.conf
ask_local KEY "Question" [default]    # same, but never shared: user names, addresses of this
machine
ask_secret KEY "Question" [default]   # hidden input, masked everywhere
value KEY [fallback]                  # read a value
require_value KEY                     # read or fail
set_value KEY VALUE [secret]          # store a computed value, e.g. a token
random_password                       # a random default password

pipeline_var KEY VALUE                # hand a variable to the pipeline
pipeline_secret KEY VALUE             # same, masked

stage ORDER PHASE NAME "MAVEN ARGS"   # PHASE is ci, cd or an environment ("prod": the last); args
may use $VARS
shell_stage ORDER PHASE NAME "CMD"    # a POSIX shell command in the project root instead of mvn
$ENVIRONMENTS                         # the deployment environments, in order, e.g. "staging
production"
upper NAME                            # STAGING: per-environment values are
<PREFIX>_<NAME>_<SETTING>
env_offset ENV                        # 0 for the last environment, 1 for the one before, ...
(default ports)
env_gates                             # the environments that wait for approval
rollback_args "$@"                    # parse [environment] [--to TAG] into ROLLBACK_ENV,
ROLLBACK_TAG
mvn_plugin KEY group:artifact VERSION GOAL   # full plugin coordinates, version overridable per
project
mvn_deploy_args SNAP_ID SNAP_URL REL_ID REL_URL   # package + attach + deploy without
distributionManagement
ask_server PREFIX "Title" [example]   # ask <PREFIX>_SERVER_URL; empty means Docker
server_external PREFIX                # true when an existing server is used
server_url PREFIX HOST_PORT [PATH]    # URL for configure hooks: the server, or DEVOPS_HOST:port
server_pipeline_url PREFIX SERVICE PORT HOST_PORT [PATH]   # URL for the pipeline
pipeline_url SERVICE PORT HOST_PORT [PATH]   # service name or DEVOPS_HOST, depending on the
orchestrator
host_url HOST_PORT [PATH]             # DEVOPS_HOST URL
github_url / github_host / github_api # github.com or GitHub Enterprise

image_secrets                         # image modules: name, module, builder, base image, port
image_env PUSH_REPO DEPLOY_REPO [USER] [PASSWORD]   # IMAGE_REPOSITORY and the rest for the
pipeline
image_stages INSECURE AUTH            # the image stage (Jib, or the project's Dockerfile)

compose ...                           # docker compose of the project, e.g. compose exec -T nexus
...
wait_http URL [timeout] [status regex]
log_step / log_ok / log_warn / log_dim / die / confirm
```

Call plugins by their coordinates and configure them with `-D` properties, so projects need no profiles (see project-requirements.md). A deploy target also needs a `<server>` with its id in `templates/settings.xml`.

Stage arguments are embedded in single-quoted strings by the orchestrators, so they may not contain `'`, `\`, `|` or `${`. Use `$VAR` instead of `${VAR}`.

A stage that needs more than one command calls a script from `templates/scripts/`: `shell_stage 80 cd deploy "sh \"\$DEVOPS_SCRIPTS/deploy.sh\""`. `DEVOPS_SCRIPTS` is that directory when the pipeline runs on this machine; maven-container, Jenkins and Concourse get a copy in `.devops/scripts` before the first stage. Scripts get every pipeline variable in the environment.

Example: a new artifact repository

```
modules/artifact/reposilite/
  module.conf
  compose.yml
  module.sh
```

```
## module.sh
## module.conf has MODULE_SERVER=REPOSILITE
module_secrets() {
  ask_server REPOSILITE Reposilite https://repo.example.com
  server_external REPOSILITE || ask REPOSILITE_HOST_PORT "Reposilite port on the Docker machine"
8085
  ask_secret REPOSILITE_TOKEN "Reposilite deploy token" "$(random_password)"
}

module_env() {
  pipeline_var REPOSILITE_URL "$(server_pipeline_url REPOSILITE reposilite 8080 "$(value
REPOSILITE_HOST_PORT 8085)")/snapshots"
  pipeline_secret REPOSILITE_TOKEN "$(value REPOSILITE_TOKEN)"
}

module_stages() {
  stage 73 cd deploy-reposilite "$(mvn_deploy_args reposilite '$REPOSILITE_URL' reposilite
'$REPOSILITE_URL')"
}
```

A new category is a directory with a `category.conf`:

```
CATEGORY_TITLE="Security scanning"
CATEGORY_ORDER=45          # position in the menu
CATEGORY_MODE=multi        # required | single (exactly one) | optional (at most one) | multi
```

Run `tests/smoke.sh` and `shellcheck` after adding a module.

8.19 · mvn-devops/docs/development.md

Development

Testing

```
shellcheck devops.sh lib/*.sh modules/*/*/module.sh tests/*.sh packaging/*.sh
tests/smoke.sh                       # no Docker needed: init, secrets, stages, render,
devops.conf, upgrade
tests/e2e.sh maven sonarqube,nexus   # real tools in Docker: setup, the whole pipeline, results in
the tools
tests/e2e.sh pipeline maven-docker-host            # a ready-made pipeline
E2E_ENVIRONMENTS="dev test staging prod" E2E_APPROVALS="staging prod" \
  tests/e2e.sh maven docker-registry,docker-host   # environments of your own
```

`tests/smoke.sh` runs on Linux, macOS and Windows (Git Bash) in the `ci` workflow. `tests/e2e.sh` builds examples/hello-maven, or examples/hello-api for the deployment modules, with each orchestrator in the `e2e` workflow, on every push to `main` and once a week; it serves the project from a local git server, so it needs no GitHub token. With `jfrog` it checks the setup only, because Artifactory OSS repositories have to be created in its Quick Setup wizard. Run it locally with Docker and internet access; `E2E_KEEP=1` leaves the containers running for a look.

Releasing mvn-devops

Every change worth telling users goes under `## Unreleased` in CHANGELOG.md. To release, rename that heading to the new version (`## 1.1.0`), set the same version in `VERSION` , commit, and push a tag, or start the release workflow by hand under Actions > release > Run workflow with the version. The workflow stops when CHANGELOG.md has no section for the version, and uses that section as the text of the GitHub release.

```
git tag v1.1.0 && git push origin v1.1.0
```

The release workflow runs the checks, builds the files with `packaging/build.sh` (zip, tar.gz, and deb and rpm with nfpm), installs the deb as a check and publishes everything as a GitHub release. `packaging/build.sh` also works locally and writes to `dist/` .

8.20 · mvn-devops/CHANGELOG.md

Changelog

Every release of mvn-devops. The section of a version is also the text of its GitHub release.

1.0.0

First release.

Setup

- A menu of modules per category: GitHub; Maven; SonarQube; Nexus, Artifactory OSS and GitHub Packages; GitHub Pages; container images; image security; deployment; GitOps; secrets; a database; monitoring; a load test.
- Orchestrators: `maven` (on this machine), `maven-container` (in a container on the Docker machine, from a fresh checkout of GitHub), Jenkins (configuration as code, no setup wizard) and Concourse.
- `setup` asks the questions, starts the tools in Docker, replaces their default passwords, creates tokens and repositories, and installs the pipeline. Every tool runs in Docker, on this machine or another one, or is an existing server with its own URL; GitHub Enterprise is supported.
- `devops.conf` in the project root holds the selected tools and every answer that is neither secret nor personal. Commit it: a teammate's `setup` asks only for their own tokens and user names. Passwords and keys stay in `.devops/`.
- 16 ready-made pipelines in `pipelines/`, from Maven with SonarQube and Nexus up to Jenkins with every kind of tool. `setup --pipeline <name or file>` sets one up and generates new passwords, tokens and keys for each project.
- No profiles, plugins or `distributionManagement` in the project's pom: plugins are called by their coordinates.

Pipeline

- Container images built with Jib (no Docker or Dockerfile needed) or with the project's Dockerfile, pushed to a registry in Docker, an existing one or the GitHub Container Registry.
- Image security: Trivy scans the image, Syft writes its SBOM, Cosign signs it and checks the signature before an approved environment gets it.
- Deployment environments of the project's own (`ENVIRONMENTS` , e.g. `dev test staging prod`), each with its own machine or namespace, port, secrets and database, and an approval where `ENV_<NAME>_APPROVAL=yes` . The same image goes through them in order.
- Deployment with Docker Compose over SSH, with Helm on Kubernetes (an existing cluster or k3s in Docker), or with Argo CD from a GitOps branch, the last environment as a canary with Argo Rollouts. Each deployment is checked and rolled back when it fails; `rollback` puts the previous image back.
- Secrets from Vault, passed to the application as environment variables, a Kubernetes Secret or a SealedSecret.
- PostgreSQL per environment, with the Flyway migrations checked against an empty database in ci.

- Prometheus, Grafana and Loki watch each environment; k6 load tests one of them after each deployment.
- `release` releases the project without maven-release-plugin.

Tools and packaging

- `doctor` checks the prerequisites; `doctor --fix` repairs CRLF line endings.
- `upgrade` replaces the copy of mvn-devops inside a project with a release, checked against `SHA256SUMS`.
- `export-compose` writes the selected tools as one docker-compose.yml.
- Packages: zip, tar.gz, deb and rpm. Runs on Linux, macOS and Windows (Git Bash).
- `docs/getting-started.md` has step-by-step guides; `examples/` has sample projects.
- Tested on every push: a smoke test on Linux, macOS and Windows, and an end-to-end test of every orchestrator and the ready-made pipelines with the real tools on GitHub's runners.